GymGramps

Documento de Projeto de Software

Fases A-1, A-2, B-3, B-4, C-5 e C-6 — Requisitos, Modelagem, Projeto de Dados, Projeto Arquitetural e Funcional, Esquema Físico Relacional (PostgreSQL) e Esquema Físico Não Relacional (MongoDB)

Integrantes: Willyam Eduardo de Souza Filho e Daniel Almeida Zuvanov

Projeto	GymGramps — Aplicativo Web de Rotina de Treino e Alimentação
Documento	Documento de Projeto de Software (entrega acumulada até a Fase C-6) — SGBDR: PostgreSQL 16; SGBD não relacional: MongoDB
Versão	6.0 — adiciona o Esquema Físico do Banco de Dados Não Relacional (Fase C-6: MongoDB, com validação $jsonSchema, índices, carga inicial e diagramas) e mantém o Esquema Físico Relacional (Fase C-5), o Projeto Arquitetural e Funcional (Fase B-4) e as correções das fases anteriores
Integrantes do Grupo	Willyam Eduardo de Souza Filho e Daniel Almeida Zuvanov
Status	Entrega da Fase C-6

Sumário

Registro de correções e novidades 3
Parte I — Requisitos (Fase A-1) 4
1. Introdução 4
2. Visão Geral do Produto 4
3. Atores e Envolvidos 5
4. Requisitos 5
5. Rastreabilidade — RN × RF × UC 6
Parte II — Modelagem (Fase A-2) 7
6. Modelo de Casos de Uso 7
6.1. Diagrama de Casos de Uso 7
6.2. Descrição Completa dos Casos de Uso 8
7. Modelo Conceitual — Diagrama de Classes (Análise) 12
7.1. Diagrama de Classes 12
7.2. Descrição Conceitual das Classes 14
7.3. Associações e Restrições 14
Parte III — Projeto de Dados (Fase B-3) 15
8. Esquema Lógico Relacional (notação pé-de-galinha) 15
8.1. Dicionário das Tabelas 17
9. Mapeamento do Modelo de Classes para o Modelo Relacional 17
10. Observações sobre o Esquema Relacional 18
Parte IV — Projeto Arquitetural e Funcional (Fase B-4) 20
11. Projeto Arquitetural (Lógico e Físico) 20
11.1. Visão geral e decisões arquiteturais 20
11.2. Visão lógica — Diagrama de Componentes 21
11.3. Visão física — Diagrama de Implantação 23
12. Projeto Funcional (Modelo de Interações) 25
12.1. Seleção dos casos de uso e dos diagramas 25
12.2. UC-01 — Criar Cadastro no Sistema (Diagrama de Atividades) 25
12.3. UC-05 — Enviar Mensagem no Chat (Diagrama de Sequência) 28
12.4. UC-06 — Excluir Conta Definitivamente (Diagrama de Estados) 30
Parte V — Esquema Físico do Banco de Dados (Fase C-5) 32
13. Esquema Físico do Banco de Dados Relacional 32
13.1. SGBDR, versão e ambiente 32
13.2. Do esquema lógico ao esquema físico 32
13.3. Regras de integridade e como são garantidas 33
13.4. Índices 34
13.5. Script 00 — criação do banco de dados 34
13.6. Script 01 — DDL do esquema físico 35
14. Carga Inicial do Banco de Dados (DML) 39
14.1. Estratégia e conteúdo da carga 39
14.2. Script 02 — DML da carga inicial 40
14.3. Resultado da carga 42
15. Testes e Verificação 42
15.1. Metodologia 42
15.2. Testes de integridade e resultado obtido 42
15.3. Consultas dos casos de uso e resultados 43
15.4. Script 03 — testes de integridade e de consultas 44

Sumário (continuação)

Parte VI — Banco de Dados Não Relacional (Fase C-6) **47**
16. Escolha do SGBD não relacional e do modelo de dados 47
16.1. Por que o MongoDB 47
16.2. Decisões de modelagem: embutir ou referenciar 47
17. Esquema Físico do Banco de Dados Não Relacional (definição de dados) 48
17.1. SGBD, versão e ambiente 48
17.2. Do esquema relacional ao esquema físico do MongoDB 48
17.3. Regras de integridade e onde são garantidas 48
17.4. Índices 49
17.5. Limitações e cuidados 50
17.6. Script 00 — criação do banco de dados e do papel da aplicação 50
17.7. Script 01 — esquema físico (coleções, validação, índices e visão) 50
17.8. Biblioteca da camada de aplicação — lib_gymgramps.js 52
18. Diagramas da organização dos dados 54
18.1. Organização física das coleções 54
18.2. Exemplo com documentos reais da carga 54
19. Carga Inicial do Banco de Dados (DML) 57
19.1. Estratégia e conteúdo da carga 57
19.2. Script 02 — DML da carga inicial 58
19.3. Resultado da carga 59
20. Testes e Verificação 59
20.1. Metodologia 59
20.2. Testes de integridade e resultado obtido 60
20.3. Consultas dos casos de uso e resultados 60
20.4. Script 03 — testes de integridade e de consultas 61
Apêndice A — Script DDL (PostgreSQL / Supabase) **63**
Referências **66**

Registro de correções e novidades

A tabela relaciona cada observação feita pelo professor nas entregas anteriores ao que foi alterado e à seção onde a correção pode ser conferida. Todas as correções abaixo foram mantidas nesta versão (6.0).

Observação do professor	O que foi feito	Onde
Mensagem: mapeamento dos relacionamentos incorreto; modelo conceitual incoerente com o lógico; a Mensagem deveria se relacionar com uma classe associativa do relacionamento Usuario–Conversa (29/09).	Criada a classe associativa **ParticipanteConversa** no relacionamento Usuario–Conversa. A Mensagem passou a se associar a ela (1 participante — 0..* mensagens). As associações diretas Conversa–Mensagem (composição) e Mensagem→Usuario foram substituídas por essa.	7.1, 7.2, 7.3
“Onde está o relacionamento entre Mensagem e Usuário?” (29/09).	Representado no diagrama como associação **derivada** /remetente (Mensagem → Usuario), obtida através de ParticipanteConversa; idem /conversa (Mensagem → Conversa). Explicado também no texto.	7.1, 7.3
Corrigir o modelo conceitual e o lógico (29/09).	Esquema lógico, dicionário, tabela de mapeamento, observações e script DDL atualizados: a tabela participante_conversa (classe associativa, agora com data_entrada) é referenciada pela FK composta de mensagem. DDL reexecutado em PostgreSQL 16 com casos de violação.	8, 9, 10, Apêndice A
Supabase não é ator: se for apenas repositório de dados/autenticação, não é ator (15/09 e 09/09).	Supabase removido da lista de atores, do diagrama de casos de uso e de todas as descrições de UC (os fluxos agora falam em “sistema”). Passa a ser tratado como infraestrutura (BaaS) nos diagramas de arquitetura.	3.1, 6.1, 6.2, 11
Diagrama de UC repetido; documento mal apresentado/estruturado (15/09).	Documento unificado: **um único** diagrama de UC e **uma única** descrição por UC (a versão curta da Fase A-1 foi fundida à completa); numeração contínua, divisão em partes por fase e sumário com páginas.	Estrutura geral, 6
Diagrama de classes muito pequeno; esperava tratamento da atividade física — treinos e rotina (15/09).	Diagrama de classes em página inteira (paisagem). A classe Atividade teve a descrição reforçada como o registro de treinos e rotina do usuário.	7.1, 7.2
Supabase usado como BD não conta para as restrições do trabalho (09/09).	O esquema relacional é PostgreSQL padrão (DDL do Apêndice A, que roda fora do Supabase exceto pela política RLS, que depende de auth.uid()). O Supabase é descrito apenas como hospedagem/infraestrutura. O esquema físico da Fase C-5 também roda em PostgreSQL puro.	8, 10.6, 11.3, 13.1

Novidades da Fase C-6

Item entregue	Seção	Script
Esquema físico não relacional (MongoDB): coleções, validadores, índices e visão	17.1 a 17.8	00_banco.js, 01_ddl.js
Diagramas da organização dos dados e exemplos de documentos reais da carga	18	Figuras 9 e 10
Carga inicial (DML) e testes de integridade e de consultas (27 testes)	19 e 20	02_carga.js, 03_testes.js

Novidades da Fase C-5

Item entregue	Seção	Script
Esquema físico — SGBDR (PostgreSQL 16.15), extensões, papéis e ambiente	13.1	—
Decisões do projeto físico, regras de integridade e índices	13.2 a 13.4	—
SQL:DDL — banco, tipos, tabelas, chaves compostas, índices, restrições, gatilhos, funções, visões e RLS	13.5, 13.6	00_banco.sql, 01_ddl.sql
SQL:DML — carga inicial (inserções e atualizações)	14	02_dml.sql
Testes de integridade, de consultas e de índices (25 testes, executados em PostgreSQL 16.15)	15	03_testes.sql

Novidades da Fase B-4

Item entregue	Seção	Figura
Projeto Arquitetural — visão lógica (Diagrama de Componentes)	11.2	Figura 4
Projeto Arquitetural — visão física (Diagrama de Implantação)	11.3	Figura 5
Projeto Funcional — Diagrama de Atividades (UC-01 Criar Cadastro)	12.2	Figura 6
Projeto Funcional — Diagrama de Sequência (UC-05 Enviar Mensagem)	12.3	Figura 7
Projeto Funcional — Diagrama de Estados (UC-06 Excluir Conta)	12.4	Figura 8

Parte I — Requisitos (Fase A-1)

1. Introdução

1.1. Objetivo e Visão Geral do Documento

Este documento apresenta o escopo, a especificação de requisitos, a modelagem e o projeto do aplicativo "GymGramps". Ele documenta as regras de negócio, os requisitos funcionais e não-funcionais, os casos de uso, o modelo conceitual de classes, o esquema lógico do banco de dados relacional, o projeto arquitetural (lógico e físico) e o projeto funcional (comportamento) do sistema. Os leitores-alvo englobam a equipe de desenvolvimento (para guiar a implementação da interface React e do acesso ao Supabase), o professor/orientador avaliador e outros envolvidos na concepção do sistema.

1.2. Termos Técnicos, Abreviações e Acrônimos

- **UI (User Interface):** Interface de Usuário.
- **Design Brutalista:** estilo de design implementado no projeto, focado em altíssima legibilidade, fontes grandes e navegação simplificada, eliminando distrações visuais.
- **RN-XX:** Regra de Negócio, identificada por número (ex.: RN-01). **RF-XX:** Requisito Funcional (ex.: RF-03). **RNF-XX:** Requisito Não-Funcional (ex.: RNF-02). **UC-XX:** Caso de Uso (Use Case), identificado por número (ex.: UC-04).
- **LGPD:** Lei Geral de Proteção de Dados (Lei nº 13.709/2018), legislação brasileira que rege o tratamento de dados pessoais.
- **SPA (Single-Page Application):** aplicação web que carrega uma única página e troca as telas no navegador, sem recarregar.
- **BaaS (Backend-as-a-Service) e Supabase:** o Supabase é a plataforma BaaS usada como *infraestrutura* do projeto (autenticação e PostgreSQL gerenciado). Não é ator do sistema (Seção 3).
- **RLS (Row Level Security):** recurso do PostgreSQL que restringe, por política, quais linhas cada usuário enxerga ou altera.
- **CN:** chave natural de uma classe (atributos que identificam um objeto no domínio).

2. Visão Geral do Produto

- **Cliente ou Público-Alvo:** o sistema atende praticantes de atividades físicas de qualquer idade, com foco principal em jovens e adultos (18 a 30 anos) que buscam organizar o acompanhamento de suas rotinas de treino e saúde com agilidade.
- **Dores ou Problemas a Serem Resolvidos:** aplicativos de academia convencionais costumam ser genéricos, bloqueados por paywalls comerciais excessivos, ou poluídos visualmente, desmotivando o público jovem que deseja uma ferramenta rápida e direta.
- **Benefícios:** o GymGramps oferece alta performance e praticidade, permitindo gerenciar tanto a agenda de treinos quanto a alimentação de forma crua, ágil e focada.
- **Macro-funcionalidades:** Autenticação e Gestão de Perfil; Dashboard Visual de Rotina (formato 24h); Módulo de Alimentação; Módulo de Chat.
- **Diferenciais:** estética Brutalist UI, priorizando velocidade e contraste; visualização da rotina em formato de "fita contínua" de 24h.
- **Fora do Escopo:** nesta etapa, não haverá pagamentos/assinaturas, integração com smartwatches, ou planos de treino gerados por IA avançada.

3. Atores e Envolvidos

Esta seção separa quem interage diretamente com o sistema (Atores) de quem é interessado no projeto sem interagir com o sistema em produção (Envolvidos/Stakeholders).

3.1. Atores (interagem diretamente com o sistema)

- **Visitante (Não-Autenticado) — Ator:** pessoa sem sessão ativa: ainda sem cadastro ou já cadastrada que ainda não fez login. Interage com os fluxos de criação de conta (UC-01) e de login (UC-02).
- **Usuário Autenticado (Praticante de Atividade Física) — Ator:** pessoa com conta ativa e sessão iniciada, que interage com rotina, alimentação, chat e perfil (UC-03 a UC-06). Em UC-05, o usuário que *recebe* a mensagem também é um Usuário Autenticado e atua como ator secundário.

> **Observação sobre o Supabase.** O Supabase **não é ator**: é a infraestrutura (BaaS) acionada internamente pelo próprio sistema para autenticar usuários e armazenar dados, isto é, um componente de autenticação e um repositório de dados. Por isso ele não aparece no diagrama de casos de uso nem nas descrições dos casos de uso, e sim como nó/componente nos diagramas de arquitetura (Seção 11).

3.2. Envolvidos (não interagem diretamente com o sistema)

- **Equipe de Desenvolvimento — Envolvido:** responsável pela implementação técnica; não é usuária do sistema em produção.
- **Professor/Orientador Avaliador — Envolvido:** avalia o escopo e a qualidade da especificação e do produto entregue; não interage com o sistema como usuário.

4. Requisitos

4.1. Regras de Negócio (RN)

As regras abaixo descrevem restrições e políticas do negócio GymGramps que existiriam independentemente de haver ou não um sistema de software apoiando-as — refletem obrigações legais e políticas de funcionamento, não comportamentos do software em si.

- **RN-01:** Apenas pessoas com cadastro/matrícula ativa podem receber acompanhamento de treino, orientação alimentar e comunicar-se com outros membros da comunidade GymGramps. *Impacto: RF-01, RF-02, RF-03, RF-04, RF-05.*
- **RN-02:** Dados de saúde e desempenho físico do usuário (peso, histórico de treino, informações alimentares) são considerados dados sensíveis pela LGPD e devem ser protegidos por criptografia e controle de acesso restrito. *Impacto: RF-04, RNF-03.*
- **RN-03:** Conforme a LGPD, o titular dos dados tem o direito de solicitar a exclusão completa e definitiva de seus dados pessoais armazenados. *Impacto: RF-06.*

4.2. Requisitos Funcionais (RF)

- **RF-01 (Cadastro):** o sistema deve permitir que novos usuários preencham um formulário de onboarding para a criação de um perfil ativo.
- **RF-02 (Login):** o sistema deve prover tela de autenticação para validar e-mail e senha de usuários já cadastrados.
- **RF-03 (Painel de Rotina):** o sistema deve exibir um painel principal onde o usuário acompanha seu cronograma de exercícios em uma linha do tempo diária (formato "fita" de 24 horas).
- **RF-04 (Alimentação):** o sistema deve possuir uma página destinada a exibir orientações, recordatórios ou o plano alimentar predefinido vinculado ao perfil do usuário.
- **RF-05 (Chat):** o sistema deve fornecer um espaço de chat (mensageria) para envio e recebimento de mensagens entre usuários autenticados.

- **RF-06 (Perfil):** o sistema deve permitir o acesso a dados da conta, customização (temas), opções de segurança e o acionamento da exclusão de conta.

4.3. Requisitos Não-Funcionais (RNF)

- **RNF-01 (Usabilidade):** o sistema deve seguir o design brutalista (modos claro/escuro marcantes, blocos chamativos), permitindo que um novo usuário conclua o onboarding sem tutorial.
- **RNF-02 (Performance):** o sistema deve ser uma Web App responsiva com React (via Vite), com troca de telas internas em até 1 segundo em condições normais de rede.
- **RNF-03 (Segurança):** a senha do usuário deve ser armazenada de forma criptografada (hash) via Supabase; dados sensíveis descritos em RN-02 devem ser encriptados antes de serem salvos.
- **RNF-04 (Compatibilidade):** o sistema deve ser funcional nas versões mais recentes de Chrome, Firefox e Edge, em resoluções desktop e mobile.
- **RNF-05 (Qualidade de Dados):** os dados inseridos pelo usuário (e-mail, senha, campos do onboarding e do perfil) devem ser validados quanto a formato e completude antes de serem persistidos, garantindo consistência dos dados armazenados.

5. Rastreabilidade — RN × RF × UC

A tabela relaciona cada requisito funcional à(s) regra(s) de negócio que o originam e ao(s) caso(s) de uso que o detalha(m), evidenciando que toda RN impacta ao menos uma funcionalidade do sistema.

Requisito	Regra(s) de Negócio	Caso(s) de Uso
RF-01	RN-01	UC-01 — Criar Cadastro no Sistema
RF-02	RN-01	UC-02 — Acessar Plataforma (Login)
RF-03	RN-01	UC-03 — Visualizar Cronograma de Rotina
RF-04	RN-01, RN-02	UC-04 — Acessar Dados de Alimentação
RF-05	RN-01	UC-05 — Enviar Mensagem no Chat
RF-06	RN-03	UC-02, UC-06 — Perfil e Exclusão de Conta

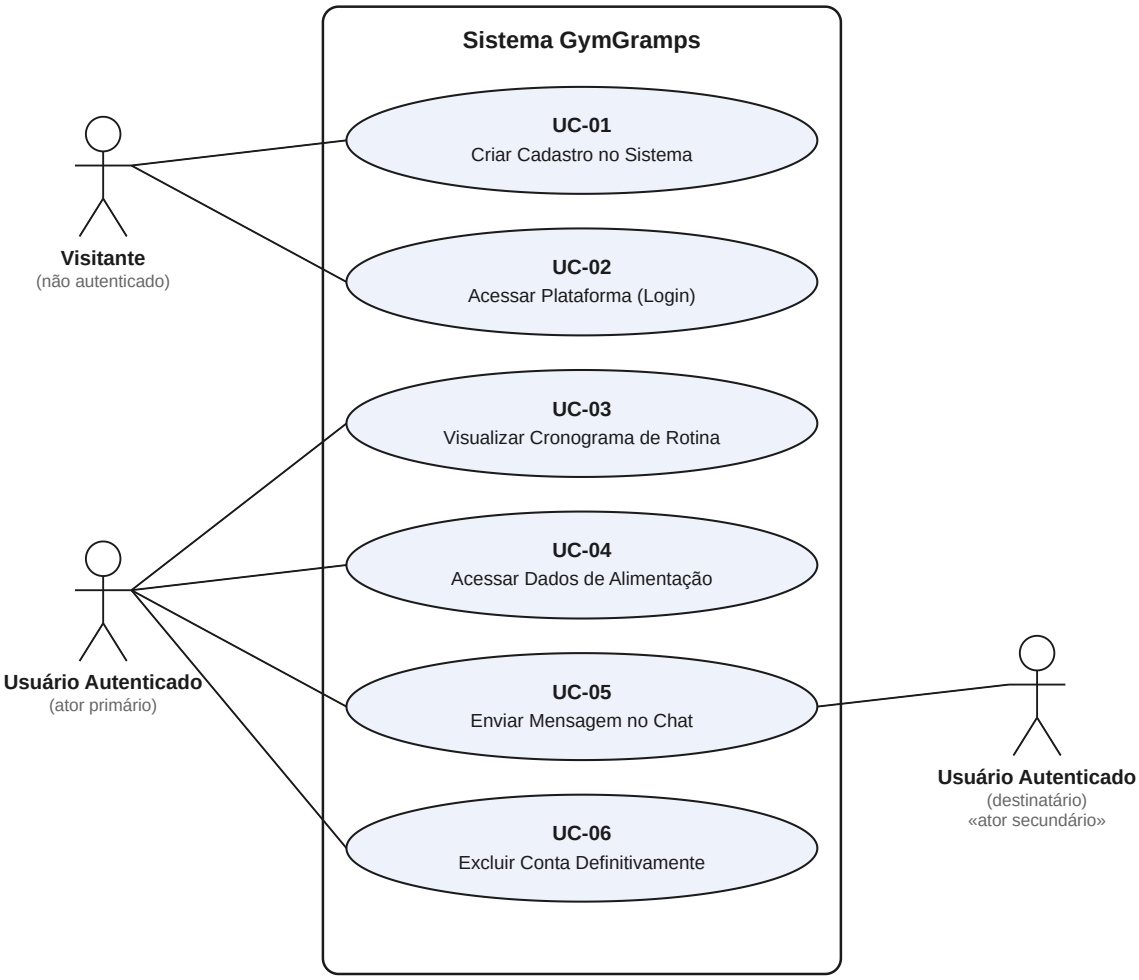
Parte II — Modelagem (Fase A-2)

6. Modelo de Casos de Uso

6.1. Diagrama de Casos de Uso

O diagrama abaixo é o **único** diagrama de casos de uso do documento. Os atores primários ficam à esquerda; o ator secundário (o destinatário em UC-05) fica à direita. O Supabase não aparece como ator (ver Seção 3.1). O Visitante interage com UC-01 e UC-02; o Usuário Autenticado, com UC-03 a UC-06.

Figura 1 — Diagrama de Casos de Uso do GymGramps



Linha contínua: associação ator–caso de uso. Atores primários à esquerda; ator secundário à direita.

Resumo dos casos de uso

ID	Título	Ator primário	Ator secundário	Tipo
UC-01	Criar Cadastro no Sistema	Visitante	—	Secundário
UC-02	Acessar Plataforma (Login)	Visitante (cadastrado)	—	Secundário
UC-03	Visualizar Cronograma de Rotina	Usuário Autenticado	—	Primário
UC-04	Acessar Dados de Alimentação	Usuário Autenticado	—	Primário
UC-05	Enviar Mensagem no Chat	Usuário Autenticado (remetente)	Usuário Autenticado (destinatário)	Primário
UC-06	Excluir Conta Definitivamente	Usuário Autenticado	—	Primário

6.2. Descrição Completa dos Casos de Uso

Classificação: **primário** = atende diretamente ao objetivo de negócio do ator; **secundário** = caso de uso de suporte, necessário para viabilizar os primários.

Identificador	**UC-01**
Título	Criar Cadastro no Sistema
Tipo do UC	Secundário (suporte: habilita o acesso aos UCs primários)
Atores	**Primário:** Visitante (não autenticado) **Secundário:** — (nenhum)
Objetivo	Permitir que um visitante crie um perfil ativo na plataforma preenchendo o formulário de onboarding.
Pré-Condições	O visitante acessa a tela inicial do sistema e ainda não possui conta ativa.
Pós-Condições	Um novo Usuario é criado e persistido com statusConta = ATIVA e sessão autenticada iniciada.
Trigger	O visitante seleciona a opção "Criar Conta".
Fluxo Básico	1. O visitante seleciona a opção "Criar Conta". 2. O sistema exibe o formulário de onboarding (nome, data de nascimento, peso, e-mail e senha). 3. O visitante preenche os campos e confirma o envio. 4. O sistema valida formato e completude dos dados (RNF-05). 5. O sistema aplica hash à senha, cifra os dados sensíveis (RNF-03) e solicita a criação do perfil. 6. O sistema cria o Usuario (statusConta = ATIVA), persiste o perfil e inicia a sessão autenticada. 7. O sistema redireciona o usuário ao painel de Rotina (UC-03).
Fluxos Alternativos	**A1 – Cancelar cadastro (passo 3):** o visitante clica em "Voltar"; o sistema descarta os dados e retorna à tela inicial.
Fluxos de Exceção	**E1 – Dado inválido ou ausente (passo 4):** o sistema impede o envio, destaca o campo pendente e retorna ao passo 3. **E2 – E-mail já cadastrado (passo 6):** o sistema exibe mensagem de erro e mantém o visitante na tela de cadastro. **E3 – Falha de comunicação (passo 5):** o sistema informa indisponibilidade e permite nova tentativa.
Referências Cruzadas	RF-01; RN-01; RN-02; RNF-01; RNF-03; RNF-05

Identificador	UC-02
Título	Acessar Plataforma (Realizar Login)
Tipo do UC	Secundário (suporte/controle de acesso)
Atores	**Primário:** Visitante (já cadastrado, sem sessão ativa) **Secundário:** — (nenhum)
Objetivo	Autenticar o usuário de forma segura para liberar acesso aos módulos de Rotina, Alimentação, Chat e Perfil.
Pré-Condições	O visitante possui cadastro com statusConta = ATIVA e não está com sessão ativa.
Pós-Condições	Sessão autenticada iniciada; a pessoa passa a atuar como Usuário Autenticado, com acesso aos módulos do sistema.
Trigger	O visitante acessa a tela de login.
Fluxo Básico	1. O visitante acessa a tela de login. 2. O visitante informa e-mail e senha e confirma. 3. O sistema valida o formato dos campos (RNF-05). 4. O sistema submete as credenciais à verificação de autenticação. 5. O sistema verifica as credenciais (a senha é conferida em hash — RNF-03). 6. O sistema inicia a sessão e redireciona o usuário ao painel de Rotina (UC-03).
Fluxos Alternativos	**A1 – Visitante sem conta (passo 1):** o visitante seleciona "Criar Conta" e o sistema inicia o UC-01.
Fluxos de Exceção	**E1 – Credenciais inválidas (passo 5):** o sistema exibe mensagem genérica de erro e retorna ao passo 2. **E2 – Campos vazios/formato inválido (passo 3):** o sistema sinaliza o campo e retorna ao passo 2. **E3 – Serviço de autenticação indisponível (passo 4):** o sistema informa falha de conexão e permite nova tentativa.
Referências Cruzadas	RF-02; RF-06; RN-01; RNF-03; RNF-05

Identificador	UC-03
Título	Visualizar Cronograma de Rotina
Tipo do UC	Primário
Atores	**Primário:** Usuário Autenticado **Secundário:** — (nenhum)
Objetivo	Exibir as atividades/treinos do dia do usuário em uma linha do tempo contínua de 24 horas.
Pré-Condições	O usuário está autenticado (UC-02).
Pós-Condições	O usuário visualiza seu cronograma diário atualizado.
Trigger	O usuário acessa a aba "Rotina" (ou é redirecionado após login/cadastro).
Fluxo Básico	1. O usuário acessa a aba "Rotina" no menu principal. 2. O sistema consulta as Atividades do usuário para a data atual. 3. O sistema ordena as atividades por horaInicio. 4. O sistema exibe as atividades na fita de 24 horas, destacando o horário atual.
Fluxos Alternativos	**A1 – Consultar outro dia (passo 4):** o usuário seleciona outra data; o sistema retorna ao passo 2 com a data escolhida.
Fluxos de Exceção	**E1 – Nenhuma atividade no dia (passo 2):** o sistema exibe a fita vazia com mensagem de ausência de treinos. **E2 – Falha na consulta (passo 2):** o sistema exibe mensagem de erro e opção "Tentar novamente". **E3 – Sessão expirada:** o sistema redireciona para o UC-02.
Referências Cruzadas	RF-03; RN-01; RNF-01; RNF-02

Identificador	UC-04
Título	Acessar Dados de Alimentação
Tipo do UC	Primário
Atores	**Primário:** Usuário Autenticado **Secundário:** — (nenhum)
Objetivo	Permitir a leitura do plano alimentar, macros e refeições vinculados ao perfil do usuário.
Pré-Condições	O usuário está autenticado (UC-02).
Pós-Condições	O usuário visualiza suas informações alimentares atualizadas; nenhum dado sensível é exposto a terceiros.
Trigger	O usuário acessa a aba "Alimentação".
Fluxo Básico	1. O usuário acessa a aba "Alimentação" no menu principal. 2. O sistema consulta o PlanoAlimentar vigente do usuário e suas Refeições (acesso restrito ao titular – RN-02). 3. O sistema decifra os dados sensíveis. 4. O sistema exibe calorias diárias, macronutrientes e a lista de refeições por horário.
Fluxos Alternativos	**A1 – Ver detalhe de refeição (passo 4):** o usuário seleciona uma refeição e o sistema exibe sua descrição e calorias.
Fluxos de Exceção	**E1 – Sem plano definido (passo 2):** o sistema exibe orientações padrão e mensagem informativa. **E2 – Falha na consulta (passo 2):** o sistema exibe erro e opção de nova tentativa.
Referências Cruzadas	RF-04; RN-01; RN-02; RNF-03

Identificador	UC-05
Título	Enviar Mensagem no Chat
Tipo do UC	Primário
Atores	**Primário:** Usuário Autenticado (remetente) **Secundário:** Usuário Autenticado (destinatário)
Objetivo	Promover comunicação, tirada de dúvidas e compartilhamento entre membros autenticados.
Pré-Condições	O remetente está autenticado e existe ao menos outro usuário ativo disponível ou conversa existente.
Pós-Condições	A Mensagem é persistida com status ENVIADA, vinculada à participação do remetente na conversa (ParticipanteConversa), e disponibilizada ao destinatário.
Trigger	O usuário acessa a aba "Chat".
Fluxo Básico	1. O usuário acessa a aba "Chat". 2. O sistema lista as conversas do usuário. 3. O usuário seleciona uma conversa. 4. O sistema exibe o histórico de mensagens. 5. O usuário digita a mensagem e confirma o envio. 6. O sistema valida que o conteúdo não está vazio. 7. O sistema registra a Mensagem, vinculada à participação do remetente na conversa. 8. O sistema disponibiliza a mensagem ao destinatário e a exibe como enviada.
Fluxos Alternativos	**A1 – Nova conversa (passo 3):** o usuário escolhe "Nova conversa" e seleciona um usuário ativo; o sistema cria a Conversa e as duas participações (ParticipanteConversa) na mesma transação e segue ao passo 5.
Fluxos de Exceção	**E1 – Mensagem vazia (passo 6):** o botão de envio permanece desabilitado. **E2 – Falha de conexão (passo 7):** o sistema marca a mensagem como NAO_ENVIADA e permite reenvio. **E3 – Destinatário excluiu a conta:** o sistema informa que a conversa não está mais disponível.
Referências Cruzadas	RF-05; RN-01; RNF-02

Identificador	UC-06
Título	Excluir Conta Definitivamente
Tipo do UC	Primário
Atores	**Primário:** Usuário Autenticado **Secundário:** — (nenhum)
Objetivo	Garantir ao titular o direito de exclusão completa e definitiva de seus dados (LGPD).
Pré-Condições	O usuário está autenticado e acessa a área Perfil/Segurança.
Pós-Condições	Usuario e todos os objetos associados (Atividades, PlanoAlimentar, Refeições, Conversas, ParticipanteConversa e Mensagens) são removidos permanentemente; sessão encerrada.
Trigger	O usuário seleciona "Excluir Conta" na área de Perfil.
Fluxo Básico	1. O usuário seleciona a opção "Excluir Conta". 2. O sistema alerta sobre a irreversibilidade e solicita confirmação com a senha. 3. O usuário informa a senha e confirma. 4. O sistema valida a senha informada. 5. O sistema aciona a exclusão em cascata dos dados do usuário. 6. O sistema encerra a sessão e exibe confirmação na tela inicial.
Fluxos Alternativos	**A1 – Cancelar (passo 3):** o usuário cancela; o sistema mantém a conta ativa e retorna à tela de Perfil.
Fluxos de Exceção	**E1 – Senha incorreta (passo 4):** o sistema exibe erro e retorna ao passo 2. **E2 – Falha na exclusão (passo 5):** o sistema não altera os dados, informa o erro e permite nova tentativa.
Referências Cruzadas	RF-06; RN-03; RN-02; RNF-03

7. Modelo Conceitual — Diagrama de Classes (Análise)

7.1. Diagrama de Classes

O diagrama de classes de análise do GymGramps aparece em página inteira (Figura 2), com chaves naturais {CN}, associações navegáveis, papéis, multiplicidades e notas de restrição.

Correção aplicada (Mensagem). Na versão anterior a Mensagem se ligava diretamente à Conversa (composição) e ao Usuario (remetente), enquanto o modelo lógico a ligava a uma tabela associativa que não existia no modelo conceitual. Agora o relacionamento Usuario–Conversa tem a classe associativa **ParticipanteConversa**, e a **Mensagem se associa a ela** (1 participação — 0..* mensagens, papel *remetente*). O relacionamento entre Mensagem e Usuario continua existindo, como associação **derivada** */remetente* (Mensagem → ParticipanteConversa → Usuario), assim como */conversa* (Mensagem → ParticipanteConversa → Conversa). Com isso o modelo conceitual e o lógico passam a ser coerentes entre si e a regra “o remetente é participante da conversa” fica garantida pela própria estrutura do modelo.

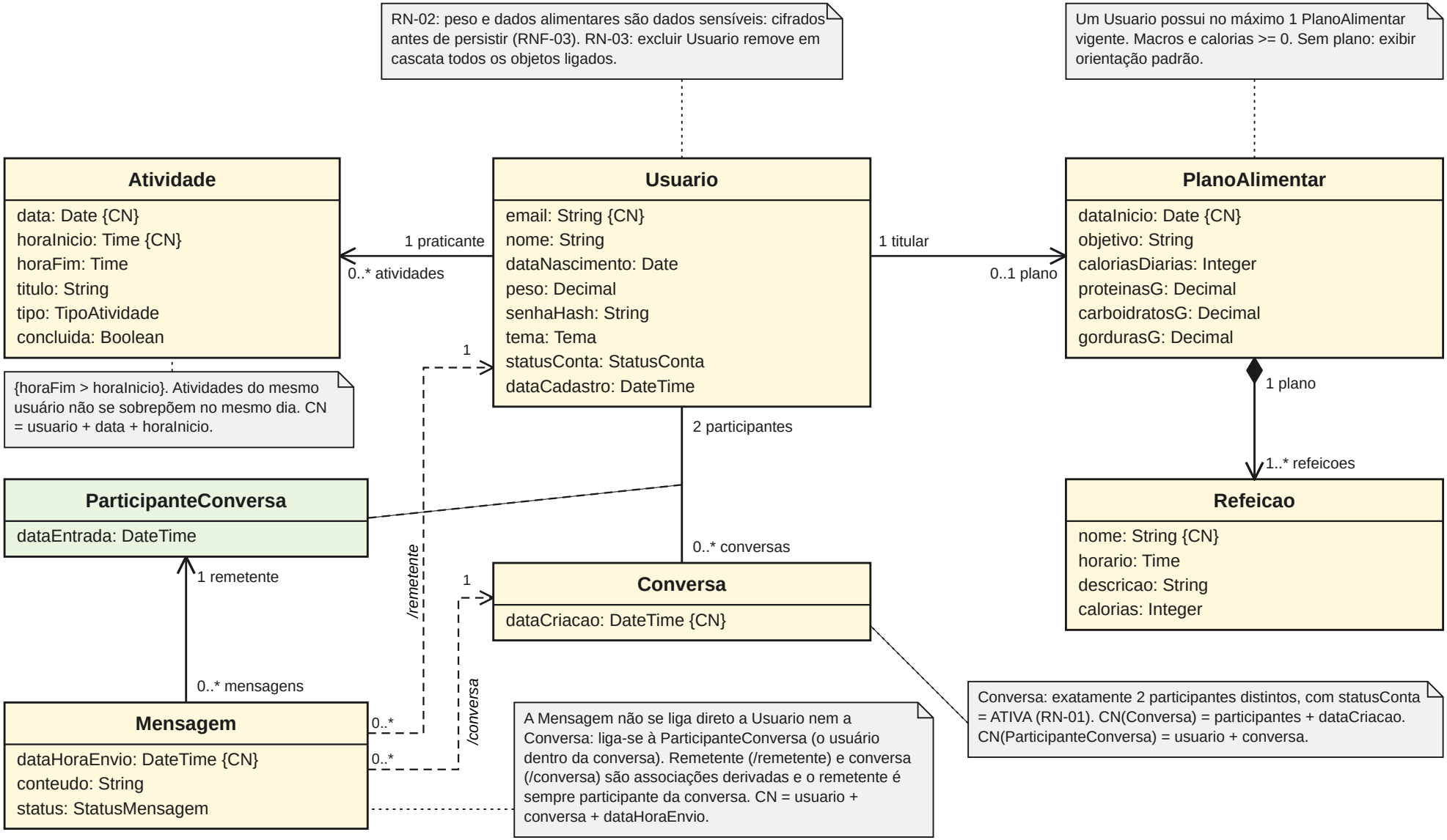


Figura 2 — Diagrama de Classes do GymGramps (modelo conceitual de análise).

7.2. Descrição Conceitual das Classes

Classe	Descrição e restrições
Usuario	Pessoa com conta no GymGramps (praticante de atividade física). Chave natural: **email** (único). Tema ∈ {CLARO, ESCURO}; StatusConta ∈ {ATIVA, INATIVA}. Apenas contas ATIVAS acessam rotina, alimentação e chat (RN-01). Peso é dado sensível (RN-02).
Atividade	Treino ou atividade física agendada na fita de 24h de um usuário; é o registro dos **treinos e da rotina** do praticante. Chave natural: **usuario + data + horaInicio**. TipoAtividade ∈ {MUSCULACAO, CARDIO, ALONGAMENTO, OUTRO}. Restrições: horaFim > horaInicio e sem sobreposição no mesmo dia.
PlanoAlimentar	Plano alimentar predefinido do usuário, com metas diárias de calorias e macronutrientes. Chave natural: **usuario + dataInicio**. Cada usuário tem no máximo um plano vigente; valores devem ser >= 0. Dado sensível (RN-02).
Refeicao	Refeição que compõe um plano alimentar (composição: não existe sem o plano). Chave natural: **plano + nome** (ex.: "Café da manhã").
Conversa	Canal de chat entre exatamente dois usuários ativos e distintos. Chave natural: **participantes (par de usuários) + dataCriacao**.
ParticipanteConversa *(classe associativa)*	Classe associativa do relacionamento Usuario–Conversa: representa **o usuário dentro de uma conversa** (a participação). Atributo: dataEntrada. Chave natural: **usuario + conversa**. É a ela que a Mensagem se associa.
Mensagem	Mensagem enviada por um participante dentro de uma conversa. Chave natural: **participante (usuario + conversa) + dataHoraEnvio**. StatusMensagem ∈ {ENVIADA, NAO_ENVIADA}. O remetente e a conversa são determinados pela ParticipanteConversa à qual a mensagem se associa.

7.3. Associações e Restrições

- **Usuario (praticante, 1) → Atividade (atividades, 0..*):** navegável do usuário para suas atividades.
- **Usuario (titular, 1) → PlanoAlimentar (plano, 0..1):** navegável do usuário para seu plano vigente.
- **PlanoAlimentar (plano, 1) ◆ → Refeicao (refeicoes, 1..*):** composição; excluir o plano exclui suas refeições.
- **Usuario (participantes, 2) ↔ Conversa (conversas, 0..*), com a classe associativa ParticipanteConversa:** relacionamento muitos-para-muitos; cada Conversa tem exatamente 2 usuários e cada Usuario pode estar em várias conversas. Cada par (usuário, conversa) é uma ParticipanteConversa.
- **ParticipanteConversa (remetente, 1) ← Mensagem (mensagens, 0..*):** navegável da mensagem para a participação do remetente; uma participação pode ter várias mensagens, e toda mensagem pertence a exatamente uma participação.
- **Mensagem → Usuario (/remetente, 1) e Mensagem → Conversa (/conversa, 1):** associações *derivadas* (não são armazenadas à parte): msg.remetente = msg.participante.usuario e msg.conversa = msg.participante.conversa.
- **Restrições gerais (notas):** dados sensíveis cifrados (RN-02/RNF-03); exclusão de Usuario remove em cascata todos os objetos associados (RN-03/UC-06); somente usuários com statusConta = ATIVA participam de conversas (RN-01).

Parte III — Projeto de Dados (Fase B-3)

8. Esquema Lógico Relacional (notação pé-de-galinha)

O esquema foi obtido aplicando ao diagrama de classes de análise (Seção 7) as regras de mapeamento de objetos para o modelo relacional (Bezerra, cap. 12) e de mapeamento ER (Aula 17). O SGBD-alvo é o PostgreSQL (versão 16 nos testes); por isso os tipos de dados do diagrama já seguem esse SGBD. O script DDL correspondente, executado e testado em PostgreSQL 16, está no Apêndice A. O esquema é PostgreSQL padrão; o Supabase entra apenas como hospedagem/infraestrutura (Seção 11).

Correção aplicada (Mensagem). A classe associativa ParticipanteConversa do modelo conceitual (Seção 7) corresponde à tabela *participante_conversa*, e a tabela *mensagem* aponta para ela por uma chave estrangeira composta (id_conversa, id_remetente). A tabela ganhou a coluna *data_entrada*, atributo da classe associativa.

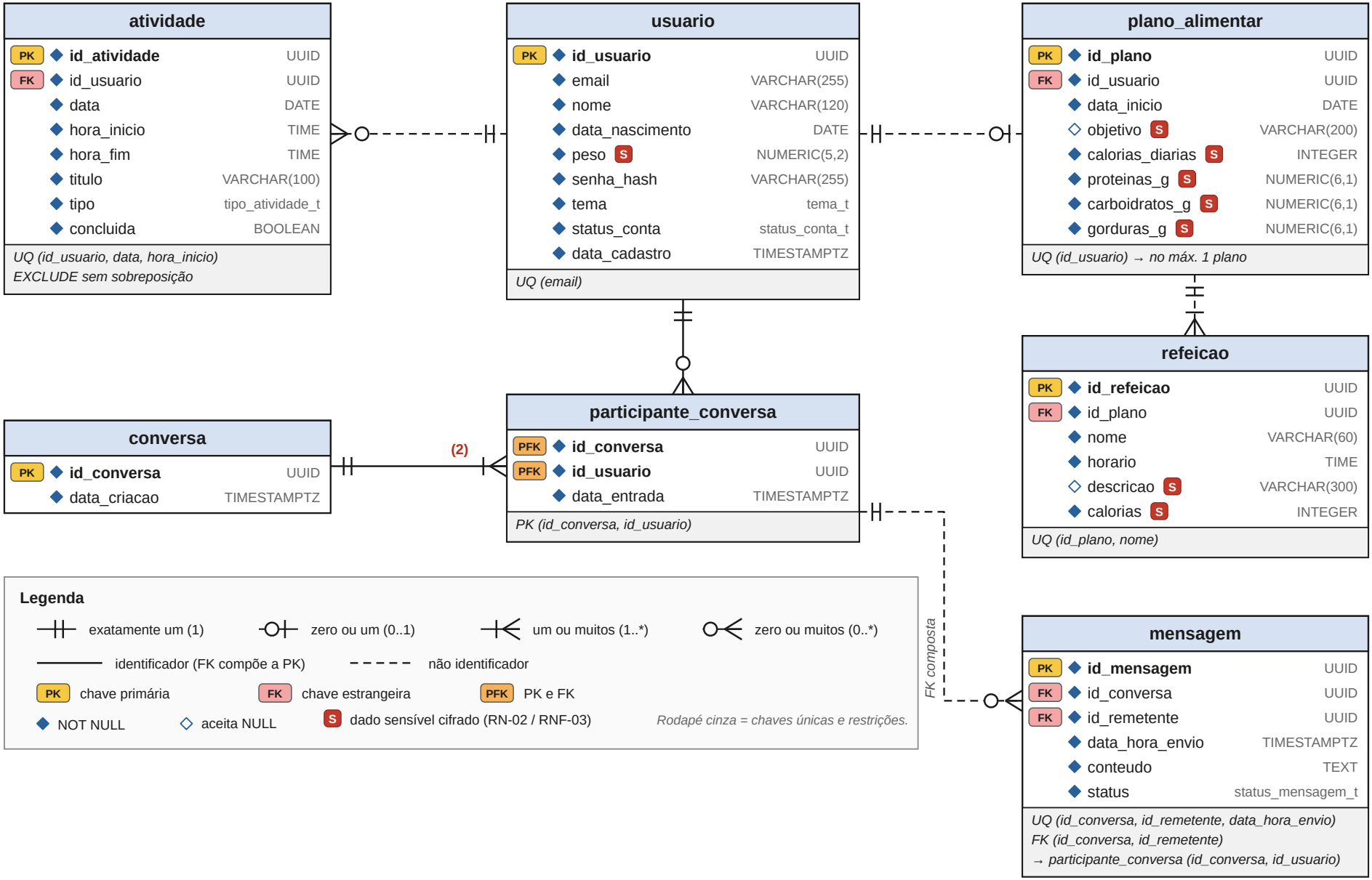


Figura 3 — Esquema lógico relacional do GymGramps (7 tabelas) em notação pé-de-galinha. Chaves primárias, estrangeiras e únicas, obrigatoriedade das colunas e marcação dos dados sensíveis (S).

8.1. Dicionário das Tabelas

Tabela	Chave primária	Chaves estrangeiras (ação na exclusão)	Chaves únicas e restrições
usuario	id_usuario	—	UQ (email). tema ∈ {CLARO, ESCURO}; status_conta ∈ {ATIVA, INATIVA}; peso > 0.
atividade	id_atividade	id_usuario → usuario (CASCADE)	UQ (id_usuario, data, hora_inicio); CHECK (hora_fim > hora_inicio); EXCLUDE: atividades do mesmo usuário não se sobrepõem; tipo ∈ {MUSCULACAO, CARDIO, ALONGAMENTO, OUTRO}.
plano_alimentar	id_plano	id_usuario → usuario (CASCADE)	UQ (id_usuario): no máximo 1 plano por usuário (implica a CN usuario + data_inicio); calorias e macros >= 0; ao menos 1 refeição (gatilho).
refeicao	id_refeicao	id_plano → plano_alimentar (CASCADE)	UQ (id_plano, nome); calorias >= 0.
conversa	id_conversa	—	Exatamente 2 participantes (gatilho verificado no COMMIT). A CN (participantes + data_criacao) fica apenas documentada (ver 10.1).
participante_conversa *(classe associativa)*	(id_conversa, id_usuario)	id_conversa → conversa (CASCADE); id_usuario → usuario (CASCADE)	A PK garante participantes distintos e a CN usuario + conversa; data_entrada NOT NULL; gatilho: no máximo 2 por conversa e somente contas ATIVAS (RN-01).
mensagem	id_mensagem	(id_conversa, id_remetente) → participante_conversa (CASCADE)	UQ (id_conversa, id_remetente, data_hora_envio); conteúdo não vazio; status ∈ {ENVIADA, NAO_ENVIADA}.

9. Mapeamento do Modelo de Classes para o Modelo Relacional

A tabela abaixo registra, para cada elemento do diagrama de classes (Figura 2), a regra de mapeamento aplicada e o resultado no esquema da Figura 3.

Elemento do diagrama de classes	Regra de mapeamento aplicada	Resultado no esquema
Classes Usuario, Atividade, PlanoAlimentar, Refeicao, Conversa, Mensagem e a classe associativa ParticipanteConversa	Cada classe persistente gera uma tabela; cada atributo simples gera uma coluna (nomes em snake_case).	7 tabelas homônimas
Identidade dos objetos e chaves naturais {CN}	Chave primária substituta (identificador de objeto, OID) do tipo UUID; a chave natural {CN} de cada classe vira restrição UNIQUE.	Colunas id_* (PK) + UQ
Enumerações Tema, StatusConta, TipoAtividade, StatusMensagem	Tipo enumerado do PostgreSQL (CREATE TYPE ... AS ENUM), que restringe o domínio da coluna.	tema_t, status_conta_t, tipo_atividade_t, status_mensagem_t
Usuario (1) — Atividade (0..*)	Associação 1:N: FK NOT NULL no lado "muitos".	atividade.id_usuario
Usuario (1) — PlanoAlimentar (0..1)	Associação 1:1 opcional: FK no lado opcional, com UNIQUE para limitar a um plano.	plano_alimentar.id_usuario (FK + UQ)
PlanoAlimentar (1) ◆ Refeicao (1..*)	Composição: FK NOT NULL na parte, com ON DELETE CASCADE (a parte não existe sem o todo).	refeicao.id_plano

Elemento do diagrama de classes	Regra de mapeamento aplicada	Resultado no esquema
Usuario (2) ↔ Conversa (0..*) com a classe associativa ParticipanteConversa	Associação N:M com classe associativa: vira a tabela da classe associativa, cuja PK é formada pelas FKs das duas classes associadas (relacionamento identificador); os atributos da classe associativa viram colunas.	participante_conversa (id_conversa, id_usuario, data_entrada)
ParticipanteConversa (1) — Mensagem (0..*)	Associação 1:N: FK NOT NULL no lado "muitos". Como a PK de participante_conversa é composta, a FK também é composta; ON DELETE CASCADE, pois a mensagem depende da participação.	mensagem.(id_conversa, id_remetente) → participante_conversa. (id_conversa, id_usuario)
Associações derivadas /remetente (Mensagem → Usuario) e /conversa (Mensagem → Conversa)	Associações derivadas não geram FK própria: seus valores vêm das colunas da FK composta acima (id_remetente = usuário; id_conversa = conversa). Isso impede mensagem fora da conversa e remetente que não participa.	— (sem coluna adicional)
Generalização/especialização	Não há herança no diagrama de classes; Visitante e Usuário Autenticado são papéis de ator (UML), não classes persistentes. Logo, as estratégias da Aula 18 não se aplicam.	—

10. Observações sobre o Esquema Relacional

10.1. Chaves substitutas e chaves naturais

Todas as tabelas usam chave primária substituta (UUID gerado pelo banco), como recomendado por Bezerra para representar a identidade dos objetos: é estável (não muda se o usuário trocar de e-mail ou renomear uma refeição), compacta para as FKs e é o padrão do Supabase. A única exceção é participante_conversa, cuja PK é naturalmente composta pelas duas FKs (e coincide com a CN usuario + conversa). As chaves naturais {CN} da Fase A-2 foram preservadas como restrições UNIQUE em usuario, atividade, refeicao e mensagem. Em plano_alimentar, a CN (usuario + dataInicio) é coberta por uma restrição mais forte, UNIQUE (id_usuario), que implementa a multiplicidade 0..1 e, por consequência, também torna única a combinação usuário + data_inicio. A CN de Conversa (participantes + dataCriacao) envolve linhas de outra tabela (participante_conversa) e por isso não pode ser declarada como UNIQUE; como a identidade da conversa é dada pela chave substituta, ela fica apenas documentada.

10.2. Restrições que não cabem em PK, FK, UNIQUE ou CHECK simples

- **Atividades sem sobreposição:** implementada declarativamente por uma restrição EXCLUDE do PostgreSQL (extensão btree_gist), que impede que dois intervalos [hora_inicio, hora_fim) do mesmo usuário e da mesma data se interceptem. Atividades encostadas (uma termina às 09:00 e outra começa às 09:00) são permitidas. Como hora_fim > hora_inicio na mesma data, uma atividade não atravessa a meia-noite, o que é coerente com a fita diária de 24h (UC-03).
- **Exatamente 2 participantes por conversa:** a notação pé-de-galinha não expressa a multiplicidade "2" (o diagrama mostra 1..* com a anotação (2)). A regra é garantida por dois gatilhos: um impede o terceiro participante (e a entrada de contas INATIVAS, RN-01); outro, adiado até o COMMIT, rejeita conversas com número de participantes diferente de 2. Assim a conversa e seus dois participantes são criados na mesma transação (UC-05, fluxo A1).
- **Plano com ao menos 1 refeição (1..*):** uma FK garante apenas o "no máximo" do lado da parte; o mínimo de uma refeição é verificado por gatilho adiado até o COMMIT.
- **Remetente é participante da conversa:** decorre diretamente da estrutura do modelo, sem precisar de gatilho: a Mensagem se associa à classe associativa ParticipanteConversa, e a FK composta (id_conversa, id_remetente) → participante_conversa rejeita mensagens de quem não participa da conversa.

10.3. Exclusão definitiva de conta (RN-03 / UC-06)

Todas as FKs que partem de dados do usuário usam ON DELETE CASCADE, de modo que excluir a linha de usuario remove atividade, plano_alimentar e, por composição, refeicao. As conversas exigem um passo a mais: como uma conversa não pode existir com um único participante (multiplicidade 2), a função excluir_conta remove primeiro as conversas das quais o usuário participa (em cascata: participante_conversa e mensagem) e depois o usuário, tudo em uma única transação, atendendo à pós-condição do UC-06 ("removidos permanentemente"). Para o outro participante, a conversa deixa de existir, o que corresponde ao fluxo de exceção E3 do UC-05.

10.4. Dados sensíveis e controle de acesso (RN-02 / RNF-03)

As colunas marcadas com **S** na Figura 3 (peso e dados alimentares) são cifradas antes de serem persistidas. O acesso restrito ao titular é garantido no próprio banco por políticas de Row Level Security do Supabase (id_usuario = auth.uid()), e não apenas pela interface. Duas consequências de projeto: (a) no esquema físico, uma coluna cifrada é armazenada como BYTEA, então as restrições de domínio desses campos (valores >= 0) são validadas antes da cifragem, conforme RNF-05; o esquema lógico mantém os tipos e CHECKs do domínio para documentar a regra; (b) em atividade, as colunas data e hora não são cifradas, pois precisam estar legíveis para a ordenação (UC-03) e para a restrição EXCLUDE; essa tabela é protegida pelo controle de acesso por titular (RLS).

10.5. Normalização (1FN, 2FN e 3FN)

- **1FN:** todas as colunas são atômicas e não há grupos repetitivos. O atributo multivalorado "participantes" de Conversa virou a tabela participante_conversa, e as refeições ficam em tabela própria em vez de colunas repetidas (refeicao1, refeicao2...).
- **2FN:** as tabelas com PK simples estão trivialmente em 2FN. A única PK composta (participante_conversa) tem o atributo não chave data_entrada, que depende da chave inteira (id_conversa, id_usuario), isto é, de quando *aquele usuário* entrou *naquela conversa*. Nas chaves candidatas compostas (UNIQUE), cada atributo não chave depende da chave inteira: em atividade, hora_fim e titulo dependem de (id_usuario, data, hora_inicio), e não apenas da data.
- **3FN:** não há dependências transitivas. Dados deriváveis não são armazenados: a idade vem de data_nascimento; o nome do remetente vem de usuario; o total de calorias das refeições é calculado. Um ponto analisado foi calorias_diarias em relação aos macronutrientes (aprox. 4·proteínas + 4·carboidratos + 9·gorduras): foi mantida como coluna porque representa a *meta* definida no plano, que pode divergir da soma (fibras, arredondamentos), e não uma dependência funcional.

10.6. Integração com o Supabase

O Supabase gerencia a autenticação na tabela interna auth.users, que já armazena o e-mail e a senha em hash (bcrypt). Na implementação física, usuario.id_usuario recebe o mesmo valor de auth.users.id, com FK e ON DELETE CASCADE, e a coluna senha_hash do esquema lógico corresponde à senha mantida pelo Supabase Auth, sem ser duplicada em public.usuario (RNF-03). Do ponto de vista do sistema, o Supabase é infraestrutura (autenticação e hospedagem do PostgreSQL), e não ator (Seção 3.1); ele aparece como nó e componentes nos diagramas da Seção 11.

10.7. Por que relacional e não NoSQL

O domínio tem relacionamentos fortes e regras de integridade que o banco precisa garantir (cascata da LGPD, unicidade, remetente participante, não sobreposição), o que favorece o modelo relacional. A parte que mais se aproximaria de um modelo NoSQL orientado a agregados é o chat (uma conversa com suas mensagens embutidas em um documento), mas o volume esperado é pequeno e o PostgreSQL já oferece mensagens em tempo real sobre tabelas relacionais, então a separação não se justifica nesta etapa.

Parte IV — Projeto Arquitetural e Funcional (Fase B-4)

Esta parte complementa as anteriores com o **Projeto Arquitetural** (visão lógica, por um Diagrama de Componentes, e visão física, por um Diagrama de Implantação) e com o **Projeto Funcional** (diagramas comportamentais de três casos de uso: atividades, sequência e estados). Os diagramas seguem a UML e o material de Bezerra (caps. 7, 9, 10 e 11).

Item entregue	Seção	Figura
Diagrama de Componentes (visão arquitetural lógica)	11.2	Figura 4
Diagrama de Implantação (visão arquitetural física)	11.3	Figura 5
Diagrama de Atividades — UC-01 Criar Cadastro no Sistema	12.2	Figura 6
Diagrama de Sequência — UC-05 Enviar Mensagem no Chat	12.3	Figura 7
Diagrama de Estados — UC-06 Excluir Conta Definitivamente	12.4	Figura 8

11. Projeto Arquitetural (Lógico e Físico)

11.1. Visão geral e decisões arquiteturais

O GymGramps é uma **aplicação de página única (SPA)** em React + Vite, organizada em camadas inspiradas na Arquitetura Limpa (Martin, 2012; Terralab/UFOP, 2021), que usa o Supabase como infraestrutura de autenticação, API de dados, tempo real e banco PostgreSQL. Não há servidor de aplicação próprio nesta etapa. O Supabase é tratado como infraestrutura (nó e componentes externos), e não como ator.

Decisão arquitetural	Justificativa
SPA em React + Vite, organizada em camadas (Apresentação, Aplicação, Domínio, Infraestrutura).	Atende RNF-02 (troca de telas sem recarregar) e separa responsabilidades: cada grupo de UC vira um serviço de aplicação.
Regra de dependência da Arquitetura Limpa: as dependências apontam para o núcleo; os serviços dependem de interfaces (portas), nunca do Supabase.	Permite testar os serviços com repositórios falsos e trocar a plataforma de dados sem alterar Domínio e Aplicação (Martin, 2012).
Supabase usado como infraestrutura (BaaS): Auth, API REST, Realtime e PostgreSQL.	Dispensa um servidor de aplicação próprio nesta etapa (escopo do projeto). Por ser infraestrutura, não é ator (Seção 3.1).
Regras críticas ficam no próprio banco: RLS, gatilhos e a função excluir_conta (Apêndice A).	A integridade é garantida onde os dados estão, mesmo se a interface for contornada (RN-02, RN-03). A chave de serviço nunca vai ao navegador.
Segurança em camadas: TLS em todos os canais, JWT do usuário, RLS, hash de senha no Auth e cifragem dos dados sensíveis pela aplicação.	Cumpre RN-02 e RNF-03 (dados de saúde protegidos).

Responsabilidades por camada

Camada	Componentes	Responsabilidades
Apresentação	UI Conta, UI Rotina, UI Alimentação, UI Chat	Telas React; validação de formato e completude (RNF-05); tema claro/escuro; chamam os casos de uso e exibem o resultado.
Aplicação	ContaService, RotinaService, AlimentacaoService, ChatService	Um serviço por grupo de casos de uso: orquestra o fluxo de cada UC, trata os fluxos de exceção e declara, como interfaces (portas), o que precisa da infraestrutura.
Domínio	Núcleo de Domínio	Entidades da Seção 7 (Usuario, Atividade, PlanoAlimentar, Refeicao, Conversa, ParticipanteConversa, Mensagem) e regras RN-01 a RN-03. Não depende de React nem do Supabase.

Camada	Componentes	Responsabilidades
Infraestrutura	ContaGateway, RotinaRepository, AlimentacaoRepository, ChatRepository, RealtimeChannel	Adaptadores que implementam as portas usando o cliente do Supabase (supabase-js); convertem linhas do banco em entidades e cifram/decifram os dados sensíveis (RNF-03).
Plataforma externa	Supabase Auth, API REST (PostgREST), Supabase Realtime, PostgreSQL 16	Serviços gerenciados que fornecem autenticação, API de dados, notificações em tempo real e o banco relacional. São infraestrutura, não atores.

11.2. Visão lógica — Diagrama de Componentes

O diagrama (Figura 4, na página seguinte) mostra os componentes em quatro camadas e a plataforma externa. Cada componente da Apresentação *requer* a interface de casos de uso fornecida pelo seu serviço de aplicação; cada serviço *requer* uma interface (porta) fornecida por um adaptador da Infraestrutura; e cada adaptador *requer* a API de um componente do Supabase. As setas tracejadas «use» para o Núcleo de Domínio mostram a regra de dependência: o núcleo não depende de ninguém.

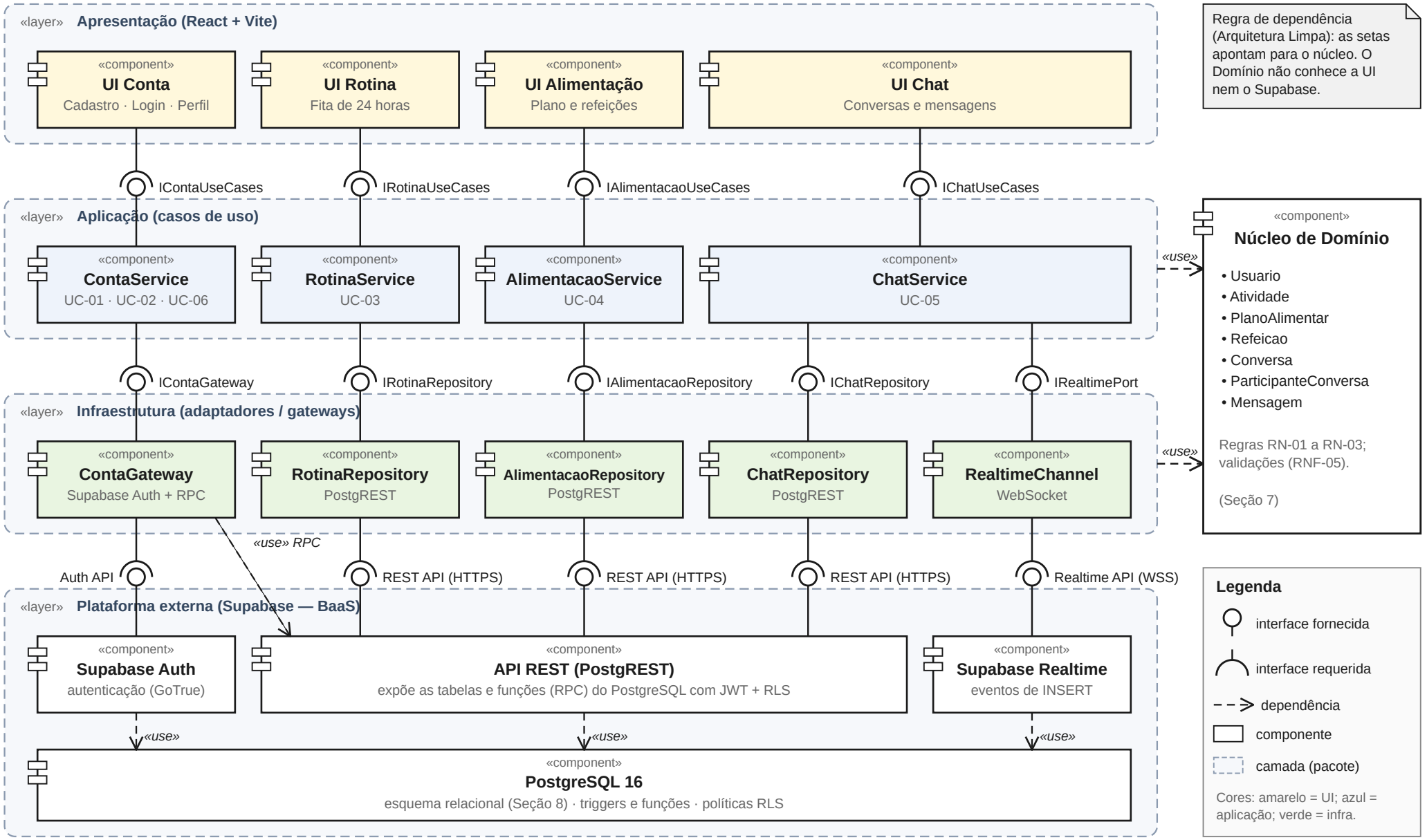


Figura 4 — Diagrama de Componentes do GymGramps (visão arquitetural lógica). Notação: bola = interface fornecida; soquete = interface requerida; seta tracejada = dependência. Cores: amarelo = apresentação; azul = aplicação; verde = infraestrutura.

Interfaces do diagrama de componentes

Interface	Fornecida por	Requerida por	Operações principais
IContaUseCases	ContaService	UI Conta	cadastrar(dados), entrar(email, senha), sair(), excluirConta(senha)
IRotinaUseCases	RotinaService	UI Rotina	atividadesDoDia(data)
IAlimentacaoUseCases	AlimentacaoService	UI Alimentação	planoVigente()
IChatUseCases	ChatService	UI Chat	listarConversas(), historico(idConversa), novaConversa(idUsuario), enviarMensagem(idConversa, conteudo)
IContaGateway	ContaGateway	ContaService	criarConta, autenticar, encerrarSessao, excluirConta
IRotinaRepository	RotinaRepository	RotinaService	buscarPorData(idUsuario, data)
IAlimentacaoRepository	AlimentacaoRepository	AlimentacaoService	buscarPlano(idUsuario)
IChatRepository	ChatRepository	ChatService	conversasDe, mensagensDe, criarConversa, registrarMensagem
IRealtimePort	RealtimeChannel	ChatService	inscrever(idConversa, aoReceber)
Auth API (HTTPS)	Supabase Auth	ContaGateway	cadastro, login, logout (JWT)
REST API (HTTPS)	API REST (PostgREST)	ContaGateway (RPC) e os três repositórios	consultas e inserções nas tabelas; RPC excluir_conta
Realtime API (WSS)	Supabase Realtime	RealtimeChannel	inscrição em eventos de INSERT em mensagem

Casos de uso × componentes

Caso de uso	Apresentação	Aplicação	Infraestrutura	Serviços do Supabase usados
UC-01	UI Conta	ContaService	ContaGateway	Auth; REST (usuario)
UC-02	UI Conta	ContaService	ContaGateway	Auth
UC-03	UI Rotina	RotinaService	RotinaRepository	REST (atividade)
UC-04	UI Alimentação	AlimentacaoService	AlimentacaoRepository	REST (plano_alimentar, refeicao) com RLS
UC-05	UI Chat	ChatService	ChatRepository; RealtimeChannel	REST (conversa, participante_conversa, mensagem); Realtime
UC-06	UI Conta (Perfil)	ContaService	ContaGateway	Auth; REST (RPC excluir_conta)

11.3. Visão física — Diagrama de Implantação

O diagrama (Figura 5, na página seguinte) mostra onde cada artefato executa. A SPA é publicada como arquivos estáticos em uma hospedagem (CDN), baixada pelo navegador do usuário, e **todos os componentes da Seção 11.2 que pertencem ao cliente executam no navegador**. O navegador conversa diretamente com três serviços do Supabase Cloud — Auth, API REST e Realtime — e estes acessam o PostgreSQL pela rede interna da plataforma.

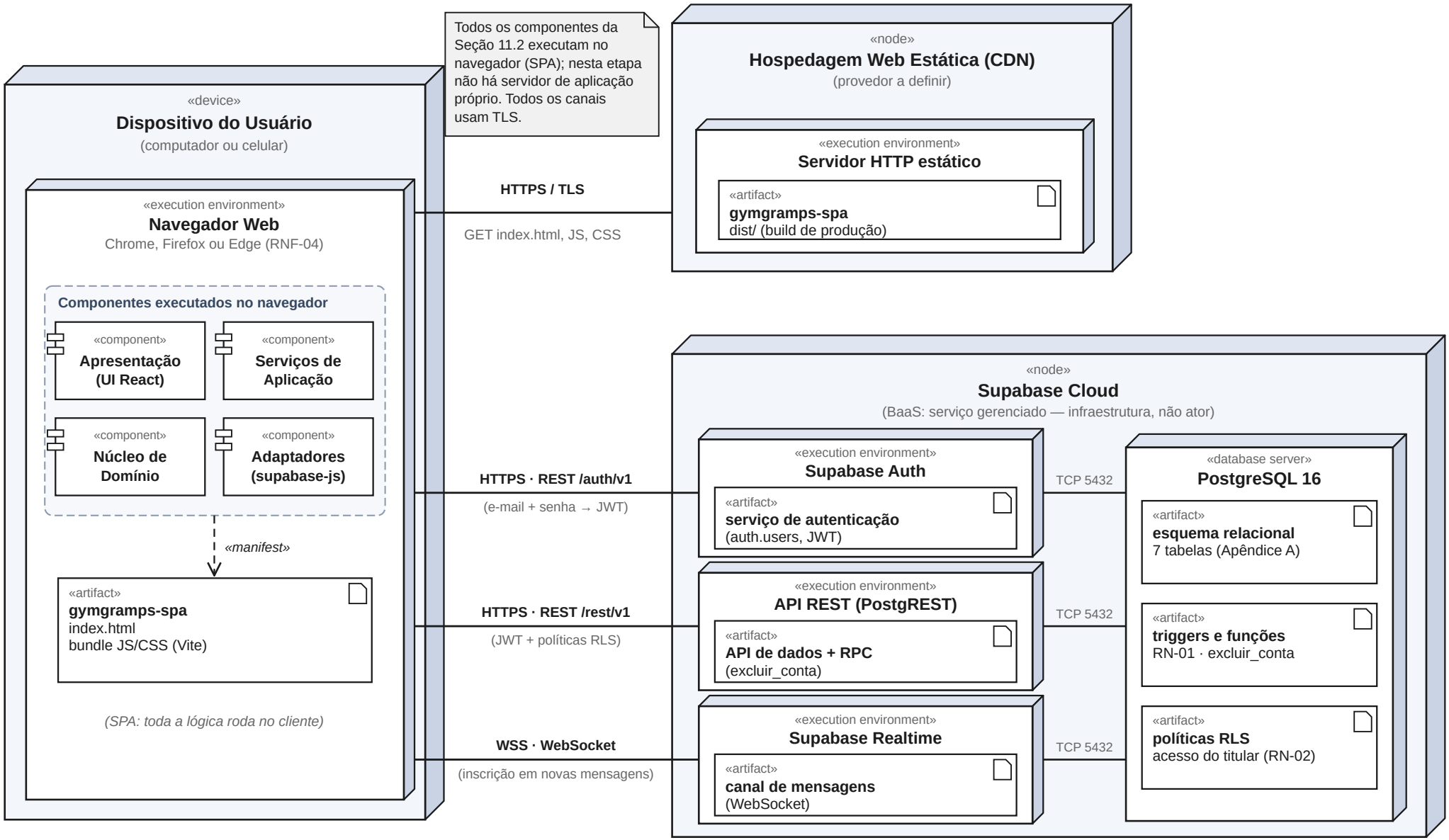


Figura 5 — Diagrama de Implantação do GymGramps (visão arquitetural física). Nós em 3D; retângulos com ícone de documento = artefatos; linhas = caminhos de comunicação, com o protocolo indicado.

Nó / ambiente	Tipo (UML)	Artefatos	Observações
Dispositivo do Usuário	«device»	—	Computador ou celular do praticante.
Navegador Web	«execution environment»	gymgramps-spa (index.html, bundle JS/CSS)	Chrome, Firefox ou Edge (RNF-04). Executa Apresentação, Aplicação, Domínio e Adaptadores.
Hospedagem Web Estática (CDN)	«node»	gymgramps-spa (dist/)	Serve os arquivos da SPA por HTTPS. Provedor a definir.
Supabase Auth	«execution environment»	serviço de autenticação (auth.users)	Cadastro e login; emite o JWT do usuário.
API REST (PostgREST)	«execution environment»	API de dados + RPC	Expõe as tabelas e a função excluir_conta, aplicando RLS.
Supabase Realtime	«execution environment»	canal de mensagens (WebSocket)	Notifica os inscritos quando uma mensagem é inserida.
PostgreSQL 16	«database server»	esquema relacional; triggers e funções; políticas RLS	Banco do Apêndice A (7 tabelas).

Canais de comunicação e segurança

- **HTTPS/TLS** entre o navegador e a hospedagem estática (download de index.html e do bundle) e entre o navegador e o Supabase (Auth e API REST); **WSS** (WebSocket seguro) para o Realtime; **TCP 5432** entre os serviços do Supabase e o PostgreSQL, dentro da rede interna do provedor.
- O navegador usa apenas a chave pública do projeto e o **JWT** do usuário autenticado; a chave de serviço nunca é enviada ao cliente. O PostgreSQL aplica as políticas **RLS** com base nesse JWT (RN-02).
- A hospedagem estática não guarda dados nem segredos: apenas serve os arquivos públicos da aplicação. O provedor ainda será escolhido; isso não altera os demais nós.

12. Projeto Funcional (Modelo de Interações)

12.1. Seleção dos casos de uso e dos diagramas

Foram escolhidos três casos de uso comuns e críticos. Como são casos de uso comuns, cada um é coberto por **um diagrama distinto e específico**, de modo que, juntos, entregam o diagrama de atividades e o diagrama de sequência (obrigatórios) e um terceiro diagrama, o de estados.

Caso de uso	Diagrama	Por que este caso de uso	Por que este diagrama	Fluxos cobertos
UC-01 Criar Cadastro	**Atividades**	É a porta de entrada do sistema: cria o perfil com validação, hash de senha e cifragem de dados sensíveis.	Há várias decisões e laços (dados inválidos, falha de comunicação, e-mail repetido) e três responsáveis (visitante, interface, serviço de conta).	Básico 1–7; A1; E1–E3
UC-05 Enviar Mensagem	**Sequência**	É o caso mais interativo: envolve dois usuários, todas as camadas e a entrega em tempo real.	O que importa é a ordem das mensagens entre interface, serviço, repositório, banco e canal em tempo real.	Básico 5–8; E1; E2 (passos 1–4 por referência)
UC-06 Excluir Conta	**Estados**	É irreversível e decorre da LGPD (RN-03): precisa ser entendido sem ambiguidade.	A conta passa por estados bem definidos (ativa, aguardando confirmação, validando senha, excluindo, excluída), com eventos e condições de guarda.	Básico 1–6; A1; E1; E2

12.2. UC-01 — Criar Cadastro no Sistema (Diagrama de Atividades)

Objetivo do diagrama: mostrar o fluxo de trabalho do cadastro, com os pontos de decisão e os laços de retorno ao formulário. O diagrama (Figura 6, na página seguinte) usa três raias (partições): *Visitante* (ator),

Interface Web (SPA) e *Serviço de Conta* (autenticação e persistência). Os números nas ações correspondem aos passos do fluxo básico do UC-01 (Seção 6.2); A1 e E1–E3 são os fluxos alternativo e de exceção.

Fluxo do UC-01	**Elemento do diagrama**
Passos 1–3	Ações 1, 2 e 3 (visitante e interface).
A1 — cancelar	Decisão “Confirma envio?”, ramo [voltar]: descarta os dados e termina.
Passo 4 e E1	Ação 4 (validação, RNF-05) e decisão “Dados válidos?”: se não, destaca o campo pendente e volta ao passo 3.
Passo 5 e E3	Ação 5 (hash, cifragem e solicitação) e decisão “Serviço respondeu?”: sem resposta, informa indisponibilidade e volta ao passo 3 para nova tentativa.
Passo 6 e E2	Decisão “E-mail já cadastrado?”: se sim, exibe erro e volta ao passo 3; se não, ação 6 cria o Usuario (ATIVA) e inicia a sessão.
Passo 7	Ação 7 redireciona ao painel de Rotina (UC-03) e a atividade termina.

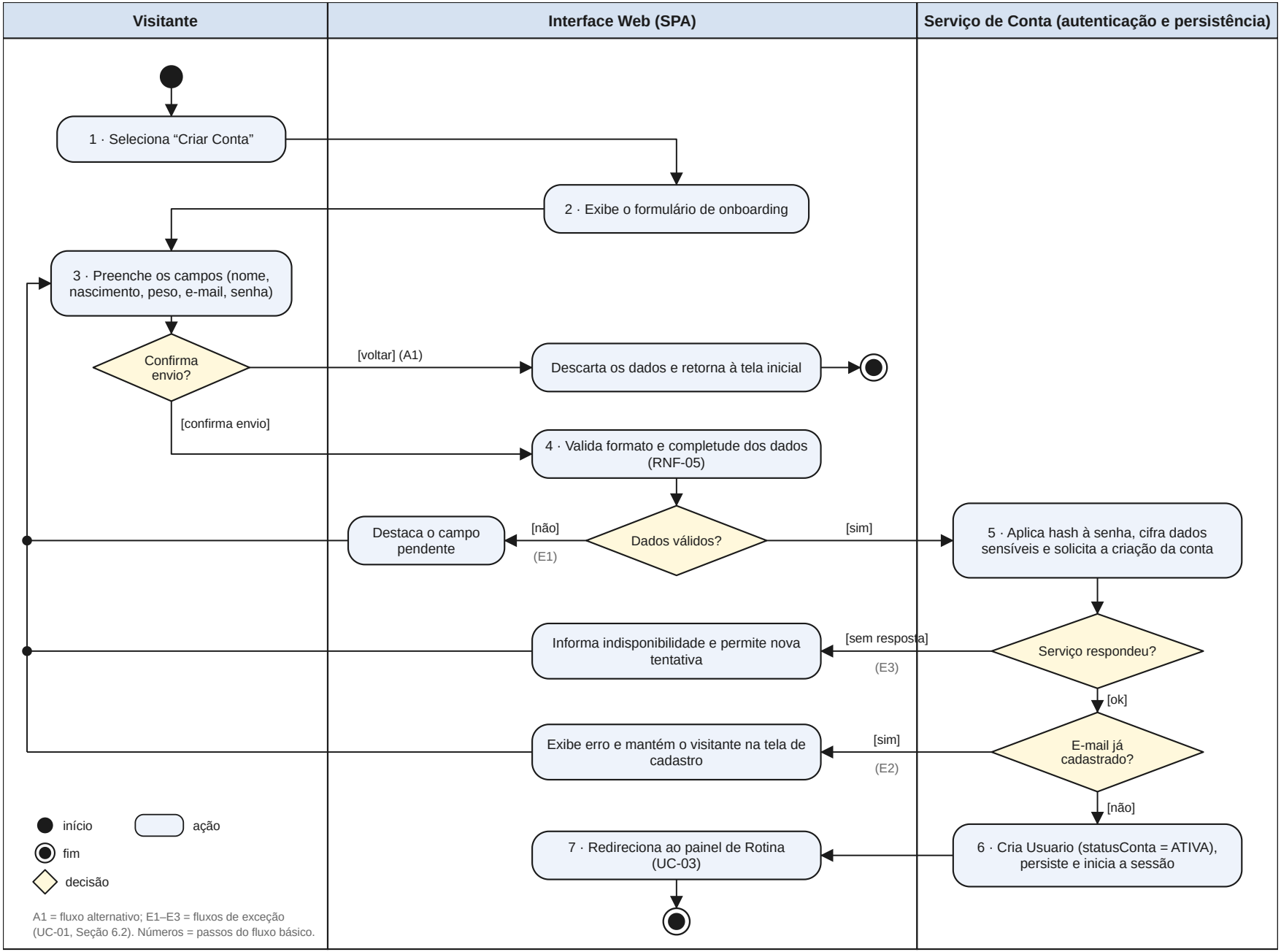


Figura 6 — Diagrama de Atividades do UC-01 — Criar Cadastro no Sistema.

12.3. UC-05 — Enviar Mensagem no Chat (Diagrama de Sequência)

Objetivo do diagrama: mostrar a troca de mensagens, em ordem temporal, entre o remetente, a interface (:ChatPage), o serviço de aplicação (:ChatService), o repositório (:ChatRepository), o banco (:PostgreSQL), o canal em tempo real (:Realtime) e o destinatário. Os passos 1–4 do fluxo básico (listar conversas e exibir o histórico) são representados por uma referência (**ref**); o diagrama detalha os passos 5–8 e as exceções E1 e E2. O registro da mensagem ocorre vinculado à participação do remetente (ParticipanteConversa), conforme o modelo corrigido.

Mensagens	Correspondência com o UC-05
1 e 2	Passos 5 e 6; o fragmento **opt** [conteúdo não vazio] representa o fluxo E1: se o conteúdo estiver vazio, o botão de envio fica desabilitado e nada é enviado.
3 a 6	Passo 7: o serviço pede ao repositório que registre a Mensagem; o banco valida a FK composta (id_conversa, id_remetente) → ParticipanteConversa e a política RLS.
7 a 9	Passo 8 (entrega): o banco gera o evento de INSERT, o canal em tempo real o repassa e a interface do destinatário exibe a mensagem.
10 a 13	Passo 8 (confirmação): o resultado volta pelas camadas e a mensagem é exibida ao remetente como enviada. Os blocos de entrega e confirmação ocorrem em paralelo (**par**).
14 a 16	Fluxo E2 (**alt** [falha de conexão]): o erro sobe até a interface, que marca a mensagem como NAO_ENVIADA e oferece o reenvio.

> Fora do diagrama: A1 (nova conversa) cria a Conversa e as duas ParticipanteConversa na mesma transação; E3 (destinatário excluiu a conta) é tratado ao listar as conversas.

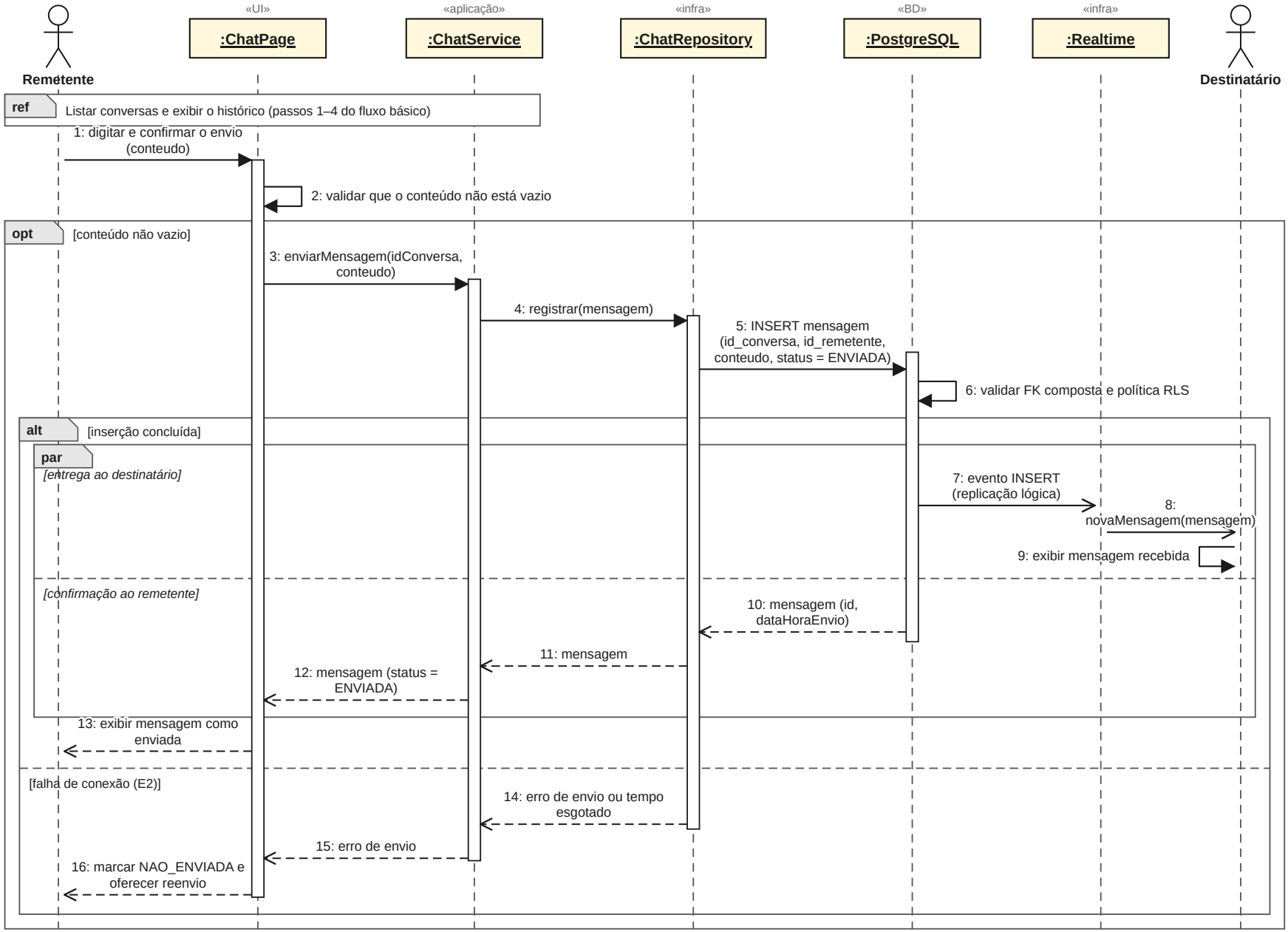


Figura 7 — Diagrama de Sequência do UC-05 — Enviar Mensagem no Chat.

12.4. UC-06 — Excluir Conta Definitivamente (Diagrama de Estados)

Objetivo do diagrama: mostrar os estados pelos quais a conta passa durante a exclusão e o que dispara cada mudança de estado. O objeto observado é a conta do Usuario; as transições usam o formato *evento [condição] / ação*. O estado final só é alcançado quando a exclusão em cascata termina com sucesso; em qualquer falha os dados são mantidos (RN-03).

Transição	Evento ou condição	Ação	Fluxo do UC-06
Ativa → Aguardando confirmação	selecionarExcluirConta	entry: alertar sobre a irreversibilidade e solicitar a senha	Passos 1–2
Aguardando confirmação → Ativa	cancelar	retornar ao Perfil	A1
Aguardando confirmação → Validando senha	confirmar(senha)	do: validar a senha informada	Passos 3–4
Validando senha → Aguardando confirmação	[senha incorreta]	exibir erro	E1
Validando senha → Excluindo dados	[senha correta]	abrir transação única; excluir_conta()	Passos 4–5
Excluindo dados → Aguardando confirmação	[falha na exclusão]	informar o erro; dados mantidos	E2
Excluindo dados → Conta excluída	[exclusão concluída]	entry: encerrar a sessão	Passos 5–6
Conta excluída → fim	—	exibir confirmação na tela inicial	Passo 6

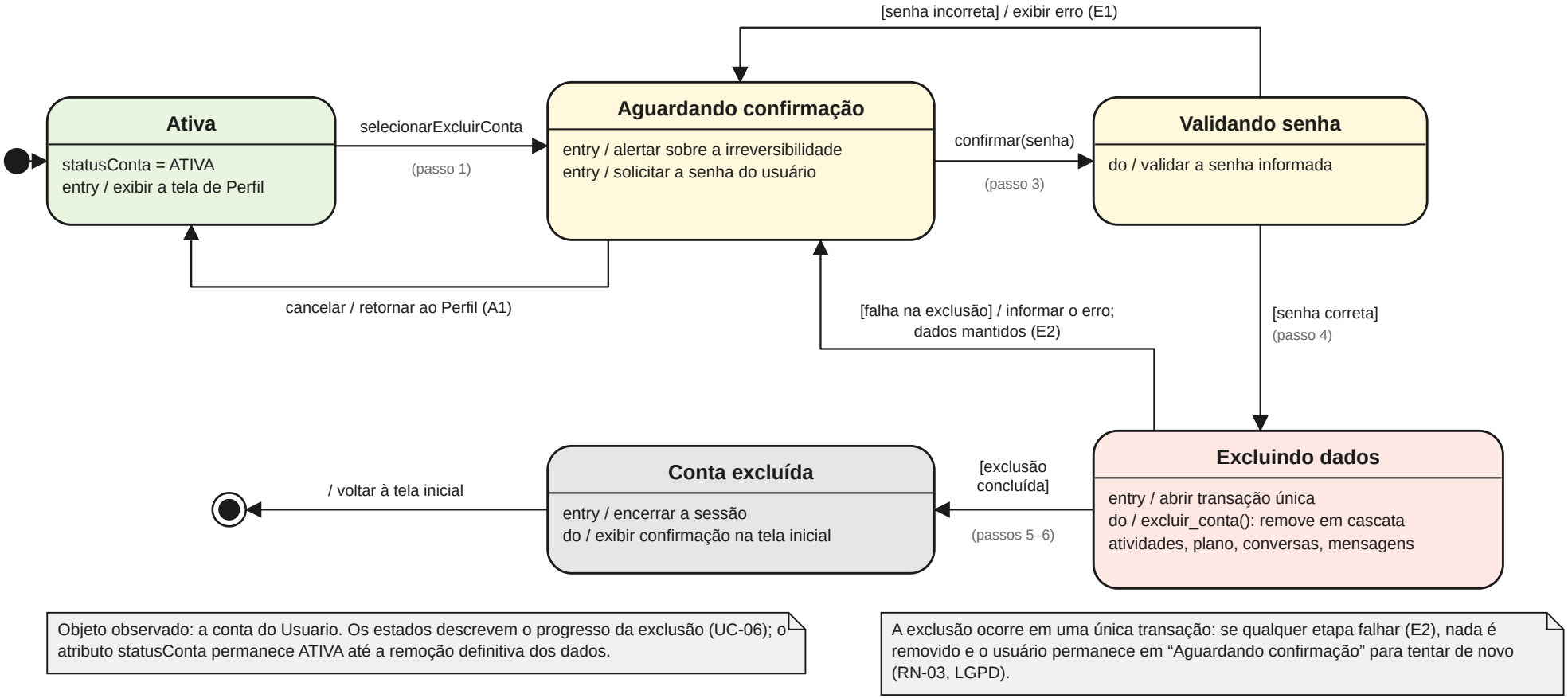


Figura 8 — Diagrama de Estados do UC-06 — Excluir Conta Definitivamente.

Parte V — Esquema Físico do Banco de Dados (Fase C-5)

Esta parte complementa o documento com o **esquema físico** do banco de dados relacional: o script SQL:DDL (banco, tipos, tabelas, colunas, chaves primárias, estrangeiras e compostas, índices, restrições, gatilhos, funções, visões e controle de acesso) e o script SQL:DML da carga inicial (inserções e atualizações). O esquema físico parte do esquema lógico da Seção 8 e do script de referência do Apêndice A, e os quatro scripts desta parte foram **executados do zero em um PostgreSQL 16** (Seção 15), inclusive com casos de violação de cada restrição.

Item entregue	Seção	Script
SGBDR, versão e detalhes de ambiente	13.1	—
Decisões do projeto físico (lógico → físico), regras de integridade e índices	13.2 a 13.4	—
SQL:DDL — criação do banco e do esquema físico	13.5 e 13.6	00_banco.sql, 01_ddl.sql
SQL:DML — carga inicial (inserções e atualizações)	14	02_dml.sql
Testes de integridade, de consultas dos casos de uso e de índices	15	03_testes.sql

13. Esquema Físico do Banco de Dados Relacional

13.1. SGBDR, versão e ambiente

O SGBDR escolhido é o **PostgreSQL**, o mesmo alvo do esquema lógico (Seção 8). Ele oferece os recursos que o modelo exige e que nem todo SGBD relacional oferece de forma declarativa: restrição de exclusão (*EXCLUDE*) para a não sobreposição de atividades, tipos enumerados, gatilhos de restrição adiados até o *COMMIT* e segurança por linha (RLS).

Item	Definição
SGBDR e versão testada	**PostgreSQL 16.15** (pacote 16.15-0ubuntu0.24.04.1, Ubuntu 24.04, x86_64).
Versão mínima	PostgreSQL 15 (visões com `security_invoker`). Não é necessário nenhum recurso exclusivo do Supabase: o mesmo script roda em um PostgreSQL comum.
Extensões	`btree_gist` 1.7 (usada na EXCLUDE de atividades, que combina igualdade de usuário e interseção de intervalos) e `pgcrypto` 1.3 (`gen_random_uuid`, bcrypt com `crypt`/`gen_salt` e cifragem simétrica AES-256 com `pgp_sym_encrypt`).
Banco e esquema	Banco `gymgramps`, codificação UTF8 (acentuação do português), fuso horário `America/Sao_Paulo`, objetos no esquema `public`. Colunas de data/hora de eventos usam `TIMESTAMPTZ`.
Papéis	`postgres` (superusuário; cria e é dono dos objetos) e `gymgramps_app` (papel da aplicação: sem login, sem superusuário, **sem** `BYPASSRLS`), com privilégios mínimos (Seção 13.6, item 9).
Parâmetros de sessão	`gymgramps.id_usuario` (quem está logado; lido pela função `usuario_corrente()` e pelas políticas RLS) e `gymgramps.chave_cifra` (chave de cifragem; nunca gravada no banco).
Contagem de objetos	7 tabelas, 4 tipos enumerados, 7 chaves primárias, 6 estrangeiras, 5 únicas, 8 CHECK e 1 EXCLUDE, 18 índices, 10 funções, 6 gatilhos, 3 visões e 10 políticas RLS.
Ordem de execução	`psql -U postgres -f 00_banco.sql` → `psql -U postgres -d gymgramps -v ON_ERROR_STOP=1 -f 01_ddl.sql` → `... -f 02_dml.sql` → (opcional) `... -f 03_testes.sql`.
Uso no Supabase	O banco gerenciado já existe e `usuario.id_usuario` recebe o mesmo valor de `auth.users.id` (Seção 10.6). Para publicar lá: executar apenas o 01 (sem `CREATE DATABASE`), trocar o corpo de `usuario_corrente()` por `auth.uid()`, usar o papel `authenticated` no lugar de `gymgramps_app` e dispensar a coluna `senha_hash`. Os testes desta parte foram feitos em PostgreSQL local, não no Supabase.

13.2. Do esquema lógico ao esquema físico

O esquema físico mantém as 7 tabelas, as chaves e as multiplicidades da Seção 8. A tabela abaixo reúne o que foi acrescentado ou alterado ao passar do esquema lógico (Apêndice A) para o físico, com o motivo.

Tema	Esquema lógico (Seção 8 / Apêndice A)	Esquema físico (Parte V)	Motivo
Dados sensíveis (RN-02, RNF-03)	Colunas marcadas S com tipo do domínio (ex.: `peso NUMERIC(5,2)`).	Colunas `*_cif BYTEA` (peso, objetivo, calorias e macros do plano, descrição e calorias da refeição), cifradas com `fn_cifrar` (AES-256).	A Seção 10.4 já previa BYTEA no esquema físico. As regras de domínio (peso > 0, valores >= 0) são validadas antes da cifragem (RNF-05).
Senha (RNF-03)	`senha_hash VARCHAR(255)`.	Mantida; o script de carga grava hash bcrypt (`crypt` + `gen_salt('bf')`).	Script roda em PostgreSQL puro. No Supabase a senha fica em `auth.users`.

Tema	Esquema lógico (Seção 8 / Apêndice A)	Esquema físico (Parte V)	Motivo
Restrições	Restrições sem nome na maior parte.	Todas nomeadas: `pk_`, `fk_`, `uq_`, `ck_`, `ex_`.	Mensagens de erro identificam a regra violada; facilita testes e manutenção.
Qualidade dos dados (RNF-05)	CHECKs de peso, horário e conteúdo.	CHECKs adicionais: e-mail em minúsculas e com formato válido; nome, título e nome de refeição não vazios; data de nascimento entre 1900 e hoje.	Dados validados no banco mesmo se a interface for contornada.
`data_hora_envio`	`DEFAULT now()`.	`DEFAULT clock_timestamp()`.	`now()` é constante dentro da transação e faria duas mensagens do mesmo remetente colidirem em `uq_mensagem`.
Índices	2 índices secundários (`ix_participante_usuario`, `ix_mensagem_historico`).	5 índices secundários, 2 deles parciais (Seção 13.4).	Atender às consultas dos UC-03, UC-04 e UC-05.
Gatilhos	3 gatilhos (participantes, plano com refeição, conversa com 2).	6 gatilhos: acrescenta a verificação ao remover participante ou a última refeição, e a regra de remetente ATIVO.	Fechar lacunas das multiplicidades 2 e 1..* (inclusão e remoção) e RN-01.
Rotinas	`excluir_conta`.	`excluir_conta` (agora só o titular) e `criar_conversa` (UC-05, A1).	Uma única transação por caso de uso; o gatilho adiado confere os 2 participantes no COMMIT.
Controle de acesso (RN-02)	RLS apenas em `plano_alimentar` (exemplo).	RLS em 7 tabelas (10 políticas), papel `gymgramps_app`, privilégios mínimos e visões `security_invoker`.	Acesso restrito ao titular garantido no banco e não só na interface.
Visões	—	`vw_plano_alimentar`, `vw_refeicao` (decifram) e `vw_mensagem_historico`.	UC-04, passo 3 (decifrar) e associações derivadas /remetente e /conversa da Seção 7.3.

13.3. Regras de integridade e como são garantidas

Cada regra do modelo (Seções 4, 7 e 10) é garantida no próprio SGBD pelo mecanismo mais simples que a expressa. Todas foram exercitadas por um teste de violação (coluna "Teste", Seção 15).

Regra	Mecanismo	Objeto	Teste
E-mail único e válido (RF-01, RNF-05)	UNIQUE e CHECK	`uq_usuario_email`, `ck_usuario_email`	T01, T02
Domínios de tema, status e tipo	Tipos ENUM	`tema_t`, `status_conta_t`, `tipo_atividade_t`, `status_mensagem_t`	T03
hora_fim > hora_inicio; chave natural da atividade	CHECK e UNIQUE	`ck_atividade_horario`, `uq_atividade`	T05, T06
Atividades do mesmo usuário não se sobrepõem (encostadas podem)	EXCLUDE (GiST)	`ex_atividade_sobreposicao`	T04, T07
No máximo 1 plano alimentar por usuário (0..1)	UNIQUE	`uq_plano_usuario`	T08
Plano com ao menos 1 refeição (1..*)	Gatilhos de restrição adiados (inclusão do plano e remoção de refeição)	`tg_plano_com_refeicao`, `tg_refeicao_remocao`	T09, T10
Conversa com exatamente 2 participantes	Gatilho BEFORE (máx. 2) e gatilhos adiados (inclusão da conversa e remoção de participante)	`tg_valida_participante`, `tg_conversa_dois_participantes`, `tg_participante_remocao`	T11, T12, T14
Somente contas ATIVAS conversam (RN-01)	Gatilhos BEFORE	`tg_valida_participante`, `tg_valida_mensagem`	T13, T17
Remetente participa da conversa	FK composta para a classe associativa	`fk_mensagem_participante`	T15
Mensagem não vazia	CHECK	`ck_mensagem_conteudo`	T16
Exclusão definitiva de conta em cascata (RN-03)	ON DELETE CASCADE e rotina transacional	`excluir_conta e as FKs`	T25
Dados sensíveis cifrados (RN-02, RNF-03)	Colunas BYTEA e funções de cifragem	`colunas *_cif, fn_cifrar, fn_decifrar`	T18, T19
Acesso restrito ao titular (RN-02)	RLS, privilégios mínimos e visões security_invoker	`pol_*, gymgramps_app, vw_*`	T24

13.4. Índices

Os índices de PK, UNIQUE e EXCLUDE são criados pelo próprio SGBD; os cinco secundários foram escolhidos a partir das consultas dos casos de uso. O tipo padrão é a **B-tree** (igualdade, faixas e ordenação), usado em todos os índices exceto no da restrição de exclusão, que precisa do **GiST** para o operador de interseção de intervalos (&&). Dois índices são **parciais** (só indexam as linhas que a consulta realmente usa), o que os mantém pequenos. Não foram criados índices em colunas de baixa seletividade (`tema`, `tipo`, `concluida`), pois o planejador não os usaria e cada índice encarece inserções e atualizações.

Índice	Tabela (colunas)	Tipo	Origem e uso
`pk_usuario`	`usuario (id_usuario)`	B-tree	PK
`uq_usuario_email`	`usuario (email)`	B-tree	UNIQUE; busca por e-mail no login (UC-02)
`ix_usuario_ativo_nome`	`usuario (nome) WHERE status_conta = 'ATIVA'`	B-tree parcial	UC-05, A1: busca de destinatário entre contas ativas
`pk_atividade`	`atividade (id_atividade)`	B-tree	PK
`uq_atividade`	`atividade (id_usuario, data, hora_inicio)`	B-tree	UNIQUE (CN); também atende o UC-03 (atividades do dia, já em ordem de hora_inicio)
`ex_atividade_sobreposicao`	`atividade (id_usuario, tsrange(data + hora_inicio, data + hora_fim))`	GiST	EXCLUDE: impede sobreposição
`pk_plano_alimentar`	`plano_alimentar (id_plano)`	B-tree	PK
`uq_plano_usuario`	`plano_alimentar (id_usuario)`	B-tree	UNIQUE (multiplicidade 0..1); UC-04: plano do usuário
`pk_refeicao`	`refeicao (id_refeicao)`	B-tree	PK
`uq_refeicao`	`refeicao (id_plano, nome)`	B-tree	UNIQUE (CN)
`ix_refeicao_plano_horario`	`refeicao (id_plano, horario)`	B-tree	UC-04: refeições do plano por horário
`pk_conversa`	`conversa (id_conversa)`	B-tree	PK
`pk_participante_conversa`	`participante_conversa (id_conversa, id_usuario)`	B-tree	PK composta; alvo da FK composta de mensagem
`ix_participante_usuario`	`participante_conversa (id_usuario)`	B-tree	UC-05, passo 2: conversas de um usuário (a PK só cobre o sentido conversa → usuário)
`pk_mensagem`	`mensagem (id_mensagem)`	B-tree	PK
`uq_mensagem`	`mensagem (id_conversa, id_remetente, data_hora_envio)`	B-tree	UNIQUE (CN); também cobre as colunas da FK composta (cascata e conferência)
`ix_mensagem_historico`	`mensagem (id_conversa, data_hora_envio)`	B-tree	UC-05, passo 4: histórico em ordem cronológica
`ix_mensagem_pendentes`	`mensagem (id_conversa, id_remetente) WHERE status = 'NAO_ENVIADA'`	B-tree parcial	UC-05, E2: fila de reenvio (poucas linhas)

O teste T23 (Seção 15.3) confirma, pelo `EXPLAIN`*, que as consultas dos UC-03 e UC-05 usam* `uq_atividade`*,* `ix_mensagem_historico` *e* `ix_participante_usuario`*. Como as tabelas da carga inicial são minúsculas, o planejador preferiria a varredura sequencial; por isso o teste desliga* `enable_seqscan` *apenas para mostrar que o índice é utilizável.*

13.5. Script 00 — criação do banco de dados

Executar como superusuário. Cria o papel da aplicação e o banco `gymgramps` (refaz o banco se já existir; usar com cuidado fora de um ambiente de testes).

```
-- ============================================================================
-- GymGramps - Fase C-5 - Script 00: criação do banco de dados
-- SGBDR: PostgreSQL 16 (testado na 16.15)  |  Executar como superusuario (postgres)
-- Uso:   psql -U postgres -f 00_banco.sql
-- ============================================================================
```

```
-- Papel (role) da aplicação: NAO e superusuario, NAO ignora RLS.
-- No Supabase, este papel corresponde ao papel "authenticated".
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'gymgramps_app') THEN
    CREATE ROLE gymgramps_app NOLOGIN NOSUPERUSER NOBYPASSRLS;
  END IF;
END $$;
```

```
DROP DATABASE IF EXISTS gymgramps;
```

```
CREATE DATABASE gymgramps
  WITH ENCODING = 'UTF8'          -- acentuação do português
       TEMPLATE = template0
       CONNECTION LIMIT = -1;
```

```
COMMENT ON DATABASE gymgramps IS
  'GymGramps - rotina de treino e alimentação (PostgreSQL 16)';
```

```
-- Datas/horas dos treinos seguem o horário de Brasilia; TIMESTAMPTZ continua
-- armazenado em UTC internamente.
ALTER DATABASE gymgramps SET timezone TO 'America/Sao_Paulo';
ALTER DATABASE gymgramps SET datestyle TO 'ISO, DMY';
```

13.6. Script 01 — DDL do esquema físico

Conectado ao banco `gymgramps`. As seções 1 a 10 do script seguem a ordem de dependência: extensões, tipos, funções utilitárias, tabelas, índices, gatilhos, rotinas, visões, segurança e comentários de catálogo.

```
-- ============================================================================
-- GymGramps - Fase C-5 - Script 01: esquema fisico (DDL)
-- SGBDR: PostgreSQL 16 (testado na 16.15; requer 15+ por security_invoker)
-- Pré-requisito: script 00_banco.sql executado.
-- Uso:   psql -U postgres -d gymgramps -v ON_ERROR_STOP=1 -f 01_ddl.sql
-- ============================================================================
SET client_min_messages TO warning;
```

```
-- ----------------------------------------------------------------------------
-- 1. Extensões
-- ----------------------------------------------------------------------------
CREATE EXTENSION IF NOT EXISTS btree_gist;  -- EXCLUDE de atividades (uuid + intervalo)
CREATE EXTENSION IF NOT EXISTS pgcrypto;    -- gen_random_uuid, crypt, pgp_sym_*
```

```
-- ----------------------------------------------------------------------------
-- 2. Tipos enumerados (domínios das classes Tema, StatusConta, ...)
-- ----------------------------------------------------------------------------
CREATE TYPE tema_t            AS ENUM ('CLARO', 'ESCURO');
CREATE TYPE status_conta_t    AS ENUM ('ATIVA', 'INATIVA');
CREATE TYPE tipo_atividade_t  AS ENUM ('MUSCULACAO', 'CARDIO', 'ALONGAMENTO', 'OUTRO');
CREATE TYPE status_mensagem_t AS ENUM ('ENVIADA', 'NAO_ENVIADA');
```

```
-- ----------------------------------------------------------------------------
-- 3. Funções utilitárias: identidade da sessão e cifragem (RN-02 / RNF-03)
-- ----------------------------------------------------------------------------
-- Identidade do usuário logado. No Supabase, trocar o corpo por: SELECT auth.uid()
CREATE FUNCTION usuario_corrente() RETURNS uuid
LANGUAGE sql STABLE AS
$$ SELECT nullif(current_setting('gymgramps.id_usuario', true), '')::uuid $$;
```

```
-- Cifra um texto (AES-256, OpenPGP simétrico). A chave vem da sessão e NUNCA
-- fica gravada no banco ou neste script (em produção: gerenciador de segredos).
CREATE FUNCTION fn_cifrar(p_valor text) RETURNS bytea
LANGUAGE plpgsql STABLE AS
$$
DECLARE v_chave text := current_setting('gymgramps.chave_cifra', true);
BEGIN
  IF v_chave IS NULL OR v_chave = '' THEN
    RAISE EXCEPTION 'Chave de cifragem não configurada (gymgramps.chave_cifra)';
  END IF;
  RETURN pgp_sym_encrypt(p_valor, v_chave, 'cipher-algo=aes256');
END $$;
```

```
CREATE FUNCTION fn_decifrar(p_valor bytea) RETURNS text
LANGUAGE plpgsql STABLE AS
$$
DECLARE v_chave text := current_setting('gymgramps.chave_cifra', true);
BEGIN
  IF v_chave IS NULL OR v_chave = '' THEN
    RAISE EXCEPTION 'Chave de cifragem não configurada (gymgramps.chave_cifra)';
  END IF;
  RETURN pgp_sym_decrypt(p_valor, v_chave);
END $$;
```

```
-- ----------------------------------------------------------------------------
-- 4. Tabelas
--    Colunas marcadas [S] guardam dado sensível CIFRADO (BYTEA, sufixo _cif).
--    As restrições de domínio (peso > 0, calorias >= 0 ...) dessas colunas são
--    validadas pela aplicação ANTES da cifragem (RNF-05).
-- ----------------------------------------------------------------------------
CREATE TABLE usuario (
  id_usuario      uuid          NOT NULL DEFAULT gen_random_uuid(),
  email           varchar(255)  NOT NULL,
  nome            varchar(120)  NOT NULL,
  data_nascimento date          NOT NULL,
  peso_cif        bytea         NOT NULL,                        -- [S] peso (kg)
  senha_hash      varchar(255)  NOT NULL,                        -- bcrypt (crypt/gen_salt)
  tema            tema_t        NOT NULL DEFAULT 'CLARO',
  status_conta    status_conta_t NOT NULL DEFAULT 'ATIVA',
  data_cadastro   timestamptz   NOT NULL DEFAULT now(),
  CONSTRAINT pk_usuario        PRIMARY KEY (id_usuario),
  CONSTRAINT uq_usuario_email  UNIQUE (email),
  CONSTRAINT ck_usuario_email  CHECK (email = lower(email)
                                 AND email ~ '^[^@[:space:]]+@[^@[:space:]]+\.[^@[:space:]]+$'),
  CONSTRAINT ck_usuario_nome   CHECK (length(btrim(nome)) > 0),
  CONSTRAINT ck_usuario_nasc   CHECK (data_nascimento BETWEEN DATE '1900-01-01' AND CURRENT_DATE),
  CONSTRAINT ck_usuario_peso   CHECK (octet_length(peso_cif) > 0)
);
```

```
CREATE TABLE atividade (
  id_atividade uuid             NOT NULL DEFAULT gen_random_uuid(),
  id_usuario   uuid             NOT NULL,
  data         date             NOT NULL,
  hora_inicio  time             NOT NULL,
  hora_fim     time             NOT NULL,
  titulo       varchar(100)     NOT NULL,
  tipo         tipo_atividade_t NOT NULL,
  concluida    boolean          NOT NULL DEFAULT false,
  CONSTRAINT pk_atividade         PRIMARY KEY (id_atividade),
  CONSTRAINT fk_atividade_usuario FOREIGN KEY (id_usuario)
      REFERENCES usuario (id_usuario) ON DELETE CASCADE,
  CONSTRAINT uq_atividade         UNIQUE (id_usuario, data, hora_inicio),   -- CN
  CONSTRAINT ck_atividade_horario CHECK (hora_fim > hora_inicio),
  CONSTRAINT ck_atividade_titulo  CHECK (length(btrim(titulo)) > 0),
  -- sem sobreposição no mesmo dia; atividades "encostadas" são permitidas ([) )
  CONSTRAINT ex_atividade_sobreposicao EXCLUDE USING gist (
      id_usuario WITH =,
      tsrange(data + hora_inicio, data + hora_fim) WITH &&)
);
```

```
CREATE TABLE plano_alimentar (
  id_plano            uuid    NOT NULL DEFAULT gen_random_uuid(),
  id_usuario          uuid    NOT NULL,
  data_inicio         date    NOT NULL,
  objetivo_cif        bytea,                                      -- [S]
  calorias_diarias_cif bytea  NOT NULL,                           -- [S] kcal
  proteinas_g_cif     bytea   NOT NULL,                           -- [S] g
  carboidratos_g_cif  bytea   NOT NULL,                           -- [S] g
  gorduras_g_cif      bytea   NOT NULL,                           -- [S] g
  CONSTRAINT pk_plano_alimentar    PRIMARY KEY (id_plano),
  CONSTRAINT fk_plano_usuario      FOREIGN KEY (id_usuario)
      REFERENCES usuario (id_usuario) ON DELETE CASCADE,
  CONSTRAINT uq_plano_usuario      UNIQUE (id_usuario)            -- multiplicidade 0..1
);
```

```
CREATE TABLE refeicao (
  id_refeicao   uuid        NOT NULL DEFAULT gen_random_uuid(),
  id_plano      uuid        NOT NULL,
  nome          varchar(60) NOT NULL,
  horario       time        NOT NULL,
  descricao_cif bytea,                                            -- [S]
  calorias_cif  bytea       NOT NULL,                             -- [S] kcal
  CONSTRAINT pk_refeicao        PRIMARY KEY (id_refeicao),
  CONSTRAINT fk_refeicao_plano  FOREIGN KEY (id_plano)
      REFERENCES plano_alimentar (id_plano) ON DELETE CASCADE,
  CONSTRAINT uq_refeicao        UNIQUE (id_plano, nome),          -- CN
  CONSTRAINT ck_refeicao_nome   CHECK (length(btrim(nome)) > 0)
);
```

```
CREATE TABLE conversa (
  id_conversa  uuid        NOT NULL DEFAULT gen_random_uuid(),
  data_criacao timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT pk_conversa PRIMARY KEY (id_conversa)
);
```

```
-- Classe associativa ParticipanteConversa (Usuario - Conversa): PK composta.
CREATE TABLE participante_conversa (
  id_conversa  uuid        NOT NULL,
  id_usuario   uuid        NOT NULL,
  data_entrada timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT pk_participante_conversa PRIMARY KEY (id_conversa, id_usuario),
  CONSTRAINT fk_participante_conversa FOREIGN KEY (id_conversa)
      REFERENCES conversa (id_conversa) ON DELETE CASCADE,
  CONSTRAINT fk_participante_usuario  FOREIGN KEY (id_usuario)
      REFERENCES usuario (id_usuario) ON DELETE CASCADE
);
```

```
-- Mensagem se associa a classe associativa por FK COMPOSTA (remetente e
-- conversa derivam dela; garante "remetente participa da conversa").
CREATE TABLE mensagem (
  id_mensagem     uuid              NOT NULL DEFAULT gen_random_uuid(),
  id_conversa     uuid              NOT NULL,
  id_remetente    uuid              NOT NULL,
  data_hora_envio timestamptz       NOT NULL DEFAULT clock_timestamp(),
  conteudo        text              NOT NULL,
  status          status_mensagem_t NOT NULL DEFAULT 'ENVIADA',
  CONSTRAINT pk_mensagem        PRIMARY KEY (id_mensagem),
  CONSTRAINT fk_mensagem_participante FOREIGN KEY (id_conversa, id_remetente)
      REFERENCES participante_conversa (id_conversa, id_usuario) ON DELETE CASCADE,
  CONSTRAINT uq_mensagem        UNIQUE (id_conversa, id_remetente, data_hora_envio),  -- CN
  CONSTRAINT ck_mensagem_conteudo CHECK (length(btrim(conteudo)) > 0)
);
```

```
-- ----------------------------------------------------------------------------
-- 5. Índices (além dos criados por PK, UNIQUE e EXCLUDE)
--    Todos B-tree, exceto o GiST da restrição ex_atividade_sobreposicao.
-- ----------------------------------------------------------------------------
-- UC-05 A1: busca de usuários ativos por nome (índice parcial: contas INATIVAS fora)
CREATE INDEX ix_usuario_ativo_nome ON usuario (nome) WHERE status_conta = 'ATIVA';
-- UC-04: refeições do plano ordenadas por horário
CREATE INDEX ix_refeicao_plano_horario ON refeicao (id_plano, horario);
-- UC-05 passo 2: conversas de um usuário (a PK cobre apenas o sentido conversa->usuário)
CREATE INDEX ix_participante_usuario ON participante_conversa (id_usuario);
-- UC-05 passo 4: histórico paginado por conversa, em ordem cronológica
CREATE INDEX ix_mensagem_historico ON mensagem (id_conversa, data_hora_envio);
-- UC-05 E2: fila de reenvio (índice parcial, pequeno)
CREATE INDEX ix_mensagem_pendentes ON mensagem (id_conversa, id_remetente)
  WHERE status = 'NAO_ENVIADA';
-- UC-03 (atividades do dia por usuário, ordenadas por hora_inicio) já é atendido
-- por uq_atividade (id_usuario, data, hora_inicio); não precisa de índice extra.
```

```
-- ----------------------------------------------------------------------------
-- 6. Regras que PK/FK/UNIQUE/CHECK não expressam (gatilhos)
--    Funções SECURITY DEFINER: precisam enxergar todas as linhas, mesmo com RLS.
-- ----------------------------------------------------------------------------
-- 6.1 Participante: no máximo 2 por conversa e somente contas ATIVAS (RN-01)
CREATE FUNCTION fn_valida_participante() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_temp AS
$$
BEGIN
  IF (SELECT count(*) FROM participante_conversa
      WHERE id_conversa = NEW.id_conversa) >= 2 THEN
    RAISE EXCEPTION 'A conversa já possui 2 participantes';
  END IF;
  IF (SELECT status_conta FROM usuario
      WHERE id_usuario = NEW.id_usuario) <> 'ATIVA' THEN
    RAISE EXCEPTION 'Somente contas ATIVAS participam de conversas (RN-01)';
  END IF;
  RETURN NEW;
END $$;
```

```
CREATE TRIGGER tg_valida_participante
  BEFORE INSERT ON participante_conversa
  FOR EACH ROW EXECUTE FUNCTION fn_valida_participante();
```

```
-- 6.2 Conversa: exatamente 2 participantes, verificado no COMMIT (inclui o DELETE
--     de um participante, que deixaria a conversa com apenas 1)
CREATE FUNCTION fn_conversa_dois_participantes() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_temp AS
$$
DECLARE v_id uuid := CASE WHEN TG_OP = 'DELETE' THEN OLD.id_conversa ELSE NEW.id_conversa END;
BEGIN
  IF EXISTS (SELECT 1 FROM conversa WHERE id_conversa = v_id)
     AND (SELECT count(*) FROM participante_conversa WHERE id_conversa = v_id) <> 2 THEN
    RAISE EXCEPTION 'Uma conversa deve ter exatamente 2 participantes';
  END IF;
  RETURN NULL;
END $$;
```

```
CREATE CONSTRAINT TRIGGER tg_conversa_dois_participantes
  AFTER INSERT ON conversa
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION fn_conversa_dois_participantes();
```

```
CREATE CONSTRAINT TRIGGER tg_participante_remocao
  AFTER DELETE ON participante_conversa
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION fn_conversa_dois_participantes();
```

```
-- 6.3 Plano alimentar: ao menos 1 refeição (1..*), verificado no COMMIT
CREATE FUNCTION fn_plano_com_refeicao() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_temp AS
$$
DECLARE v_id uuid := CASE WHEN TG_OP = 'DELETE' THEN OLD.id_plano ELSE NEW.id_plano END;
BEGIN
  IF EXISTS (SELECT 1 FROM plano_alimentar WHERE id_plano = v_id)
     AND NOT EXISTS (SELECT 1 FROM refeicao WHERE id_plano = v_id) THEN
    RAISE EXCEPTION 'Um plano alimentar deve ter ao menos 1 refeição';
  END IF;
  RETURN NULL;
END $$;
```

```
CREATE CONSTRAINT TRIGGER tg_plano_com_refeicao
  AFTER INSERT ON plano_alimentar
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION fn_plano_com_refeicao();
```

```
CREATE CONSTRAINT TRIGGER tg_refeicao_remocao
  AFTER DELETE ON refeicao
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION fn_plano_com_refeicao();
```

```
-- 6.4 Mensagem: o remetente precisa estar com a conta ATIVA (RN-01)
CREATE FUNCTION fn_valida_mensagem() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_temp AS
$$
BEGIN
  IF (SELECT status_conta FROM usuario
       WHERE id_usuario = NEW.id_remetente) <> 'ATIVA' THEN
    RAISE EXCEPTION 'Somente contas ATIVAS enviam mensagens (RN-01)';
  END IF;
  RETURN NEW;
END $$;
```

```
CREATE TRIGGER tg_valida_mensagem
  BEFORE INSERT ON mensagem
  FOR EACH ROW EXECUTE FUNCTION fn_valida_mensagem();
```

```
-- ------------------------------------------------------------------------------
-- 7. Rotinas dos casos de uso (executam em uma única transação)
-- ------------------------------------------------------------------------------
-- UC-05 A1: cria a Conversa e as duas participações na mesma transação
CREATE FUNCTION criar_conversa(p_usuario_a uuid, p_usuario_b uuid) RETURNS uuid
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_temp AS
$$
DECLARE v_id uuid;
BEGIN
  IF p_usuario_a = p_usuario_b THEN
    RAISE EXCEPTION 'Os participantes da conversa devem ser distintos';
  END IF;
  IF usuario_corrente() IS NOT NULL AND usuario_corrente() <> p_usuario_a THEN
    RAISE EXCEPTION 'Você só pode iniciar conversas em seu próprio nome';
  END IF;
  INSERT INTO conversa DEFAULT VALUES RETURNING id_conversa INTO v_id;
  INSERT INTO participante_conversa (id_conversa, id_usuario)
       VALUES (v_id, p_usuario_a), (v_id, p_usuario_b);
  RETURN v_id;
END $$;
```

```
-- UC-06 / RN-03: exclusão definitiva. Remove primeiro as conversas do usuário
-- (cascata: participante_conversa e mensagem) e depois o próprio usuario
-- (cascata: atividade; plano_alimentar -> refeição).
CREATE FUNCTION excluir_conta(p_id_usuario uuid) RETURNS void
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_temp AS
$$
BEGIN
  IF usuario_corrente() IS NOT NULL AND usuario_corrente() <> p_id_usuario THEN
    RAISE EXCEPTION 'Você só pode excluir a sua própria conta';
  END IF;
  DELETE FROM conversa
   WHERE id_conversa IN (SELECT id_conversa FROM participante_conversa
                         WHERE id_usuario = p_id_usuario);
  DELETE FROM usuario WHERE id_usuario = p_id_usuario;
END $$;
```

```
-- ------------------------------------------------------------------------------
-- 8. Visões (decifram os dados sensíveis; security_invoker => RLS do chamador)
-- ------------------------------------------------------------------------------
CREATE VIEW vw_plano_alimentar WITH (security_invoker = true) AS
SELECT p.id_plano, p.id_usuario, p.data_inicio,
       fn_decifrar(p.objetivo_cif)                    AS objetivo,
       fn_decifrar(p.calorias_diarias_cif)::integer   AS calorias_diarias,
       fn_decifrar(p.proteinas_g_cif)::numeric(6,1)   AS proteinas_g,
       fn_decifrar(p.carboidratos_g_cif)::numeric(6,1) AS carboidratos_g,
       fn_decifrar(p.gorduras_g_cif)::numeric(6,1)    AS gorduras_g
  FROM plano_alimentar p;
```

```
CREATE VIEW vw_refeicao WITH (security_invoker = true) AS
SELECT r.id_refeicao, r.id_plano, r.nome, r.horario,
       fn_decifrar(r.descricao_cif)         AS descricao,
       fn_decifrar(r.calorias_cif)::integer AS calorias
  FROM refeicao r;
```

```
-- Histórico de chat: /remetente e /conversa (associações derivadas) via participante
CREATE VIEW vw_mensagem_historico WITH (security_invoker = true) AS
SELECT m.id_conversa, m.data_hora_envio, u.nome AS remetente, m.conteudo, m.status
  FROM mensagem m
  JOIN participante_conversa pc
    ON pc.id_conversa = m.id_conversa AND pc.id_usuario = m.id_remetente
  JOIN usuario u ON u.id_usuario = pc.id_usuario;
```

```
-- ------------------------------------------------------------------------------
-- 9. Segurança: RLS (acesso restrito ao titular - RN-02) e privilégios
-- ------------------------------------------------------------------------------
-- Auxiliar SECURITY DEFINER (evita recursao de politica em participante_conversa)
CREATE FUNCTION fn_participa(p_id_conversa uuid) RETURNS boolean
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public, pg_temp AS
$$ SELECT EXISTS (SELECT 1 FROM participante_conversa
                  WHERE id_conversa = p_id_conversa
                    AND id_usuario = usuario_corrente()) $$;
```

```
ALTER TABLE usuario              ENABLE ROW LEVEL SECURITY;
ALTER TABLE atividade            ENABLE ROW LEVEL SECURITY;
ALTER TABLE plano_alimentar      ENABLE ROW LEVEL SECURITY;
ALTER TABLE refeicao             ENABLE ROW LEVEL SECURITY;
ALTER TABLE conversa             ENABLE ROW LEVEL SECURITY;
ALTER TABLE participante_conversa ENABLE ROW LEVEL SECURITY;
ALTER TABLE mensagem             ENABLE ROW LEVEL SECURITY;
```

```
-- usuario: o titular lê/edita o próprio registro; contas ATIVAS ficam visíveis
-- para busca de destinatário (UC-05 A1), mas o peso só é legível decifrando com a chave.
CREATE POLICY pol_usuario_sel ON usuario FOR SELECT TO gymgramps_app
  USING (id_usuario = usuario_corrente() OR status_conta = 'ATIVA');
CREATE POLICY pol_usuario_upd ON usuario FOR UPDATE TO gymgramps_app
  USING (id_usuario = usuario_corrente()) WITH CHECK (id_usuario = usuario_corrente());
```

```
CREATE POLICY pol_atividade_titular ON atividade FOR ALL TO gymgramps_app
  USING (id_usuario = usuario_corrente()) WITH CHECK (id_usuario = usuario_corrente());
```

```
CREATE POLICY pol_plano_titular ON plano_alimentar FOR ALL TO gymgramps_app
  USING (id_usuario = usuario_corrente()) WITH CHECK (id_usuario = usuario_corrente());
```

```
CREATE POLICY pol_refeicao_titular ON refeicao FOR ALL TO gymgramps_app
  USING (EXISTS (SELECT 1 FROM plano_alimentar p
                 WHERE p.id_plano = refeicao.id_plano
                   AND p.id_usuario = usuario_corrente()))
  WITH CHECK (EXISTS (SELECT 1 FROM plano_alimentar p
                 WHERE p.id_plano = refeicao.id_plano
                   AND p.id_usuario = usuario_corrente()));
```

```
CREATE POLICY pol_conversa_sel ON conversa FOR SELECT TO gymgramps_app
  USING (fn_participa(id_conversa));
CREATE POLICY pol_participante_sel ON participante_conversa FOR SELECT TO gymgramps_app
  USING (fn_participa(id_conversa));
CREATE POLICY pol_mensagem_sel ON mensagem FOR SELECT TO gymgramps_app
  USING (fn_participa(id_conversa));
CREATE POLICY pol_mensagem_ins ON mensagem FOR INSERT TO gymgramps_app
  WITH CHECK (id_remetente = usuario_corrente());
CREATE POLICY pol_mensagem_upd ON mensagem FOR UPDATE TO gymgramps_app
  USING (id_remetente = usuario_corrente()) WITH CHECK (id_remetente = usuario_corrente());
```

```
-- Privilégios mínimos para o papel da aplicação (conversa/participante só por rotina)
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM PUBLIC;
REVOKE EXECUTE ON FUNCTION criar_conversa(uuid, uuid), excluir_conta(uuid), fn_participa(uuid),
      usuario_corrente(), fn_cifrar(text), fn_decifrar(bytea) FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO gymgramps_app;
GRANT SELECT, UPDATE         ON usuario               TO gymgramps_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON atividade, plano_alimentar, refeicao TO gymgramps_app;
GRANT SELECT                 ON conversa, participante_conversa TO gymgramps_app;
GRANT SELECT, INSERT, UPDATE ON mensagem               TO gymgramps_app;
GRANT SELECT ON vw_plano_alimentar, vw_refeicao, vw_mensagem_historico TO gymgramps_app;
GRANT EXECUTE ON FUNCTION usuario_corrente(), fn_cifrar(text), fn_decifrar(bytea),
      fn_participa(uuid), criar_conversa(uuid, uuid), excluir_conta(uuid)
  TO gymgramps_app;
```

```
-- ---------------------------------------------------------------------------
-- 10. Documentação no catálogo (\d+ e \dt+ no psql)
-- ---------------------------------------------------------------------------
COMMENT ON TABLE usuario              IS 'Praticante com conta no GymGramps (classe Usuario)';
COMMENT ON TABLE atividade            IS 'Treino/atividade agendada na fita de 24h (classe Atividade)';
COMMENT ON TABLE plano_alimentar      IS 'Plano alimentar vigente do usuário, no máximo 1 (classe PlanoAlimentar)';
COMMENT ON TABLE refeicao             IS 'Refeição que compõe o plano (composição 1..*)';
COMMENT ON TABLE conversa             IS 'Canal de chat entre exatamente 2 usuários ativos';
COMMENT ON TABLE participante_conversa IS 'Classe associativa Usuário-Conversa: o usuário dentro da conversa';
COMMENT ON TABLE mensagem             IS 'Mensagem enviada por um participante (FK composta para participante_conversa)';
COMMENT ON COLUMN usuario.peso_cif    IS 'Dado sensível (RN-02): peso em kg cifrado com fn_cifrar';
COMMENT ON COLUMN usuario.senha_hash  IS 'Hash bcrypt. No Supabase fica em auth.users e esta coluna é dispensada';
COMMENT ON COLUMN atividade.data      IS 'Dia da atividade; hora_fim > hora_inicio garante que não cruza a meia-noite';
```

14. Carga Inicial do Banco de Dados (DML)

14.1. Estratégia e conteúdo da carga

A carga inicial popula todas as tabelas com dados fictícios, suficientes para demonstrar cada caso de uso, e inclui os casos de borda previstos nos fluxos de exceção. Os e-mails usam o domínio `example.com`, reservado para exemplos. O script inteiro roda em **uma única transação**: ou entra tudo, ou nada, e as restrições adiadas (2 participantes por conversa; plano com ao menos 1 refeição) são conferidas no *COMMIT*.

Tabela	Linhas	Conteúdo e finalidade
usuario	6	5 contas ATIVAS e 1 INATIVA (Felipe, inativada por UPDATE); peso cifrado; senha de demonstração em bcrypt; temas CLARO e ESCURO.
atividade	16	Treinos de 08/10 a 13/10/2026 dos 6 usuários. Carla tem duas atividades "encostadas" (08:00–09:00 e 09:00–10:00), permitidas pela EXCLUDE.
plano_alimentar	4	Planos de Ana, Bruno, Carla e Diego. Elisa e Felipe ficam sem plano (UC-04, fluxo E1). Todos os campos sensíveis cifrados.
refeicao	16	4 refeições por plano (café da manhã, almoço, lanche, jantar), inseridas na mesma transação do plano (regra 1..*).
conversa	3	Ana–Bruno, Ana–Carla e Bruno–Diego, com identificadores fixos para facilitar a referência.
participante_conversa	6	Duas participações por conversa (classe associativa), criadas junto com a conversa.
mensagem	11	Histórico das três conversas; uma mensagem começa como NAO_ENVIADA (UC-05, E2) e é reenviada por UPDATE.

Atualizações (UPDATE) incluídas na carga:

- conclusão das atividades de dias já passados (UC-03);
- reagendamento de uma atividade, que a EXCLUDE revalida (HIIT do Bruno, 12:00–12:40 para 12:15–12:55);
- troca de tema no perfil (RF-06);
- ajuste da meta de calorias do plano, com os dados sensíveis recifrados (RN-02);
- reenvio de mensagem que havia falhado (UC-05, E2);
- inativação da conta do Felipe, que passa a ficar fora de conversas (RN-01).

Atenção. *A chave de cifragem (*`chave-demo-gymgramps-nao-usar-em-producao`*) e a senha de demonstração existem só para esta carga de testes. Em produção a chave vem de um gerenciador de segredos e as senhas são cadastradas pelo próprio usuário (UC-01).*

14.2. Script 02 — DML da carga inicial

```
-- ============================================================================
-- GymGramps - Fase C-5 - Script 02: carga inicial (DML)
-- SGBDR: PostgreSQL 16  |  Pré-requisito: 01_ddl.sql executado
-- Uso:   psql -U postgres -d gymgramps -v ON_ERROR_STOP=1 -f 02_dml.sql
-- A carga inteira roda em UMA transação: ou entra tudo, ou nada. As restrições
-- adiadas (2 participantes por conversa; plano com ao menos 1 refeição) são
-- verificadas no COMMIT.
-- ============================================================================
BEGIN;
```

```
-- Chave de cifragem DE DEMONSTRAÇÃO (apenas para esta sessão). Em produção ela
-- vem de um gerenciador de segredos e nunca é gravada em script.
SET gymgramps.chave_cifra = 'chave-demo-gymgramps-nao-usar-em-producao';
```

```
-- ----------------------------------------------------------------------------
-- 1. Usuários (senha de demonstração em bcrypt; peso cifrado)
-- ----------------------------------------------------------------------------
INSERT INTO usuario (email, nome, data_nascimento, peso_cif, senha_hash, tema, status_conta)
SELECT v.email, v.nome, v.nascimento::date, fn_cifrar(v.peso),
       crypt('Demo@2026', gen_salt('bf', 10)), v.tema::tema_t, 'ATIVA'
  FROM (VALUES
    ('ana.lima@example.com',       'Ana Beatriz Lima',    '2002-03-14', '62.50', 'ESCURO'),
    ('bruno.carvalho@example.com', 'Bruno Carvalho',      '1999-07-21', '82.30', 'CLARO'),
    ('carla.menezes@example.com',  'Carla Menezes',       '2001-11-05', '58.00', 'CLARO'),
    ('diego.ramos@example.com',    'Diego Ramos',         '1997-01-30', '91.70', 'ESCURO'),
    ('elisa.teixeira@example.com', 'Elisa Teixeira',      '2004-09-18', '54.20', 'CLARO'),
    ('felipe.andrade@example.com', 'Felipe Andrade',      '2000-05-09', '76.00', 'CLARO')
  ) AS v(email, nome, nascimento, peso, tema);
```

```
-- ----------------------------------------------------------------------------
-- 2. Atividades (fita de 24h). Atenção: Carla tem duas atividades "encostadas"
--    (09:00 termina uma e começa a outra), o que a restrição EXCLUDE permite.
-- ----------------------------------------------------------------------------
INSERT INTO atividade (id_usuario, data, hora_inicio, hora_fim, titulo, tipo)
SELECT u.id_usuario, v.data::date, v.ini::time, v.fim::time, v.titulo, v.tipo::tipo_atividade_t
  FROM (VALUES
    ('ana.lima@example.com',       '2026-10-08', '18:00', '19:00', 'Costas e bíceps',            'MUSCULACAO'),
    ('ana.lima@example.com',       '2026-10-09', '07:00', '08:00', 'Treino de pernas',           'MUSCULACAO'),
    ('ana.lima@example.com',       '2026-10-12', '06:30', '07:30', 'Corrida leve',               'CARDIO'),
    ('ana.lima@example.com',       '2026-10-12', '18:00', '19:00', 'Peito e tríceps',            'MUSCULACAO'),
    ('ana.lima@example.com',       '2026-10-12', '19:00', '19:15', 'Alongamento final',          'ALONGAMENTO'),
    ('bruno.carvalho@example.com', '2026-10-08', '19:00', '20:00', 'Corrida no parque',          'CARDIO'),
    ('bruno.carvalho@example.com', '2026-10-12', '07:00', '08:00', 'Superiores (push)',          'MUSCULACAO'),
    ('bruno.carvalho@example.com', '2026-10-12', '12:00', '12:40', 'HIIT',                       'CARDIO'),
    ('bruno.carvalho@example.com', '2026-10-13', '07:00', '08:00', 'Pernas',                     'MUSCULACAO'),
    ('carla.menezes@example.com',  '2026-10-12', '08:00', '09:00', 'Alongamento e mobilidade',   'ALONGAMENTO'),
    ('carla.menezes@example.com',  '2026-10-12', '09:00', '10:00', 'Bicicleta ergométrica',      'CARDIO'),
    ('carla.menezes@example.com',  '2026-10-13', '17:30', '18:30', 'Treino full body',           'MUSCULACAO'),
    ('diego.ramos@example.com',    '2026-10-12', '05:45', '06:45', 'Peito e ombros',             'MUSCULACAO'),
    ('diego.ramos@example.com',    '2026-10-12', '21:00', '21:30', 'Caminhada',                  'OUTRO'),
    ('elisa.teixeira@example.com', '2026-10-12', '16:00', '17:00', 'Treino full body',           'MUSCULACAO'),
    ('felipe.andrade@example.com', '2026-10-05', '18:00', '19:00', 'Treino de pernas',           'MUSCULACAO')
  ) AS v(email, data, ini, fim, titulo, tipo)
  JOIN usuario u ON u.email = v.email;
```

```
-- ----------------------------------------------------------------------------
-- 3. Planos alimentares (Elisa e Felipe ficam sem plano: UC-04, fluxo E1) e
--    refeições. Plano e refeições entram na mesma transação (regra 1..*).
-- ----------------------------------------------------------------------------
INSERT INTO plano_alimentar (id_usuario, data_inicio, objetivo_cif, calorias_diarias_cif,
                             proteinas_g_cif, carboidratos_g_cif, gorduras_g_cif)
SELECT u.id_usuario, v.inicio::date, fn_cifrar(v.objetivo), fn_cifrar(v.kcal),
       fn_cifrar(v.prot), fn_cifrar(v.carb), fn_cifrar(v.gord)
  FROM (VALUES
    ('ana.lima@example.com',       '2026-10-01', 'Hipertrofia com leve superávit', '2400', '150.0', '300.0', '70.0'),
    ('bruno.carvalho@example.com', '2026-10-01', 'Emagrecimento com ganho de massa', '2000', '160.0', '180.0', '60.0'),
    ('carla.menezes@example.com',  '2026-10-01', 'Manutenção de peso',              '1900', '100.0', '240.0', '55.0'),
    ('diego.ramos@example.com',    '2026-10-01', 'Hipertrofia',                     '3000', '190.0', '380.0', '85.0')
  ) AS v(email, inicio, objetivo, kcal, prot, carb, gord)
  JOIN usuario u ON u.email = v.email;
```

```
INSERT INTO refeicao (id_plano, nome, horario, descricao_cif, calorias_cif)
SELECT p.id_plano, v.nome, v.horario::time, fn_cifrar(v.descricao), fn_cifrar(v.kcal)
  FROM (VALUES
    ('ana.lima@example.com',       'Café da manhã',   '07:30', 'Ovos mexidos, pão integral e fruta',       '550'),
    ('ana.lima@example.com',       'Almoço',          '12:30', 'Arroz, feijão, frango grelhado e salada',  '750'),
    ('ana.lima@example.com',       'Lanche da tarde', '16:00', 'Iogurte com aveia e banana',               '350'),
    ('ana.lima@example.com',       'Jantar',          '20:00', 'Omelete de legumes e batata-doce',         '600'),
    ('bruno.carvalho@example.com', 'Café da manhã',   '07:00', 'Tapioca com queijo e café sem açúcar',     '400'),
    ('bruno.carvalho@example.com', 'Almoço',          '12:30', 'Frango com batata-doce e brócolis',        '650'),
    ('bruno.carvalho@example.com', 'Lanche da tarde', '16:30', 'Whey protein com fruta',                   '250'),
    ('bruno.carvalho@example.com', 'Jantar',          '20:00', 'Peixe assado com legumes',                 '500'),
    ('carla.menezes@example.com',  'Café da manhã',   '08:00', 'Pão integral com ovo e mamão',             '420'),
    ('carla.menezes@example.com',  'Almoço',          '12:00', 'Macarrão integral com carne moída',        '650'),
    ('carla.menezes@example.com',  'Lanche da tarde', '16:00', 'Castanhas e fruta',                        '250'),
    ('carla.menezes@example.com',  'Jantar',          '19:30', 'Sopa de legumes com frango',               '400'),
    ('diego.ramos@example.com',    'Café da manhã',   '06:30', 'Panqueca de aveia, ovos e banana',         '700'),
    ('diego.ramos@example.com',    'Almoço',          '12:30', 'Arroz, feijão, carne e legumes',           '950'),
    ('diego.ramos@example.com',    'Lanche da tarde', '16:00', 'Sanduíche de frango e suco natural',       '600'),
    ('diego.ramos@example.com',    'Jantar',          '20:30', 'Massa com atum e salada',                  '750')
  ) AS v(email, nome, horario, descricao, kcal)
  JOIN usuario u        ON u.email = v.email
  JOIN plano_alimentar p ON p.id_usuario = u.id_usuario;
```

```
-- ----------------------------------------------------------------------------
-- 4. Conversas: Conversa + as duas ParticipanteConversa (UC-05, fluxo A1) e mensagens.
--    Identificadores fixos para as conversas, para facilitar as referências.
-- ----------------------------------------------------------------------------
INSERT INTO conversa (id_conversa, data_criacao) VALUES
  ('c1000000-0000-4000-8000-000000000001', '2026-10-08 18:40:00-03'),   -- Ana e Bruno
  ('c1000000-0000-4000-8000-000000000002', '2026-10-07 20:00:00-03'),   -- Ana e Carla
  ('c1000000-0000-4000-8000-000000000003', '2026-10-06 21:25:00-03');   -- Bruno e Diego
```

```
INSERT INTO participante_conversa (id_conversa, id_usuario, data_entrada)
SELECT v.conversa::uuid, u.id_usuario, v.entrada::timestamptz
  FROM (VALUES
    ('c1000000-0000-4000-8000-000000000001', 'ana.lima@example.com',       '2026-10-08 18:40:00-03'),
    ('c1000000-0000-4000-8000-000000000001', 'bruno.carvalho@example.com', '2026-10-08 18:40:00-03'),
    ('c1000000-0000-4000-8000-000000000002', 'ana.lima@example.com',       '2026-10-07 20:00:00-03'),
    ('c1000000-0000-4000-8000-000000000002', 'carla.menezes@example.com',  '2026-10-07 20:00:00-03'),
    ('c1000000-0000-4000-8000-000000000003', 'bruno.carvalho@example.com', '2026-10-06 21:25:00-03'),
    ('c1000000-0000-4000-8000-000000000003', 'diego.ramos@example.com',    '2026-10-06 21:25:00-03')
  ) AS v(conversa, email, entrada)
  JOIN usuario u ON u.email = v.email;
```

```
INSERT INTO mensagem (id_conversa, id_remetente, data_hora_envio, conteudo, status)
SELECT v.conversa::uuid, u.id_usuario, v.enviada::timestamptz, v.conteudo, v.status::status_mensagem_t
  FROM (VALUES
    ('c1000000-0000-4000-8000-000000000001', 'ana.lima@example.com',
     '2026-10-08 18:42:00-03', 'Fala, Bruno! Bora treinar de manhã amanhã?', 'ENVIADA'),
    ('c1000000-0000-4000-8000-000000000001', 'bruno.carvalho@example.com',
     '2026-10-08 18:45:00-03', 'Bora! Chego às 7h na academia.', 'ENVIADA'),
    ('c1000000-0000-4000-8000-000000000001', 'ana.lima@example.com',
     '2026-10-08 18:46:00-03', 'Fechado. Levo a planilha de cargas.', 'ENVIADA'),
    ('c1000000-0000-4000-8000-000000000001', 'bruno.carvalho@example.com',
     '2026-10-09 07:12:00-03', 'Cheguei, te espero na entrada.', 'ENVIADA'),
    ('c1000000-0000-4000-8000-000000000001', 'ana.lima@example.com',
     '2026-10-09 07:13:00-03', 'Já estou subindo!', 'NAO_ENVIADA'),
    ('c1000000-0000-4000-8000-000000000002', 'carla.menezes@example.com',
     '2026-10-07 20:05:00-03', 'Ana, que horas é o alongamento de segunda?', 'ENVIADA'),
    ('c1000000-0000-4000-8000-000000000002', 'ana.lima@example.com',
     '2026-10-07 20:09:00-03', 'Às 19h, logo depois do treino de peito.', 'ENVIADA'),
    ('c1000000-0000-4000-8000-000000000002', 'carla.menezes@example.com',
     '2026-10-07 20:10:00-03', 'Perfeito, vou no mesmo horário.', 'ENVIADA'),
    ('c1000000-0000-4000-8000-000000000003', 'diego.ramos@example.com',
     '2026-10-06 21:30:00-03', 'Bruno, me passa a receita do frango com batata-doce?', 'ENVIADA'),
    ('c1000000-0000-4000-8000-000000000003', 'bruno.carvalho@example.com',
     '2026-10-06 21:34:00-03', 'Mando amanhã, já anotei no meu plano.', 'ENVIADA'),
    ('c1000000-0000-4000-8000-000000000003', 'diego.ramos@example.com',
     '2026-10-06 21:35:00-03', 'Valeu!', 'ENVIADA')
  ) AS v(conversa, email, enviada, conteudo, status)
  JOIN usuario u ON u.email = v.email;
```

```
-- ----------------------------------------------------------------------------
-- 5. Atualizações (UPDATE) que refletem o uso do sistema
-- ----------------------------------------------------------------------------
-- 5.1 Atividades de dias que já passaram foram concluídas (UC-03)
UPDATE atividade SET concluida = true WHERE data < DATE '2026-10-09';
```

```
-- 5.2 Reagendamento: o HIIT do Bruno passa para 12:15-12:55 (a EXCLUDE valida o novo intervalo)
UPDATE atividade
   SET hora_inicio = '12:15', hora_fim = '12:55'
 WHERE titulo = 'HIIT'
   AND id_usuario = (SELECT id_usuario FROM usuario WHERE email = 'bruno.carvalho@example.com');
```

```
-- 5.3 Perfil: troca de tema (UC-06 / RF-06)
UPDATE usuario SET tema = 'ESCURO' WHERE email = 'bruno.carvalho@example.com';
```

```
-- 5.4 Ajuste de meta do plano do Bruno (dados sensíveis recifrados)
UPDATE plano_alimentar
   SET calorias_diarias_cif = fn_cifrar('2200'),
       objetivo_cif         = fn_cifrar('Emagrecimento com ganho de massa (meta ajustada)')
 WHERE id_usuario = (SELECT id_usuario FROM usuario WHERE email = 'bruno.carvalho@example.com');
```

```
-- 5.5 Reenvio de mensagem que havia falhado (UC-05, fluxo E2)
UPDATE mensagem SET status = 'ENVIADA' WHERE status = 'NAO_ENVIADA';
```

```
-- 5.6 Felipe pediu a inativação da conta: sai de rotina, alimentação e chat (RN-01)
UPDATE usuario SET status_conta = 'INATIVA' WHERE email = 'felipe.andrade@example.com';
```

```
COMMIT;
```

```
-- ----------------------------------------------------------------------------
-- 6. Conferência da carga
-- ----------------------------------------------------------------------------
SELECT 'usuario' AS tabela, count(*) AS linhas FROM usuario
UNION ALL SELECT 'atividade',           count(*) FROM atividade
UNION ALL SELECT 'plano_alimentar',     count(*) FROM plano_alimentar
UNION ALL SELECT 'refeição',            count(*) FROM refeicao
UNION ALL SELECT 'conversa',            count(*) FROM conversa
UNION ALL SELECT 'participante_conversa', count(*) FROM participante_conversa
UNION ALL SELECT 'mensagem',            count(*) FROM mensagem;
```

14.3. Resultado da carga

Saída do `psql` para o script 02 (todas as instruções aceitas e conferência final de linhas por tabela). Os valores "UPDATE" correspondem, na ordem, aos itens 5.1 a 5.6 do script (3 atividades concluídas e 1 em cada um dos demais).

```
BEGIN
SET
INSERT 0 6
INSERT 0 16
INSERT 0 4
INSERT 0 16
INSERT 0 3
INSERT 0 6
INSERT 0 11
UPDATE 3
UPDATE 1
UPDATE 1
UPDATE 1
UPDATE 1
UPDATE 1
COMMIT
        tabela         | linhas
-----------------------+--------
 usuario               |      6
 atividade             |     16
 plano_alimentar       |      4
 refeição              |     16
 conversa              |      3
 participante_conversa |      6
 mensagem              |     11
(7 rows)
```

15. Testes e Verificação

15.1. Metodologia

Os quatro scripts foram executados em sequência sobre um servidor PostgreSQL 16.15 recém-iniciado: o banco foi criado, o esquema montado com `ON_ERROR_STOP=1` (nenhum erro), a carga inicial aceita sem erros e, por fim, o script 03 submeteu ao banco 25 testes. Os erros do script 03 são **esperados**: cada um prova que uma regra foi barrada pelo SGBD (20 erros no total, mais os casos que devem ser aceitos). Os testes que dependem do *COMMIT* (T09, T10, T11 e T14) usam transação explícita, pois essas restrições são verificadas ao final dela.

15.2. Testes de integridade e resultado obtido

Cada linha traz o teste, a regra verificada e a resposta real do PostgreSQL 16.15 (mensagens em inglês são as do próprio SGBD; as em português vêm das funções e gatilhos do esquema).

Nº	O que se testa	Resultado obtido (mensagem do PostgreSQL)
T01	E-mail já cadastrado	ERROR: duplicate key value violates unique constraint "uq_usuario_email"
T02	E-mail com formato inválido	ERROR: violates check constraint "ck_usuario_email"
T03	Tema fora do domínio (AZUL)	ERROR: invalid input value for enum tema_t: "AZUL"
T04	Atividade 08:30–09:30 sobreposta a 08:00–09:00 (mesmo usuário e dia)	ERROR: conflicting key value violates exclusion constraint "ex_atividade_sobreposicao"
T05	hora_fim anterior a hora_inicio	ERROR: violates check constraint "ck_atividade_horario"
T06	Mesma chave natural (usuário, data, hora_inicio)	ERROR: duplicate key value violates unique constraint "uq_atividade"
T07	Atividade encostada (07:30 logo após 06:30–07:30) deve ser aceita	INSERT 0 1 (aceita; desfeita com ROLLBACK)
T08	Segundo plano alimentar para o mesmo usuário	ERROR: duplicate key value violates unique constraint "uq_plano_usuario"
T09	Plano sem nenhuma refeição (no COMMIT)	ERROR: Um plano alimentar deve ter ao menos 1 refeição

Nº	O que se testa	Resultado obtido (mensagem do PostgreSQL)
T10	Remoção da última refeição de um plano (no COMMIT)	ERROR: Um plano alimentar deve ter ao menos 1 refeição
T11	Conversa com apenas 1 participante (no COMMIT)	ERROR: Uma conversa deve ter exatamente 2 participantes
T12	Terceiro participante na mesma conversa	ERROR: A conversa já possui 2 participantes
T13	Conta INATIVA em uma nova conversa	ERROR: Somente contas ATIVAS participam de conversas (RN-01)
T14	Remoção de um participante, deixando 1 (no COMMIT)	ERROR: Uma conversa deve ter exatamente 2 participantes
T15	Mensagem de quem não participa da conversa	ERROR: violates foreign key constraint "fk_mensagem_participante"
T16	Mensagem só com espaços	ERROR: violates check constraint "ck_mensagem_conteudo"
T17	Remetente com conta INATIVA envia mensagem	ERROR: Somente contas ATIVAS enviam mensagens (RN-01)
T18	Cifrar sem chave configurada	ERROR: Chave de cifragem não configurada (gymgramps.chave_cifra)
T19	Peso armazenado cifrado e decifrado com a chave	71 bytes cifrados; decifrado: 62.50 kg
T20	UC-04: plano e refeições decifrados pelas visões	Plano de 2400 kcal e 4 refeições (Seção 15.3)
T21	UC-03: fita do dia 12/10 em ordem de hora_inicio	3 atividades ordenadas (Seção 15.3)
T22	UC-05: histórico da conversa com remetente derivado	5 mensagens em ordem cronológica (Seção 15.3)
T23	EXPLAIN das consultas dos UC-03 e UC-05	Usa uq_atividade, ix_mensagem_historico e ix_participante_usuario
T24	RLS e privilégios com o papel gymgramps_app (sessão da Ana)	Vê só seus dados (5, 1, 4, 2 e 8 linhas); UPDATE 0; violação de RLS; permission denied; "Você só pode excluir a sua própria conta"
T25	UC-06: exclusão definitiva de um usuário de teste com atividade, plano, refeição, conversa e mensagem	Contagens voltam ao estado anterior à criação (Seção 15.3)

15.3. Consultas dos casos de uso e resultados

T19 — dado sensível cifrado e decifrado

```
       nome       | bytes_cifrados | peso_kg
------------------+----------------+---------
 Ana Beatriz Lima |             71 |   62.50
```

T20 — UC-04: plano alimentar e refeições (visões que decifram)

```
            objetivo            | calorias_diarias | proteinas_g | carboidratos_g | gorduras_g
--------------------------------+------------------+-------------+----------------+------------
 Hipertrofia com leve superávit |             2400 |       150.0 |          300.0 |       70.0

      nome       | horario  | calorias |                descricao
-----------------+----------+----------+-----------------------------------------
 Café da manhã   | 07:30:00 |      550 | Ovos mexidos, pão integral e fruta
 Almoço          | 12:30:00 |      750 | Arroz, feijão, frango grelhado e salada
 Lanche da tarde | 16:00:00 |      350 | Iogurte com aveia e banana
 Jantar          | 20:00:00 |      600 | Omelete de legumes e batata-doce
```

T21 — UC-03: fita do dia

```
 hora_inicio | hora_fim |      titulo       |    tipo     | concluida
-------------+----------+-------------------+-------------+-----------
 06:30:00    | 07:30:00 | Corrida leve      | CARDIO      | f
 18:00:00    | 19:00:00 | Peito e tríceps   | MUSCULACAO  | f
 19:00:00    | 19:15:00 | Alongamento final | ALONGAMENTO | f
```

T22 — UC-05: histórico da conversa

```
    data_hora_envio     |    remetente     |                  conteudo                  | status
------------------------+------------------+--------------------------------------------+---------
 2026-10-08 18:42:00-03 | Ana Beatriz Lima | Fala, Bruno! Bora treinar de manhã amanhã? | ENVIADA
 2026-10-08 18:45:00-03 | Bruno Carvalho   | Bora! Chego às 7h na academia.             | ENVIADA
 2026-10-08 18:46:00-03 | Ana Beatriz Lima | Fechado. Levo a planilha de cargas.        | ENVIADA
 2026-10-09 07:12:00-03 | Bruno Carvalho   | Cheguei, te espero na entrada.             | ENVIADA
 2026-10-09 07:13:00-03 | Ana Beatriz Lima | Já estou subindo!                          | ENVIADA
```

T23 — planos de execução (uso de índices)

```
                              QUERY PLAN
----------------------------------------------------------------------
 Index Scan using uq_atividade on atividade
   Index Cond: ((id_usuario = $0) AND (data = '2026-10-12'::date))
   InitPlan 1 (returns $0)
     ->  Index Scan using uq_usuario_email on usuario
           Index Cond: ((email)::text = 'ana.lima@example.com'::text)

                                    QUERY PLAN
-------------------------------------------------------------------------------------------
 Sort
   Sort Key: data_hora_envio
   ->  Bitmap Heap Scan on mensagem
         Recheck Cond: (id_conversa = 'c1000000-0000-4000-8000-000000000001'::uuid)
         ->  Bitmap Index Scan on ix_mensagem_historico
               Index Cond: (id_conversa = 'c1000000-0000-4000-8000-000000000001'::uuid)

                              QUERY PLAN
----------------------------------------------------------------------
 Bitmap Heap Scan on participante_conversa
   Recheck Cond: (id_usuario = $0)
   InitPlan 1 (returns $0)
     ->  Index Scan using uq_usuario_email on usuario
           Index Cond: ((email)::text = 'ana.lima@example.com'::text)
   ->  Bitmap Index Scan on ix_participante_usuario
         Index Cond: (id_usuario = $0)
```

T24 — RLS e privilégios (papel gymgramps_app, sessão da Ana)

```
 minhas_atividades | meus_planos | minhas_refeicoes | minhas_conversas | mensagens_visiveis
-------------------+-------------+------------------+------------------+--------------------
                 5 |           1 |                4 |                2 |                  8

-- Ana tenta alterar a atividade do Bruno (0 linhas):
UPDATE 0
-- Ana tenta enviar mensagem como se fosse o Bruno:
ERROR:  new row violates row-level security policy for table "mensagem"
-- Ana tenta apagar usuário direto (sem privilégio):
ERROR:  permission denied for table usuario
-- Ana tenta excluir a conta do Bruno pela rotina:
ERROR:  Você só pode excluir a sua própria conta
```

T25 — UC-06: exclusão definitiva em cascata

```
 momento | usuarios | atividades | planos | refeicoes | conversas | particip | mensagens
---------+----------+------------+--------+-----------+-----------+----------+-----------
 ANTES   |        7 |         17 |      5 |        17 |         4 |        8 |        12

 excluir_conta
---------------


 momento | usuarios | atividades | planos | refeicoes | conversas | particip | mensagens
---------+----------+------------+--------+-----------+-----------+----------+-----------
 DEPOIS  |        6 |         16 |      4 |        16 |         3 |        6 |        11
```

15.4. Script 03 — testes de integridade e de consultas

Executado sem `ON_ERROR_STOP`, pois os erros são esperados.

```
-- ============================================================================
-- GymGramps - Fase C-5 - Script 03: testes de integridade e de consultas
-- Executar após 01_ddl.sql e 02_dml.sql (SEM ON_ERROR_STOP: os erros são esperados).
-- ============================================================================
\set VERBOSITY terse
\pset pager off
\pset null '(nulo)'
RESET gymgramps.chave_cifra;
```

```
\echo '== T01 e-mail duplicado (uq_usuario_email)'
INSERT INTO usuario (email, nome, data_nascimento, peso_cif, senha_hash)
VALUES ('ana.lima@example.com', 'Outra Ana', '2000-01-01', '\x01', 'x');
```

```
\echo '== T02 e-mail com formato inválido (ck_usuario_email)'
INSERT INTO usuario (email, nome, data_nascimento, peso_cif, senha_hash)
VALUES ('sem-arroba', 'Fulano', '2000-01-01', '\x01', 'x');
```

```
\echo '== T03 tema fora do domínio (enum tema_t)'
INSERT INTO usuario (email, nome, data_nascimento, peso_cif, senha_hash, tema)
VALUES ('azul@example.com', 'Fulano', '2000-01-01', '\x01', 'x', 'AZUL');
```

```
\echo '== T04 atividade sobreposta no mesmo dia (ex_atividade_sobreposicao)'
INSERT INTO atividade (id_usuario, data, hora_inicio, hora_fim, titulo, tipo)
SELECT id_usuario, '2026-10-12', '08:30', '09:30', 'Conflito', 'OUTRO'
  FROM usuario WHERE email = 'carla.menezes@example.com';
```

```
\echo '== T05 hora_fim <= hora_inicio (ck_atividade_horario)'
INSERT INTO atividade (id_usuario, data, hora_inicio, hora_fim, titulo, tipo)
SELECT id_usuario, '2026-10-14', '10:00', '09:00', 'Invertida', 'OUTRO'
  FROM usuario WHERE email = 'carla.menezes@example.com';
```

```
\echo '== T06 mesma chave natural (usuario, data, hora_inicio) (uq_atividade)'
INSERT INTO atividade (id_usuario, data, hora_inicio, hora_fim, titulo, tipo)
SELECT id_usuario, '2026-10-12', '06:30', '06:45', 'Duplicada', 'OUTRO'
  FROM usuario WHERE email = 'ana.lima@example.com';
```

```
\echo '== T07 atividade encostada é PERMITIDA (07:30 logo após 06:30-07:30)'
BEGIN;
INSERT INTO atividade (id_usuario, data, hora_inicio, hora_fim, titulo, tipo)
SELECT id_usuario, '2026-10-12', '07:30', '08:00', 'Encostada', 'OUTRO'
  FROM usuario WHERE email = 'ana.lima@example.com';
ROLLBACK;
```

```
\echo '== T08 segundo plano alimentar para o mesmo usuário (uq_plano_usuario)'
INSERT INTO plano_alimentar (id_usuario, data_inicio, calorias_diarias_cif, proteinas_g_cif,
                             carboidratos_g_cif, gorduras_g_cif)
SELECT id_usuario, '2026-10-09', '\x01', '\x01', '\x01', '\x01'
  FROM usuario WHERE email = 'ana.lima@example.com';
```

```
\echo '== T09 plano sem nenhuma refeição, verificado no COMMIT'
SET gymgramps.chave_cifra = 'chave-demo-gymgramps-nao-usar-em-producao';
BEGIN;
INSERT INTO plano_alimentar (id_usuario, data_inicio, calorias_diarias_cif, proteinas_g_cif,
                             carboidratos_g_cif, gorduras_g_cif)
SELECT id_usuario, '2026-10-09', fn_cifrar('1800'), fn_cifrar('90'), fn_cifrar('200'), fn_cifrar('50')
  FROM usuario WHERE email = 'elisa.teixeira@example.com';
COMMIT;
```

```
\echo '== T10 remover a última refeição de um plano, verificado no COMMIT'
BEGIN;
DELETE FROM refeicao
 WHERE id_plano = (SELECT p.id_plano FROM plano_alimentar p JOIN usuario u USING (id_usuario)
                   WHERE u.email = 'carla.menezes@example.com');
COMMIT;
```

```
\echo '== T11 conversa com apenas 1 participante, verificado no COMMIT'
BEGIN;
WITH c AS (INSERT INTO conversa DEFAULT VALUES RETURNING id_conversa)
INSERT INTO participante_conversa (id_conversa, id_usuario)
SELECT c.id_conversa, u.id_usuario FROM c, usuario u WHERE u.email = 'diego.ramos@example.com';
COMMIT;
```

```
\echo '== T12 terceiro participante na mesma conversa (tg_valida_participante)'
INSERT INTO participante_conversa (id_conversa, id_usuario)
SELECT 'c1000000-0000-4000-8000-000000000001', id_usuario FROM usuario WHERE email = 'diego.ramos@example.com';
```

```
\echo '== T13 conta INATIVA não entra em conversa (RN-01)'
SELECT criar_conversa((SELECT id_usuario FROM usuario WHERE email = 'ana.lima@example.com'),
                      (SELECT id_usuario FROM usuario WHERE email = 'felipe.andrade@example.com'));
```

```
\echo '== T14 remover um participante deixaria a conversa com 1, verificado no COMMIT'
BEGIN;
DELETE FROM participante_conversa
 WHERE id_conversa = 'c1000000-0000-4000-8000-000000000003'
   AND id_usuario = (SELECT id_usuario FROM usuario WHERE email = 'diego.ramos@example.com');
COMMIT;
```

```
\echo '== T15 remetente que não participa da conversa (FK composta fk_mensagem_participante)'
INSERT INTO mensagem (id_conversa, id_remetente, conteudo)
SELECT 'c1000000-0000-4000-8000-000000000001', id_usuario, 'Intruso'
  FROM usuario WHERE email = 'diego.ramos@example.com';
```

```
\echo '== T16 mensagem vazia (ck_mensagem_conteudo)'
INSERT INTO mensagem (id_conversa, id_remetente, conteudo)
SELECT 'c1000000-0000-4000-8000-000000000001', id_usuario, '   '
  FROM usuario WHERE email = 'ana.lima@example.com';
```

```
\echo '== T17 remetente com conta INATIVA não envia mensagem (RN-01)'
BEGIN;
UPDATE usuario SET status_conta = 'INATIVA' WHERE email = 'diego.ramos@example.com';
INSERT INTO mensagem (id_conversa, id_remetente, conteudo)
SELECT 'c1000000-0000-4000-8000-000000000003', id_usuario, 'Oi'
  FROM usuario WHERE email = 'diego.ramos@example.com';
ROLLBACK;
```

```
\echo '== T18 cifragem sem chave configurada'
RESET gymgramps.chave_cifra;
SELECT fn_cifrar('62.5');
SET gymgramps.chave_cifra = 'chave-demo-gymgramps-nao-usar-em-producao';
```

```
\echo '== T19 dado sensível armazenado cifrado (BYTEA) e decifrado com a chave'
SELECT u.nome, octet_length(u.peso_cif) AS bytes_cifrados,
       fn_decifrar(u.peso_cif)::numeric(5,2) AS peso_kg
  FROM usuario u WHERE u.email = 'ana.lima@example.com';
```

```
\echo '== T20 UC-04: plano e refeições decifrados (Ana)'
SELECT v.objetivo, v.calorias_diarias, v.proteinas_g, v.carboidratos_g, v.gorduras_g
  FROM vw_plano_alimentar v JOIN usuario u USING (id_usuario) WHERE u.email = 'ana.lima@example.com';
SELECT r.nome, r.horario, r.calorias, r.descricao
  FROM vw_refeicao r JOIN plano_alimentar p USING (id_plano) JOIN usuario u USING (id_usuario)
 WHERE u.email = 'ana.lima@example.com' ORDER BY r.horario;
```

```
\echo '== T21 UC-03: fita do dia 2026-10-12 da Ana (ordenada por hora_inicio)'
SELECT a.hora_inicio, a.hora_fim, a.titulo, a.tipo, a.concluida
  FROM atividade a JOIN usuario u USING (id_usuario)
 WHERE u.email = 'ana.lima@example.com' AND a.data = '2026-10-12' ORDER BY a.hora_inicio;
```

```
\echo '== T22 UC-05: histórico da conversa Ana-Bruno (remetente derivado da participação)'
SELECT data_hora_envio, remetente, conteudo, status
  FROM vw_mensagem_historico
 WHERE id_conversa = 'c1000000-0000-4000-8000-000000000001' ORDER BY data_hora_envio;
```

```
\echo '== T23 plano de execução: índices usados (seqscan desligado só para a demonstração)'
SET enable_seqscan = off;
EXPLAIN (COSTS OFF)
SELECT * FROM atividade
 WHERE id_usuario = (SELECT id_usuario FROM usuario WHERE email = 'ana.lima@example.com')
   AND data = '2026-10-12' ORDER BY hora_inicio;
EXPLAIN (COSTS OFF)
SELECT * FROM mensagem
 WHERE id_conversa = 'c1000000-0000-4000-8000-000000000001' ORDER BY data_hora_envio;
EXPLAIN (COSTS OFF)
SELECT id_conversa FROM participante_conversa
 WHERE id_usuario = (SELECT id_usuario FROM usuario WHERE email = 'ana.lima@example.com');
RESET enable_seqscan;
```

```
\echo '== T24 RLS: sessão da Ana (papel gymgramps_app) só enxerga os próprios dados'
SELECT id_usuario AS id_ana FROM usuario WHERE email = 'ana.lima@example.com' \gset
SET gymgramps.id_usuario = :'id_ana';
SET ROLE gymgramps_app;
SELECT (SELECT count(*) FROM atividade)        AS minhas_atividades,
       (SELECT count(*) FROM plano_alimentar)  AS meus_planos,
       (SELECT count(*) FROM refeicao)         AS minhas_refeicoes,
       (SELECT count(*) FROM conversa)         AS minhas_conversas,
       (SELECT count(*) FROM mensagem)         AS mensagens_visiveis;
\echo '-- Ana tenta alterar a atividade do Bruno (0 linhas):'
UPDATE atividade SET concluida = true WHERE titulo = 'Pernas';
\echo '-- Ana tenta enviar mensagem como se fosse o Bruno:'
INSERT INTO mensagem (id_conversa, id_remetente, conteudo)
SELECT 'c1000000-0000-4000-8000-000000000001', id_usuario, 'falsa' FROM usuario WHERE email = 'bruno.carvalho@example.com';
\echo '-- Ana tenta apagar usuário direto (sem privilégio):'
DELETE FROM usuario;
\echo '-- Ana tenta excluir a conta do Bruno pela rotina:'
SELECT excluir_conta((SELECT id_usuario FROM usuario WHERE email = 'bruno.carvalho@example.com'));
RESET ROLE;
RESET gymgramps.id_usuario;
```

```
\echo '== T25 UC-06 / RN-03: exclusão definitiva em cascata (usuário de teste)'
BEGIN;
INSERT INTO usuario (email, nome, data_nascimento, peso_cif, senha_hash)
VALUES ('gustavo.teste@example.com', 'Gustavo Teste', '2000-02-02', fn_cifrar('80'), crypt('x', gen_salt('bf', 4)));
INSERT INTO atividade (id_usuario, data, hora_inicio, hora_fim, titulo, tipo)
SELECT id_usuario, '2026-10-12', '10:00', '11:00', 'Teste', 'OUTRO' FROM usuario WHERE email = 'gustavo.teste@example.com';
INSERT INTO plano_alimentar (id_usuario, data_inicio, calorias_diarias_cif, proteinas_g_cif,
                             carboidratos_g_cif, gorduras_g_cif)
SELECT id_usuario, '2026-10-12', fn_cifrar('2000'), fn_cifrar('100'), fn_cifrar('250'), fn_cifrar('60')
  FROM usuario WHERE email = 'gustavo.teste@example.com';
INSERT INTO refeicao (id_plano, nome, horario, calorias_cif)
SELECT p.id_plano, 'Almoço', '12:00', fn_cifrar('700') FROM plano_alimentar p JOIN usuario u USING (id_usuario)
 WHERE u.email = 'gustavo.teste@example.com';
SELECT criar_conversa((SELECT id_usuario FROM usuario WHERE email = 'gustavo.teste@example.com'),
                      (SELECT id_usuario FROM usuario WHERE email = 'elisa.teixeira@example.com')) AS conversa_teste \gset
INSERT INTO mensagem (id_conversa, id_remetente, conteudo)
SELECT :'conversa_teste', id_usuario, 'olá' FROM usuario WHERE email = 'gustavo.teste@example.com';
COMMIT;
SELECT 'ANTES' AS momento, (SELECT count(*) FROM usuario) AS usuarios, (SELECT count(*) FROM atividade) AS atividades,
       (SELECT count(*) FROM plano_alimentar) AS planos, (SELECT count(*) FROM refeicao) AS refeicoes,
       (SELECT count(*) FROM conversa) AS conversas, (SELECT count(*) FROM participante_conversa) AS particip,
       (SELECT count(*) FROM mensagem) AS mensagens;
BEGIN;
SELECT excluir_conta((SELECT id_usuario FROM usuario WHERE email = 'gustavo.teste@example.com'));
COMMIT;
SELECT 'DEPOIS' AS momento, (SELECT count(*) FROM usuario) AS usuarios, (SELECT count(*) FROM atividade) AS atividades,
       (SELECT count(*) FROM plano_alimentar) AS planos, (SELECT count(*) FROM refeicao) AS refeicoes,
       (SELECT count(*) FROM conversa) AS conversas, (SELECT count(*) FROM participante_conversa) AS particip,
       (SELECT count(*) FROM mensagem) AS mensagens;
```

Resumo: *todas as regras da Tabela de integridade (Seção 13.3: UNIQUE, CHECK, ENUM, EXCLUDE, FK composta, gatilhos, cascata, cifragem e RLS) foram verificadas com violações reais e o banco permaneceu consistente depois delas; os índices secundários são utilizados pelas consultas dos casos de uso.*

Parte VI — Banco de Dados Não Relacional (Fase C-6)

Esta parte complementa o documento com o esquema físico do GymGramps em um banco de dados não convencional (não relacional). O SGBD escolhido é o MongoDB, orientado a documentos. O mesmo modelo conceitual da Seção 7 e as mesmas regras de negócio (RN-01 a RN-03) são remodelados como coleções de documentos BSON, acompanhados do script de definição de dados (coleções, validação de esquema, índices, visão e papel de acesso), da carga inicial e de dois diagramas que exemplificam a organização física dos dados. Os scripts desta parte foram executados do zero em um servidor MongoDB real, inclusive com casos de violação de cada regra (Seção 20).

Critério do enunciado (Fase C-6)	O que foi entregue	Seção	Arquivo / figura
Diagrama de exemplos — diagrama exemplificando a organização dos dados	Organização física das 5 coleções, dos subdocumentos embutidos e das referências; exemplo com documentos reais da carga	18	Figuras 9 e 10
Script de criação — definição de dados	SGBD, banco, coleções com validação $jsonSchema, chaves/índices (únicos, compostos, parciais, multichave), visão e papel	17	`00_banco.js`, `01_ddl.js`
Carga inicial — manipulação de dados	Inserções (6 usuários, 11 agendas diárias, 4 planos, 3 conversas, 11 mensagens) e 6 atualizações	19	`02_carga.js` (usa `lib_gymgramps.js`)
Verificação (complementar)	27 testes de integridade, de consultas dos casos de uso e de índices	20	`03_testes.js`

Os cinco arquivos de código desta parte (`00_banco.js`, `01_ddl.js`, `lib_gymgramps.js`, `02_carga.js` e `03_testes.js`) estão reproduzidos nas Seções 17, 19 e 20. Um sexto arquivo, `executar_tudo.sh`, apenas chama os quatro scripts principais em sequência.

16. Escolha do SGBD não relacional e do modelo de dados

16.1. Por que o MongoDB

Os bancos NoSQL se dividem em quatro famílias: chave-valor, documentos, colunares e grafos. O domínio do GymGramps tem agregados que são sempre lidos e gravados juntos: a agenda de um dia (UC-03), o plano alimentar com suas refeições (UC-04) e a conversa com seus dois participantes (UC-05). Isso aponta para o modelo de documentos. Um banco chave-valor guardaria esses agregados, mas não consultaria dentro deles nem criaria índices por campo; um colunar de grande escala (como o Cassandra) serve a volumes de escrita que este produto não tem; e um banco de grafos se justificaria para uma rede social com muitas relações, e o chat do GymGramps é sempre de um para um. A Seção 10.7 já apontava o chat como a parte mais próxima de um modelo orientado a agregados; nesta fase o domínio inteiro é remodelado em documentos, para que o esquema físico não relacional fique completo.

O MongoDB atende ao projeto por quatro motivos: (a) o documento BSON é praticamente o JSON que a SPA em React já manipula; (b) a validação declarativa de esquema ($jsonSchema e $expr) garante, no próprio banco, tipos, domínios e multiplicidades; (c) há índices únicos, compostos, parciais e multichave (sobre arrays), suficientes para as consultas dos casos de uso; (d) o pipeline de agregação ($lookup) permite expor uma visão com o remetente de cada mensagem, equivalente à associação derivada /remetente do modelo conceitual.

16.2. Decisões de modelagem: embutir ou referenciar

A regra seguida foi: embutir o que tem ciclo de vida igual ao do pai, é lido junto com ele e tem tamanho limitado; referenciar (guardar o ObjectId) o que é compartilhado ou cresce sem limite. As decisões abaixo transformam as 7 tabelas do esquema relacional (Seção 8) em 5 coleções e 1 visão.

Esquema relacional (Fase C-5)	MongoDB (Fase C-6)	Motivo
`usuario`	Coleção `usuarios` (documento raiz, referenciado por ObjectId)	Entidade raiz; todas as outras coleções apontam para ela.
`atividade` (N por usuário)	Array `atividades` embutido na coleção `agendas_dia`: um documento por usuário e por dia (padrão “balde”, bucket)	A fita de 24 h do UC-03 é lida com uma única consulta. O tamanho é limitado (validador: no máximo 288 atividades por dia). Como o dia inteiro está em um só documento, a não sobreposição pode ser checada e gravada de forma atômica.
`plano_alimentar` + `refeicao`	Coleção `planos_alimentares` com o array `refeicoes` embutido (1 a 12 itens)	Composição (a refeição não existe sem o plano) e sempre lida junto (UC-04). O mínimo de 1 refeição vira `minItems: 1`.
`conversa` + `participante_conversa`	Coleção `conversas` com o array `participantes` embutido (exatamente 2)	A classe associativa vira subdocumento. A lista de conversas de um usuário usa índice multichave sobre o array.
`mensagem`	Coleção própria `mensagens`, com `conversaId` e `remetenteId` (referências)	Cresce sem limite (um documento MongoDB tem no máximo 16 MiB) e o histórico é paginado; embutir na conversa seria um erro de modelagem.

Esquema relacional (Fase C-5)	MongoDB (Fase C-6)	Motivo
Associações derivadas /remetente e /conversa	Visão `vw_mensagem_historico` ($lookup em `usuarios`)	Não se grava o nome do remetente na mensagem (evita duplicar dado); ele é obtido na leitura.
PK `UUID`, FK, UNIQUE	`_id` (ObjectId), campos `...Id` (referência) e índices únicos	O MongoDB não tem chave estrangeira: a referência é só um valor; sua validade é conferida pela aplicação (Seção 17.3).
Colunas `*_cif` (BYTEA)	Campos `*Cif` do tipo `binData`	Dados sensíveis cifrados antes de gravar (RN-02, RNF-03).

17. Esquema Físico do Banco de Dados Não Relacional (definição de dados)

17.1. SGBD, versão e ambiente

Item	Definição
SGBD e tipo	MongoDB — banco de dados orientado a documentos (BSON).
Versão testada	MongoDB 3.6.8 (servidor real, pacote `mongodb-server-core` do Ubuntu 20.04, executado em Ubuntu 24.04, sem autenticação e sem replica set). Cliente: `mongosh` 1.10.6.
Versão mínima e recomendada	Mínima 3.6 (primeira com `$jsonSchema`, `$expr` em validadores, `$[]` e `arrayFilters`). Recomendada: 7.0 ou serviço gerenciado (MongoDB Atlas). Os scripts só usam recursos existentes desde a 3.6; não foram executados em versões mais novas neste ambiente (nelas, a mensagem de falha de validação traz mais detalhes). Transações multidocumento exigem 4.0+ em replica set (usadas apenas como recomendação, Seção 17.5).
Banco e codificação	Banco `gymgramps`; BSON em UTF-8 (acentuação do português). Datas são BSON `Date` em UTC; a conversão para America/Sao_Paulo é feita na apresentação. Horas do dia são texto `"HH:MM"` (comparável como texto).
Contagem de objetos	5 coleções, 1 visão, 5 validadores, 8 índices secundários (13 índices contando o `_id` de cada coleção) e 1 papel.
Papel	`gymgramps_app` (privilégios mínimos por coleção, Seção 17.3).
Parâmetro de sessão	`GYMGRAMPS_CHAVE` (variável de ambiente, opcional): chave de cifragem; sem ela o script usa uma chave de demonstração.
Ordem de execução	`mongosh URI --file 00_banco.js` → `--file 01_ddl.js` → `--file 02_carga.js` → (opcional) `--file 03_testes.js`, na pasta dos scripts (`lib_gymgramps.js` é carregado por `load()`). `executar_tudo.sh` faz as quatro chamadas.

17.2. Do esquema relacional ao esquema físico do MongoDB

Tema	Esquema físico relacional (Parte V)	Esquema físico MongoDB (Parte VI)	Motivo
Identificador	`id_* uuid` (gen_random_uuid)	`_id` ObjectId (gerado pelo SGBD)	Chave primária substituta, como no relacional.
Nomes	`snake_case`	`camelCase`	Convenção de campos JSON/JavaScript.
Chave estrangeira	`FOREIGN KEY ... ON DELETE CASCADE`	Campo `...Id` (ObjectId) sem restrição no SGBD	O MongoDB não tem integridade referencial; a cascata é feita por `excluirConta`.
Data e hora	`date`, `time`, `timestamptz`	`Date` (00:00 UTC para dias) e texto `"HH:MM"`	Não há tipo apenas-hora; o formato é validado por expressão regular.
Domínios	`CREATE TYPE ... AS ENUM`	`enum` no `$jsonSchema`	Mesmos valores (CLARO/ESCURO, ATIVA/INATIVA, tipos de atividade, status da mensagem).
Dado sensível	`peso_cif bytea` via `fn_cifrar`	`pesoCif` binData: AES-256-GCM (iv, tag, texto cifrado)	Mesma regra: valida-se o domínio antes de cifrar (RNF-05).
Senha	`senha_hash` bcrypt (`crypt`)	`senhaHash` (scrypt, na carga de demonstração)	Em produção a senha fica no serviço de autenticação (RNF-03), não na coleção.
Regras complexas	EXCLUDE, gatilhos, RLS, funções PL/pgSQL	$jsonSchema, $expr, índices únicos/parciais e rotinas da aplicação	Seção 17.3 mostra, regra a regra, onde cada uma é garantida.

17.3. Regras de integridade e onde são garantidas

No PostgreSQL várias regras ficavam em gatilhos, na restrição EXCLUDE e na segurança por linha. O MongoDB não tem esses recursos, então cada regra foi levada ao mecanismo mais forte disponível: validador do esquema ou índice (garantido pelo SGBD, inclusive contra gravações feitas fora da aplicação) ou rotina da aplicação (`lib_gymgramps.js`, garantida só para quem passa por ela). A coluna “Onde” deixa essa diferença explícita. Todas as regras foram exercitadas por um teste (Seção 20).

Regra	Relacional (C-5)	MongoDB (C-6)	Onde	Teste
E-mail único (RF-01)	UNIQUE	índice único `uq_usuarios_email`	SGBD	T01
E-mail válido e em minúsculas (RNF-05)	CHECK	`pattern` do $jsonSchema	SGBD	T02, T03

Regra	Relacional (C-5)	MongoDB (C-6)	Onde	Teste
Domínios (tema, status, tipo)	ENUM	`enum` do $jsonSchema	SGBD	T04
Documento só com campos previstos	—	`additionalProperties: false`	SGBD	T05
hora_fim > hora_inicio e formato HH:MM	CHECK	`$expr` (`$allElementsTrue`) e `pattern`	SGBD	T07, T08
Um documento de agenda por usuário e dia	UNIQUE (usuário, data, hora_inicio)	índice único `uq_agenda_usuario_dia`	SGBD	T09
Atividades do dia sem sobreposição (encostadas podem)	EXCLUDE (GiST)	`$push` condicional atômico (`$not`/`$elemMatch`); em conflito o upsert colide com o índice único	Aplicação + índice	T06, T10
No máximo 1 plano por usuário (0..1)	UNIQUE	índice único `uq_plano_usuario`	SGBD	T11
Plano com ao menos 1 refeição (1..*)	gatilhos adiados	`minItems: 1`; remoção só se restar outra refeição (`refeicoes.1` existe)	SGBD + aplicação	T12, T14
Nome de refeição único no plano	UNIQUE (plano, nome)	`$push` condicional (`refeicoes.nome: {$ne}`)	Aplicação	T13
Conversa com exatamente 2 participantes	gatilhos	`minItems: 2` e `maxItems: 2`	SGBD	T15, T16
Participantes distintos	PK composta	`$expr` compara os dois `usuarioId`	SGBD	T17
Somente contas ATIVAS conversam (RN-01)	gatilhos	`exigirAtiva` em `criarConversa` e `enviarMensagem`	Aplicação	T18, T21
Remetente participa da conversa	FK composta	consulta prévia em `enviarMensagem`	Aplicação	T19
Mensagem não vazia	CHECK	`pattern: \S`	SGBD	T20
Exclusão definitiva em cascata (RN-03)	ON DELETE CASCADE + função	`excluirConta`: `deleteMany` dos dependentes e depois do usuário	Aplicação	T27
Dados sensíveis cifrados (RN-02, RNF-03)	BYTEA + fn_cifrar	binData + `cifrar`/`decifrar` (AES-256-GCM)	Aplicação	T22, T23
Acesso restrito ao titular (RN-02)	RLS + papel	papel `gymgramps_app` (por coleção) + filtro `usuarioId` = usuário da sessão	SGBD (parcial) + aplicação	não testado*

* O MongoDB controla acesso por coleção e por operação, não por documento: o isolamento “cada usuário só vê os próprios dados” fica a cargo da aplicação (filtros pelo usuário autenticado). O papel foi criado e é aceito pelo SGBD, mas os testes rodaram em um servidor sem autenticação, portanto a restrição por papel não foi exercitada.

17.4. Índices

Os cinco índices `_id` (um por coleção) são criados pelo SGBD. Os oito índices secundários foram escolhidos a partir das consultas dos casos de uso; todos são B-tree. Dois são parciais (indexam só as linhas que a consulta usa) e um é multichave (indexa cada elemento de um array). Não há índices em campos de baixa seletividade (`tema`, `tipo`, `concluida`).

Índice	Coleção (campos)	Tipo	Origem e uso
`uq_usuarios_email`	usuarios (email)	único	UC-01 E2 (e-mail já cadastrado); busca no login (UC-02)
`ix_usuarios_ativos_nome`	usuarios (nome), só `statusConta: ATIVA`	parcial	UC-05 A1: busca de destinatário entre contas ativas
`uq_agenda_usuario_dia`	agendas_dia (usuarioId, dia)	único, composto	UC-03: fita do dia em uma leitura; garante 1 documento por usuário e dia (e a checagem de sobreposição)
`uq_plano_usuario`	planos_alimentares (usuarioId)	único	Multiplicidade 0..1; UC-04: plano do usuário
`ix_conversas_participante`	conversas (participantes.usuarioId, ultimaMensagemEm ↓)	multichave, composto	UC-05 passo 2: conversas do usuário, da mais recente para a mais antiga
`ix_mensagens_historico`	mensagens (conversaId, dataHoraEnvio)	composto	UC-05 passo 4: histórico em ordem cronológica
`uq_mensagens_cn`	mensagens (conversaId, remetenteId, dataHoraEnvio)	único, composto	Chave natural da Mensagem (Seção 7.2)
`ix_mensagens_pendentes`	mensagens (conversaId, remetenteId), só `NAO_ENVIADA`	parcial, composto	UC-05 E2: fila de reenvio (poucos documentos)

O teste T26 (Seção 20.3) confirma, pelo plano de execução, que as consultas dos UC-02 a UC-05 usam o índice previsto; diferentemente do relacional, não foi preciso desligar a varredura de coleção para isso.

17.5. Limitações e cuidados

- Sem FK, gatilhos, EXCLUDE e RLS. As regras marcadas “Aplicação” na tabela da Seção 17.3 só valem para quem usa a biblioteca; uma gravação direta no banco pode burlá-las. Por isso o acesso deve passar apenas pelos serviços da aplicação e o papel `gymgramps_app` tem privilégios mínimos.
- Exclusão de conta sem transação. Em servidor isolado, `excluirConta` apaga primeiro os dependentes e por último o usuário; se falhar no meio, basta repetir (a operação é idempotente). Em replica set ou Atlas (MongoDB 4.0+), as chamadas devem ficar dentro de `session.withTransaction()`.
- Resolução de milissegundos. `dataHoraEnvio` tem precisão de ms e faz parte do índice único `uq_mensagens_cn`: duas mensagens do mesmo remetente na mesma conversa e no mesmo milissegundo colidem (E11000) e a aplicação deve reenviar.
- Limite de tamanho. Cada documento tem no máximo 16 MiB; por isso as mensagens ficam fora da conversa e os arrays embutidos têm limite (288 atividades por dia, 12 refeições por plano).
- Cifragem. A chave de demonstração do script existe só para os testes. Em produção a chave vem de um gerenciador de segredos (ou da cifragem de campos no cliente do MongoDB). Campos cifrados não podem ser filtrados no banco, o mesmo limite do `bytea` no relacional.

17.6. Script 00 — criação do banco de dados e do papel da aplicação

Executar com um usuário que possa criar bancos e papéis. Refaz o banco `gymgramps` se ele já existir (usar com cuidado fora de um ambiente de testes).

```
// ============================================================================
// GymGramps - Fase C-6 - Script 00: criação do banco de dados e do papel da aplicação
// SGBD: MongoDB (documentos). Compatível com MongoDB 3.6+ (testado na 3.6.8); recomendado 7.0+.
// Uso:  mongosh "mongodb://localhost:27017" --file 00_banco.js
// Atenção: refaz o banco "gymgramps" se ele já existir (usar só em ambiente de testes).
// ============================================================================
const bd = db.getSiblingDB('gymgramps');
bd.dropDatabase();                       // o MongoDB cria o banco ao criar a primeira coleção
try { bd.dropRole('gymgramps_app'); } catch (e) { /* papel ainda não existia */ }

// Papel da aplicação: privilégios mínimos por coleção (equivalente ao papel gymgramps_app do
// PostgreSQL). Não há RLS no MongoDB: o filtro por titular (usuarioId = id do JWT) é feito
// pela camada de aplicação (Seção 17.3).
const col = (c) => ({ db: 'gymgramps', collection: c });
bd.createRole({
  role: 'gymgramps_app',
  privileges: [
    { resource: col('usuarios'),          actions: ['find', 'update', 'remove'] },
    { resource: col('agendas_dia'),       actions: ['find', 'insert', 'update', 'remove'] },
    { resource: col('planos_alimentares'),actions: ['find', 'insert', 'update', 'remove'] },
    { resource: col('conversas'),         actions: ['find', 'insert', 'update', 'remove'] },
    { resource: col('mensagens'),         actions: ['find', 'insert', 'update', 'remove'] },
    { resource: col('vw_mensagem_historico'), actions: ['find'] }
  ],
  roles: []
});
print('Banco gymgramps pronto; papel gymgramps_app criado.');
```

17.7. Script 01 — esquema físico (coleções, validação, índices e visão)

O script cria as cinco coleções com validador estrito (`validationLevel: strict`, `validationAction: error`), depois os índices e a visão.

```
// ============================================================================
// GymGramps - Fase C-6 - Script 01: esquema físico NoSQL (definição de dados)
// SGBD: MongoDB (orientado a documentos). Compatível com 3.6+ (testado na 3.6.8).
// Pré-requisito: 00_banco.js.   Uso: mongosh "mongodb://localhost:27017/gymgramps" --file 01_ddl.js
// 5 coleções (+1 visão). Validação declarativa com $jsonSchema + $expr (validationAction: error).
// Campos terminados em "Cif" guardam dado sensível CIFRADO (binData; RN-02 / RNF-03).
// ============================================================================
const bd = db.getSiblingDB('gymgramps');

const HORA = '^([01][0-9]|2[0-3]):[0-5][0-9]$';   // "HH:MM" (comparável como texto)
const NAO_VAZIO = '\\S';                           // ao menos 1 caractere que não é espaço

// ----------------------------------------------------------------------------
// 1. usuarios  (classe Usuario)
// ----------------------------------------------------------------------------
bd.createCollection('usuarios', {
  validationLevel: 'strict', validationAction: 'error',
  validator: { $and: [
    { $jsonSchema: {
        bsonType: 'object',
        required: ['email', 'nome', 'dataNascimento', 'pesoCif', 'senhaHash', 'tema', 'statusConta', 'dataCadastro'],
        additionalProperties: false,
        properties: {
          _id:            { bsonType: 'objectId' },
          email:          { bsonType: 'string', maxLength: 255, pattern: '^[^@\\sA-Z]+@[^@\\sA-Z]+\\.[^@\\sA-Z]+$',
                            description: 'e-mail em minúsculas e com formato válido (RNF-05)' },
          nome:           { bsonType: 'string', maxLength: 120, pattern: NAO_VAZIO },
          dataNascimento: { bsonType: 'date' },
          pesoCif:        { bsonType: 'binData', description: '[S] peso em kg, cifrado (RN-02)' },
          senhaHash:      { bsonType: 'string', maxLength: 255 },
          tema:           { enum: ['CLARO', 'ESCURO'] },
          statusConta:    { enum: ['ATIVA', 'INATIVA'] },
          dataCadastro:   { bsonType: 'date' }
        } } },
    { $expr: { $gte: ['$dataNascimento', new Date('1900-01-01T00:00:00Z')] } }
  ] }
```

```
});

// ---------------------------------------------------------------------------
// 2. agendas_dia  (classe Atividade) - padrão "balde por dia": UM documento por usuário e por dia,
//    com as atividades embutidas em array (a fita de 24h do UC-03 é lida em uma única leitura).
// ---------------------------------------------------------------------------
bd.createCollection('agendas_dia', {
  validationLevel: 'strict', validationAction: 'error',
  validator: { $and: [
    { $jsonSchema: {
        bsonType: 'object',
        required: ['usuarioId', 'dia', 'atividades'],
        additionalProperties: false,
        properties: {
          _id:       { bsonType: 'objectId' },
          usuarioId: { bsonType: 'objectId', description: 'referência a usuarios._id' },
          dia:       { bsonType: 'date', description: 'dia da agenda (00:00 UTC)' },
          atividades: {
            bsonType: 'array', maxItems: 288,
            items: {
              bsonType: 'object',
              required: ['atividadeId', 'horaInicio', 'horaFim', 'titulo', 'tipo', 'concluida'],
              additionalProperties: false,
              properties: {
                atividadeId: { bsonType: 'objectId' },
                horaInicio:  { bsonType: 'string', pattern: HORA },
                horaFim:     { bsonType: 'string', pattern: HORA },
                titulo:      { bsonType: 'string', maxLength: 100, pattern: NAO_VAZIO },
                tipo:        { enum: ['MUSCULACAO', 'CARDIO', 'ALONGAMENTO', 'OUTRO'] },
                concluida:   { bsonType: 'bool' }
              } } } } } },
    // horaFim > horaInicio em todas as atividades do dia
    { $expr: { $allElementsTrue: [ { $map: { input: '$atividades', as: 'a',
                                             in: { $gt: ['$$a.horaFim', '$$a.horaInicio'] } } } ] } }
  ] }
});

// ---------------------------------------------------------------------------
// 3. planos_alimentares  (classes PlanoAlimentar + Refeicao, composição 1..*) - refeições EMBUTIDAS
// ---------------------------------------------------------------------------
bd.createCollection('planos_alimentares', {
  validationLevel: 'strict', validationAction: 'error',
  validator: { $jsonSchema: {
    bsonType: 'object',
    required: ['usuarioId', 'dataInicio', 'caloriasDiariasCif', 'proteinasGCif', 'carboidratosGCif', 'gordurasGCif', 'refeicoes'],
    additionalProperties: false,
    properties: {
      _id:               { bsonType: 'objectId' },
      usuarioId:         { bsonType: 'objectId' },
      dataInicio:        { bsonType: 'date' },
      objetivoCif:       { bsonType: 'binData' },
      caloriasDiariasCif: { bsonType: 'binData' },
      proteinasGCif:     { bsonType: 'binData' },
      carboidratosGCif:  { bsonType: 'binData' },
      gordurasGCif:      { bsonType: 'binData' },
      refeicoes: {
        bsonType: 'array', minItems: 1, maxItems: 12,      // multiplicidade 1..* da composição
        items: {
          bsonType: 'object',
          required: ['refeicaoId', 'nome', 'horario', 'caloriasCif'],
          additionalProperties: false,
          properties: {
            refeicaoId:  { bsonType: 'objectId' },
            nome:        { bsonType: 'string', maxLength: 60, pattern: NAO_VAZIO },
            horario:     { bsonType: 'string', pattern: HORA },
            descricaoCif: { bsonType: 'binData' },
            caloriasCif:  { bsonType: 'binData' }
          } } } } } }
});

// ---------------------------------------------------------------------------
// 4. conversas  (classes Conversa + ParticipanteConversa) - participantes EMBUTIDOS (exatamente 2)
// ---------------------------------------------------------------------------
bd.createCollection('conversas', {
  validationLevel: 'strict', validationAction: 'error',
  validator: { $and: [
    { $jsonSchema: {
        bsonType: 'object',
        required: ['dataCriacao', 'participantes'],
        additionalProperties: false,
        properties: {
          _id:              { bsonType: 'objectId' },
          dataCriacao:      { bsonType: 'date' },
          ultimaMensagemEm:  { bsonType: 'date', description: 'cópia derivada, para ordenar a lista de conversas' },
          participantes: {
            bsonType: 'array', minItems: 2, maxItems: 2,    // exatamente 2 participantes
            items: {
              bsonType: 'object',
              required: ['usuarioId', 'dataEntrada'],
              additionalProperties: false,
              properties: { usuarioId: { bsonType: 'objectId' }, dataEntrada: { bsonType: 'date' } } } } } } },
    // os dois participantes são usuários distintos
    { $expr: { $ne: [ { $arrayElemAt: ['$participantes.usuarioId', 0] },
                      { $arrayElemAt: ['$participantes.usuarioId', 1] } ] } }
  ] }
});

// ---------------------------------------------------------------------------
// 5. mensagens  (classe Mensagem) - coleção própria (cresce sem limite; não se embute na conversa)
// ---------------------------------------------------------------------------
bd.createCollection('mensagens', {
  validationLevel: 'strict', validationAction: 'error',
  validator: { $jsonSchema: {
```

```
    bsonType: 'object',
    required: ['conversaId', 'remetenteId', 'dataHoraEnvio', 'conteudo', 'status'],
    additionalProperties: false,
    properties: {
      _id:          { bsonType: 'objectId' },
      conversaId:   { bsonType: 'objectId', description: 'referência a conversas._id' },
      remetenteId:  { bsonType: 'objectId', description: 'referência a usuarios._id (deve ser participante da conversa)' },
      dataHoraEnvio: { bsonType: 'date' },
      conteudo:     { bsonType: 'string', pattern: NAO_VAZIO },
      status:       { enum: ['ENVIADA', 'NAO_ENVIADA'] }
    } } }
});

// ------------------------------------------------------------------------
// 6. Índices (além do índice automático de _id)
// ------------------------------------------------------------------------
bd.usuarios.createIndex({ email: 1 }, { name: 'uq_usuarios_email', unique: true });                      // UC-01 E2, UC-02
bd.usuarios.createIndex({ nome: 1 }, { name: 'ix_usuarios_ativos_nome',                                  // UC-05 A1 (parcial)
                                  partialFilterExpression: { statusConta: 'ATIVA' } });
bd.agendas_dia.createIndex({ usuarioId: 1, dia: 1 }, { name: 'uq_agenda_usuario_dia', unique: true });   // UC-03
bd.planos_alimentares.createIndex({ usuarioId: 1 }, { name: 'uq_plano_usuario', unique: true });         // 0..1 plano; UC-04
bd.conversas.createIndex({ 'participantes.usuarioId': 1, ultimaMensagemEm: -1 },                         // UC-05 passo 2 (multichave)
                         { name: 'ix_conversas_participante' });
bd.mensagens.createIndex({ conversaId: 1, dataHoraEnvio: 1 }, { name: 'ix_mensagens_historico' });       // UC-05 passo 4
bd.mensagens.createIndex({ conversaId: 1, remetenteId: 1, dataHoraEnvio: 1 }, { name: 'uq_mensagens_cn', unique: true }); // CN
bd.mensagens.createIndex({ conversaId: 1, remetenteId: 1 }, { name: 'ix_mensagens_pendentes',            // UC-05 E2 (parcial)
                                                    partialFilterExpression: { status: 'NAO_ENVIADA' } });

// ------------------------------------------------------------------------
// 7. Visão: histórico do chat com o nome do remetente (associação derivada /remetente)
// ------------------------------------------------------------------------
bd.createView('vw_mensagem_historico', 'mensagens', [
  { $lookup: { from: 'usuarios', localField: 'remetenteId', foreignField: '_id', as: 'rem' } },
  { $project: { _id: 0, conversaId: 1, dataHoraEnvio: 1, remetente: { $arrayElemAt: ['$rem.nome', 0] },
                conteudo: 1, status: 1 } }
]);

print('Esquema físico criado: ' + bd.getCollectionNames().filter(n => n !== 'system.views').sort().join(', '));
```

17.8. Biblioteca da camada de aplicação — lib_gymgramps.js

Reúne o que, no PostgreSQL, eram gatilhos e funções: cifragem (`cifrar`/`decifrar`), a checagem de sobreposição de atividades, o nome único de refeição, o mínimo de uma refeição, as contas ativas no chat, o remetente que participa da conversa e a exclusão em cascata. É carregada pelos scripts 02 e 03 com `load()`. No sistema real este código fica nos serviços da aplicação (camada de Infraestrutura da Seção 11).

```
// ============================================================================
// GymGramps - Fase C-6 - Biblioteca da camada de aplicação (carregada por 02 e 03 com load())
// Faz no código o que, no PostgreSQL, eram gatilhos e funções: cifragem (RN-02), regras que o
// $jsonSchema não expressa (sobreposição, nomes únicos, contas ATIVAS, remetente participante)
// e a exclusão em cascata (RN-03). No GymGramps real este código fica nos serviços da SPA/API.
// ============================================================================
const crypto = require('crypto');
const bd = db.getSiblingDB('gymgramps');

// ---- Cifragem AES-256-GCM (equivalente a fn_cifrar/fn_decifrar). Chave de DEMONSTRAÇÃO:
//      em produção vem de um gerenciador de segredos (ou de Client-Side Field Level Encryption).
const CHAVE = crypto.scryptSync(process.env.GYMGRAMPS_CHAVE || 'chave-demo-gymgramps-nao-usar-em-producao',
                                'gymgramps-sal', 32);
function cifrar(valor) {                          // texto -> BinData (iv 12 B | tag 16 B | cifrado)
  const iv = crypto.randomBytes(12);
  const c = crypto.createCipheriv('aes-256-gcm', CHAVE, iv);
  const ct = Buffer.concat([c.update(String(valor), 'utf8'), c.final()]);
  return BinData(0, Buffer.concat([iv, c.getAuthTag(), ct]).toString('base64'));
}
function decifrar(bin) {                          // BinData -> texto
  const b = Buffer.from(bin.read(0, bin.length()));
  const d = crypto.createDecipheriv('aes-256-gcm', CHAVE, b.subarray(0, 12));
  d.setAuthTag(b.subarray(12, 28));
  return Buffer.concat([d.update(b.subarray(28)), d.final()]).toString('utf8');
}
function hashSenha(senha) {                       // scrypt (no Supabase, o Auth guarda o bcrypt)
  const sal = crypto.randomBytes(16).toString('hex');
  return 'scrypt$' + sal + '$' + crypto.scryptSync(senha, sal, 32).toString('hex');
}
const dia = (s) => new Date(s + 'T00:00:00Z');    // "2026-10-12" -> Date (00:00 UTC)

// ---- Atividades: sem sobreposição no mesmo dia (a EXCLUDE do PostgreSQL) --------------------
// Atômico em um único documento: o $push só ocorre se NENHUMA atividade do dia se intercepta com
// [ini, fim). Se houver sobreposição o filtro não casa, o upsert tenta criar o mesmo (usuarioId, dia)
// e o índice único uq_agenda_usuario_dia recusa (E11000). Atividades "encostadas" são permitidas.
function agendarAtividade(usuarioId, diaData, ini, fim, titulo, tipo) {
  if (!(fim > ini)) throw new Error('hora_fim deve ser maior que hora_inicio');
  const atv = { atividadeId: new ObjectId(), horaInicio: ini, horaFim: fim, titulo, tipo, concluida: false };
  try {
    bd.agendas_dia.updateOne(
      { usuarioId, dia: diaData, atividades: { $not: { $elemMatch: { horaInicio: { $lt: fim }, horaFim: { $gt: ini } } } } },
      { $push: { atividades: atv } }, { upsert: true });
  } catch (e) {
    if (e.code === 11000) throw new Error('Atividade sobrepõe outra do mesmo dia (' + ini + '-' + fim + ')');
    throw e;
  }
  return atv.atividadeId;
}

// ---- Plano alimentar: refeições com nome único e ao menos 1 refeição -----------------------
function adicionarRefeicao(planoId, nome, horario, descricao, kcal) {
  const r = bd.planos_alimentares.updateOne(
    { _id: planoId, 'refeicoes.nome': { $ne: nome } },                    // nome ainda não usado no plano
    { $push: { refeicoes: { refeicaoId: new ObjectId(), nome, horario,
```

```
                          descricaoCif: cifrar(descricao), caloriasCif: cifrar(kcal) } } });
  if (r.matchedCount === 0) throw new Error('Plano inexistente ou já possui refeição "' + nome + '"');
}
function removerRefeicao(planoId, nome) {
  const r = bd.planos_alimentares.updateOne(
    { _id: planoId, 'refeicoes.1': { $exists: true } },                  // só remove se restar ao menos 1
    { $pull: { refeicoes: { nome } } });
  if (r.modifiedCount === 0) throw new Error('Um plano alimentar deve ter ao menos 1 refeição');
}

// ---- Chat ------------------------------------------------------------------------
function exigirAtiva(usuarioId) {
  const u = bd.usuarios.findOne({ _id: usuarioId }, { statusConta: 1 });
  if (!u || u.statusConta !== 'ATIVA') throw new Error('Somente contas ATIVAS usam o chat (RN-01)');
}
function criarConversa(usuarioA, usuarioB, quando) {                     // UC-05, fluxo A1
  if (usuarioA.equals(usuarioB)) throw new Error('Os participantes da conversa devem ser distintos');
  exigirAtiva(usuarioA); exigirAtiva(usuarioB);
  const t = quando || new Date();
  return bd.conversas.insertOne({ dataCriacao: t, participantes: [
    { usuarioId: usuarioA, dataEntrada: t }, { usuarioId: usuarioB, dataEntrada: t } ] }).insertedId;
}
function enviarMensagem(conversaId, remetenteId, conteudo, opc) {         // UC-05, passo 7
  opc = opc || {};
  if (bd.conversas.countDocuments({ _id: conversaId, 'participantes.usuarioId': remetenteId }) === 0)
    throw new Error('O remetente não participa da conversa');             // = FK composta do modelo relacional
  exigirAtiva(remetenteId);
  const quando = opc.quando || new Date();
  const id = bd.mensagens.insertOne({ conversaId, remetenteId, dataHoraEnvio: quando, conteudo,
                                      status: opc.status || 'ENVIADA' }).insertedId;
  bd.conversas.updateOne({ _id: conversaId, $or: [{ ultimaMensagemEm: { $lt: quando } }, { ultimaMensagemEm: { $exists: false } }] },
                         { $set: { ultimaMensagemEm: quando } });
  return id;
}

// ---- Exclusão definitiva de conta (RN-03 / UC-06) --------------------------------------------
// Dependentes primeiro e o usuário por último: se falhar no meio, basta repetir (idempotente).
// Em um replica set / Atlas (MongoDB 4.0+), envolver as chamadas em session.withTransaction().
function excluirConta(usuarioId) {
  const conversas = bd.conversas.find({ 'participantes.usuarioId': usuarioId }, { _id: 1 }).toArray().map(c => c._id);
  bd.mensagens.deleteMany({ conversaId: { $in: conversas } });
  bd.conversas.deleteMany({ _id: { $in: conversas } });
  bd.agendas_dia.deleteMany({ usuarioId });
  bd.planos_alimentares.deleteMany({ usuarioId });
  return bd.usuarios.deleteOne({ _id: usuarioId }).deletedCount;
}
```

18. Diagramas da organização dos dados

18.1. Organização física das coleções

A Figura 9 mostra como os dados ficam organizados no MongoDB: as cinco coleções e a visão, os campos de cada documento com seus tipos BSON, os arrays de subdocumentos embutidos (caixas tracejadas), as referências por ObjectId entre coleções (setas), as cardinalidades herdadas do modelo conceitual e os índices de cada coleção. As coleções da linha de cima (`agendas_dia`, `usuarios`, `planos_alimentares`) guardam o perfil, a rotina e a alimentação; as da linha de baixo (`mensagens`, `conversas` e a visão) formam o chat.

18.2. Exemplo com documentos reais da carga

A Figura 10 mostra documentos reais gerados pelo script 02 (usuária Ana e sua agenda de 12/10/2026, seu plano alimentar, a conversa com o Bruno e a primeira mensagem), com as referências indicadas por setas. Ela exemplifica o que a Figura 9 descreve: o campo `usuarioId` da agenda e do plano contém o `_id` da Ana; a conversa guarda os dois participantes embutidos; e a mensagem aponta para a conversa e para o remetente, que é um dos participantes. Como os identificadores são gerados a cada execução, os valores mostrados mudam se a carga for refeita.

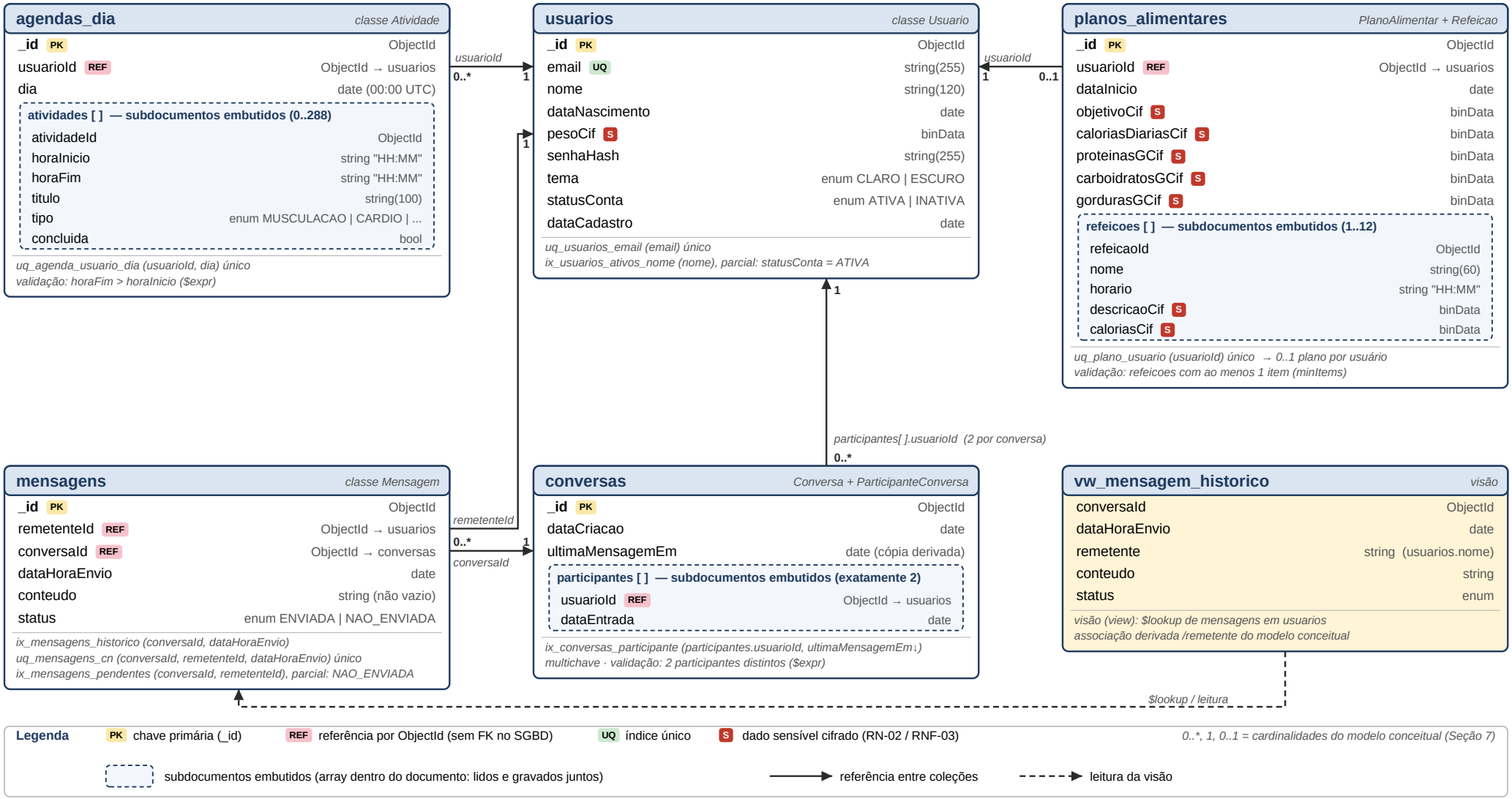


Figura 9 — Organização física dos dados do GymGramps no MongoDB. 5 coleções e 1 visão; subdocumentos embutidos (caixas tracejadas), referências por ObjectId (setas), tipos BSON, dados sensíveis cifrados (S) e índices de cada coleção.

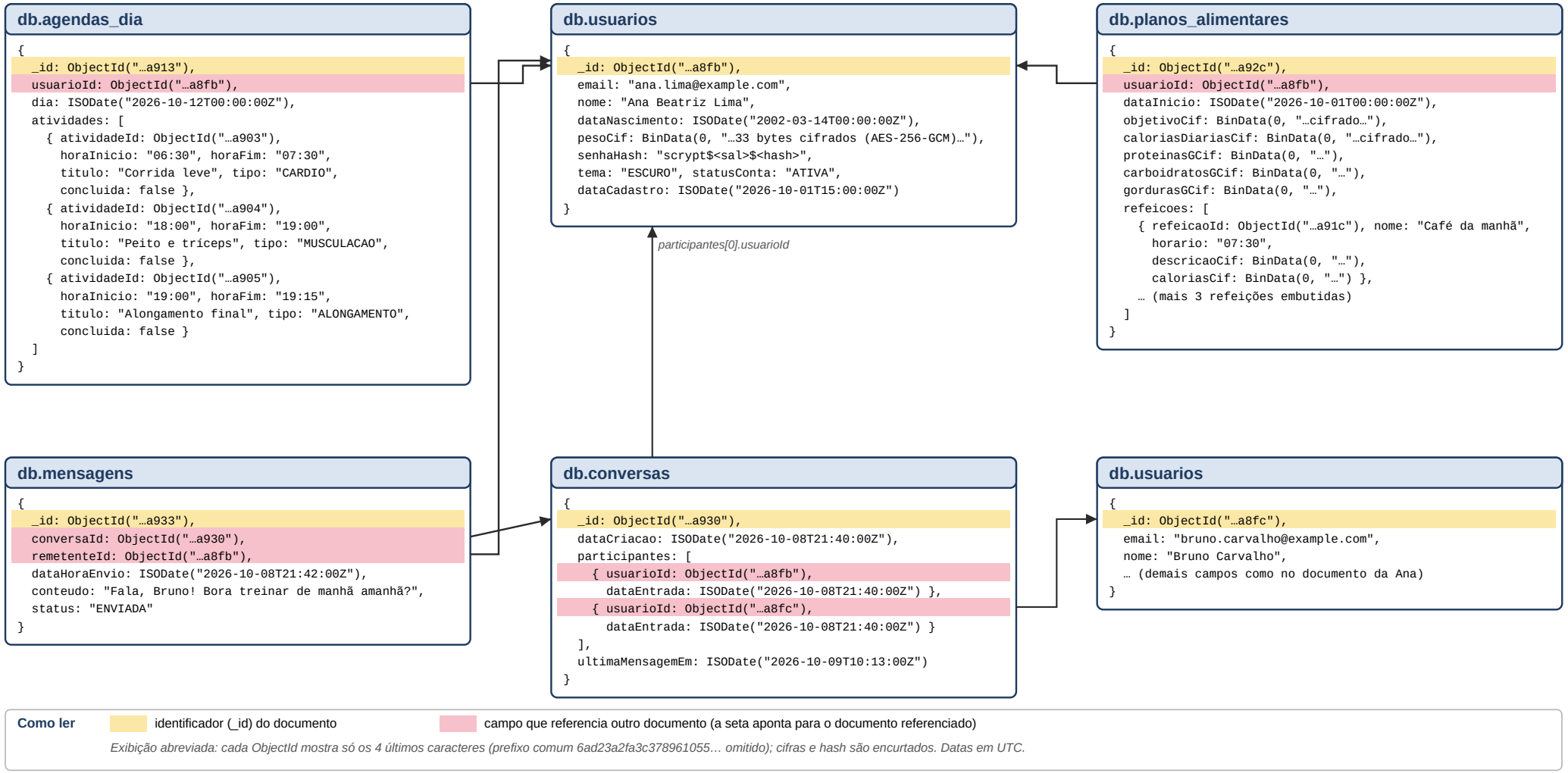


Figura 10 — Exemplo de documentos reais da carga inicial e de suas referências. A agenda do dia e o plano alimentar apontam para a usuária; a mensagem aponta para a conversa e para o remetente; os dois participantes da conversa apontam para os usuários.

19. Carga Inicial do Banco de Dados (DML)

19.1. Estratégia e conteúdo da carga

A carga repete, com os mesmos dados fictícios, a carga relacional da Seção 14, de modo que os dois bancos possam ser comparados. Os e-mails usam o domínio `example.com`, reservado para exemplos. Como cada coleção é uma unidade de escrita, a ordem é: usuários, agendas, planos, conversas, mensagens (estas pela rotina `enviarMensagem`, que confere remetente e conta ativa) e, por fim, as atualizações.

Coleção	Documentos	Conteúdo e finalidade
`usuarios`	6	5 contas ATIVAS e 1 INATIVA (Felipe, inativada por atualização); peso cifrado; senha de demonstração com hash; temas CLARO e ESCURO.
`agendas_dia`	11 (16 atividades embutidas)	Treinos de 05/10 a 13/10/2026 dos 6 usuários. Carla tem duas atividades “encostadas” (08:00–09:00 e 09:00–10:00), permitidas.
`planos_alimentares`	4 (16 refeições embutidas)	Planos de Ana, Bruno, Carla e Diego, com 4 refeições cada. Elisa e Felipe ficam sem plano (UC-04, fluxo E1). Todos os campos sensíveis cifrados.
`conversas`	3 (6 participantes embutidos)	Ana–Bruno, Ana–Carla e Bruno–Diego; a lista é ordenada por `ultimaMensagemEm`.
`mensagens`	11	Histórico das três conversas; uma mensagem começa como NAO_ENVIADA (UC-05, E2) e é reenviada por atualização.

Atualizações (equivalentes aos UPDATE 5.1 a 5.6 da carga relacional):

- conclusão das atividades de dias já passados, com o operador posicional `$[]` (UC-03);
- reagendamento de uma atividade (HIIT do Bruno, 12:00–12:40 para 12:15–12:55), validando que não colide com outra do dia, com `arrayFilters`;
- troca de tema no perfil (RF-06);
- ajuste da meta de calorias do plano do Bruno, com os dados sensíveis recifrados (RN-02);
- reenvio da mensagem que havia falhado (UC-05, E2);
- inativação da conta do Felipe, que passa a ficar fora de conversas (RN-01).

Atenção. A chave de cifragem (`chave-demo-gymgramps-nao-usar-em-producao`) e a senha de demonstração existem só para esta carga de testes. Em produção a chave vem de um gerenciador de segredos e as senhas são cadastradas pelo próprio usuário (UC-01).

19.2. Script 02 — DML da carga inicial

```
// ============================================================================
// GymGramps - Fase C-6 - Script 02: carga inicial (manipulação de dados)
// Pré-requisito: 01_ddl.js e lib_gymgramps.js (na mesma pasta).
// Uso (na pasta dos scripts): mongosh "mongodb://localhost:27017/gymgramps" --file 02_carga.js
// Mesmos dados fictícios da carga relacional (Seção 14): 6 usuários, 16 atividades, 4 planos com
// 16 refeições, 3 conversas e 11 mensagens, além de 6 atualizações. Chave e senha são de DEMONSTRAÇÃO.
// ============================================================================
load('lib_gymgramps.js');

// ----------------------------------------------------------------------------
// 1. usuarios (peso cifrado; senha de demonstração com hash)
// ----------------------------------------------------------------------------
const U = {};                                          // apelido -> _id
[ ['ana',    'ana.lima@example.com',        'Ana Beatriz Lima',  '2002-03-14', '62.50', 'ESCURO'],
  ['bruno',  'bruno.carvalho@example.com',  'Bruno Carvalho',    '1999-07-21', '82.30', 'CLARO'],
  ['carla',  'carla.menezes@example.com',   'Carla Menezes',     '2001-11-05', '58.00', 'CLARO'],
  ['diego',  'diego.ramos@example.com',     'Diego Ramos',       '1997-01-30', '91.70', 'ESCURO'],
  ['elisa',  'elisa.teixeira@example.com',  'Elisa Teixeira',    '2004-09-18', '54.20', 'CLARO'],
  ['felipe', 'felipe.andrade@example.com',  'Felipe Andrade',    '2000-05-09', '76.00', 'CLARO']
].forEach(([ap, email, nome, nasc, peso, tema]) => {
  U[ap] = bd.usuarios.insertOne({ email, nome, dataNascimento: dia(nasc), pesoCif: cifrar(peso),
    senhaHash: hashSenha('Demo@2026'), tema, statusConta: 'ATIVA',
    dataCadastro: new Date('2026-10-01T12:00:00-03:00') }).insertedId;
});

// ----------------------------------------------------------------------------
// 2. agendas_dia: um documento por usuário e dia, com as atividades embutidas.
//    Carla tem duas atividades "encostadas" (08:00-09:00 e 09:00-10:00), permitidas.
// ----------------------------------------------------------------------------
const atv = (ini, fim, titulo, tipo) =>
  ({ atividadeId: new ObjectId(), horaInicio: ini, horaFim: fim, titulo, tipo, concluida: false });
bd.agendas_dia.insertMany([
  { usuarioId: U.ana,    dia: dia('2026-10-08'), atividades: [ atv('18:00','19:00','Costas e bíceps','MUSCULACAO') ] },
  { usuarioId: U.ana,    dia: dia('2026-10-09'), atividades: [ atv('07:00','08:00','Treino de pernas','MUSCULACAO') ] },
  { usuarioId: U.ana,    dia: dia('2026-10-12'), atividades: [ atv('06:30','07:30','Corrida leve','CARDIO'),
                                                                atv('18:00','19:00','Peito e tríceps','MUSCULACAO'),
                                                                atv('19:00','19:15','Alongamento final','ALONGAMENTO') ] },
  { usuarioId: U.bruno,  dia: dia('2026-10-08'), atividades: [ atv('19:00','20:00','Corrida no parque','CARDIO') ] },
  { usuarioId: U.bruno,  dia: dia('2026-10-12'), atividades: [ atv('07:00','08:00','Superiores (push)','MUSCULACAO'),
                                                                atv('12:00','12:40','HIIT','CARDIO') ] },
  { usuarioId: U.bruno,  dia: dia('2026-10-13'), atividades: [ atv('07:00','08:00','Pernas','MUSCULACAO') ] },
  { usuarioId: U.carla,  dia: dia('2026-10-12'), atividades: [ atv('08:00','09:00','Alongamento e mobilidade','ALONGAMENTO'),
                                                                atv('09:00','10:00','Bicicleta ergométrica','CARDIO') ] },
  { usuarioId: U.carla,  dia: dia('2026-10-13'), atividades: [ atv('17:30','18:30','Treino full body','MUSCULACAO') ] },
  { usuarioId: U.diego,  dia: dia('2026-10-12'), atividades: [ atv('05:45','06:45','Peito e ombros','MUSCULACAO'),
                                                                atv('21:00','21:30','Caminhada','OUTRO') ] },
  { usuarioId: U.elisa,  dia: dia('2026-10-12'), atividades: [ atv('16:00','17:00','Treino full body','MUSCULACAO') ] },
  { usuarioId: U.felipe, dia: dia('2026-10-05'), atividades: [ atv('18:00','19:00','Treino de pernas','MUSCULACAO') ] }
]);

// ----------------------------------------------------------------------------
// 3. planos_alimentares: plano + 4 refeições embutidas (Elisa e Felipe ficam sem plano: UC-04, E1)
// ----------------------------------------------------------------------------
const ref = (nome, horario, descricao, kcal) => ({ refeicaoId: new ObjectId(), nome, horario,
                                                   descricaoCif: cifrar(descricao), caloriasCif: cifrar(kcal) });
const plano = (u, inicio, objetivo, kcal, prot, carb, gord, refeicoes) => ({ usuarioId: U[u], dataInicio: dia(inicio),
  objetivoCif: cifrar(objetivo), caloriasDiariasCif: cifrar(kcal), proteinasGCif: cifrar(prot),
  carboidratosGCif: cifrar(carb), gordurasGCif: cifrar(gord), refeicoes });
bd.planos_alimentares.insertMany([
  plano('ana', '2026-10-01', 'Hipertrofia com leve superávit', '2400', '150.0', '300.0', '70.0', [
    ref('Café da manhã', '07:30', 'Ovos mexidos, pão integral e fruta', '550'),
    ref('Almoço', '12:30', 'Arroz, feijão, frango grelhado e salada', '750'),
    ref('Lanche da tarde', '16:00', 'Iogurte com aveia e banana', '350'),
    ref('Jantar', '20:00', 'Omelete de legumes e batata-doce', '600') ]),
  plano('bruno', '2026-10-01', 'Emagrecimento com ganho de massa', '2000', '160.0', '180.0', '60.0', [
    ref('Café da manhã', '07:00', 'Tapioca com queijo e café sem açúcar', '400'),
    ref('Almoço', '12:30', 'Frango com batata-doce e brócolis', '650'),
    ref('Lanche da tarde', '16:30', 'Whey protein com fruta', '250'),
    ref('Jantar', '20:00', 'Peixe assado com legumes', '500') ]),
  plano('carla', '2026-10-01', 'Manutenção de peso', '1900', '100.0', '240.0', '55.0', [
    ref('Café da manhã', '08:00', 'Pão integral com ovo e mamão', '420'),
    ref('Almoço', '12:00', 'Macarrão integral com carne moída', '650'),
    ref('Lanche da tarde', '16:00', 'Castanhas e fruta', '250'),
    ref('Jantar', '19:30', 'Sopa de legumes com frango', '400') ]),
  plano('diego', '2026-10-01', 'Hipertrofia', '3000', '190.0', '380.0', '85.0', [
    ref('Café da manhã', '06:30', 'Panqueca de aveia, ovos e banana', '700'),
    ref('Almoço', '12:30', 'Arroz, feijão, carne e legumes', '950'),
    ref('Lanche da tarde', '16:00', 'Sanduíche de frango e suco natural', '600'),
    ref('Jantar', '20:30', 'Massa com atum e salada', '750') ])
]);

// ----------------------------------------------------------------------------
// 4. conversas (com os 2 participantes embutidos) e mensagens
// ----------------------------------------------------------------------------
const t = (s) => new Date(s);
const C = {
  anaBruno:   bd.conversas.insertOne({ dataCriacao: t('2026-10-08T18:40:00-03:00'), participantes: [
                { usuarioId: U.ana,   dataEntrada: t('2026-10-08T18:40:00-03:00') },
                { usuarioId: U.bruno, dataEntrada: t('2026-10-08T18:40:00-03:00') } ] }).insertedId,
  anaCarla:   bd.conversas.insertOne({ dataCriacao: t('2026-10-07T20:00:00-03:00'), participantes: [
                { usuarioId: U.ana,   dataEntrada: t('2026-10-07T20:00:00-03:00') },
                { usuarioId: U.carla, dataEntrada: t('2026-10-07T20:00:00-03:00') } ] }).insertedId,
  brunoDiego: bd.conversas.insertOne({ dataCriacao: t('2026-10-06T21:25:00-03:00'), participantes: [
                { usuarioId: U.bruno, dataEntrada: t('2026-10-06T21:25:00-03:00') },
                { usuarioId: U.diego, dataEntrada: t('2026-10-06T21:25:00-03:00') } ] }).insertedId
};
[ [C.anaBruno, 'ana',   '2026-10-08T18:42:00-03:00', 'Fala, Bruno! Bora treinar de manhã amanhã?', 'ENVIADA'],
  [C.anaBruno, 'bruno', '2026-10-08T18:45:00-03:00', 'Bora! Chego às 7h na academia.', 'ENVIADA'],
  [C.anaBruno, 'ana',   '2026-10-08T18:46:00-03:00', 'Fechado. Levo a planilha de cargas.', 'ENVIADA'],
  [C.anaBruno, 'bruno', '2026-10-09T07:12:00-03:00', 'Cheguei, te espero na entrada.', 'ENVIADA'],
```

```
  [C.anaBruno, 'ana',   '2026-10-09T07:13:00-03:00', 'Já estou subindo!', 'NAO_ENVIADA'],
  [C.anaCarla, 'carla', '2026-10-07T20:05:00-03:00', 'Ana, que horas é o alongamento de segunda?', 'ENVIADA'],
  [C.anaCarla, 'ana',   '2026-10-07T20:09:00-03:00', 'Às 19h, logo depois do treino de peito.', 'ENVIADA'],
  [C.anaCarla, 'carla', '2026-10-07T20:10:00-03:00', 'Perfeito, vou no mesmo horário.', 'ENVIADA'],
  [C.brunoDiego, 'diego', '2026-10-06T21:30:00-03:00', 'Bruno, me passa a receita do frango com batata-doce?', 'ENVIADA'],
  [C.brunoDiego, 'bruno', '2026-10-06T21:34:00-03:00', 'Mando amanhã, já anotei no meu plano.', 'ENVIADA'],
  [C.brunoDiego, 'diego', '2026-10-06T21:35:00-03:00', 'Valeu!', 'ENVIADA']
].forEach(([conv, rem, quando, conteudo, status]) => enviarMensagem(conv, U[rem], conteudo, { quando: t(quando), status }));

// ------------------------------------------------------------------------------
// 5. Atualizações (equivalentes aos UPDATE 5.1 a 5.6 da carga relacional)
// ------------------------------------------------------------------------------
// 5.1 atividades de dias que já passaram foram concluídas (UC-03)
const r51 = bd.agendas_dia.updateMany({ dia: { $lt: dia('2026-10-09') } }, { $set: { 'atividades.$[].concluida': true } });
// 5.2 reagendamento: o HIIT do Bruno passa para 12:15-12:55 (checando que não colide com outra atividade)
const r52 = bd.agendas_dia.updateOne(
  { usuarioId: U.bruno, dia: dia('2026-10-12'),
    atividades: { $not: { $elemMatch: { titulo: { $ne: 'HIIT' }, horaInicio: { $lt: '12:55' }, horaFim: { $gt: '12:15' } } } } },
  { $set: { 'atividades.$[a].horaInicio': '12:15', 'atividades.$[a].horaFim': '12:55' } },
  { arrayFilters: [ { 'a.titulo': 'HIIT' } ] });
// 5.3 perfil: troca de tema (RF-06)
const r53 = bd.usuarios.updateOne({ email: 'bruno.carvalho@example.com' }, { $set: { tema: 'ESCURO' } });
// 5.4 ajuste de meta do plano do Bruno (dados sensíveis recifrados)
const r54 = bd.planos_alimentares.updateOne({ usuarioId: U.bruno },
  { $set: { caloriasDiariasCif: cifrar('2200'), objetivoCif: cifrar('Emagrecimento com ganho de massa (meta ajustada)') } });
// 5.5 reenvio da mensagem que havia falhado (UC-05, E2)
const r55 = bd.mensagens.updateMany({ status: 'NAO_ENVIADA' }, { $set: { status: 'ENVIADA' } });
// 5.6 Felipe pediu a inativação da conta: sai de rotina, alimentação e chat (RN-01)
const r56 = bd.usuarios.updateOne({ email: 'felipe.andrade@example.com' }, { $set: { statusConta: 'INATIVA' } });
print('UPDATE 5.1..5.6 (modificados): ' + [r51, r52, r53, r54, r55, r56].map(r => r.modifiedCount).join(', '));

// ------------------------------------------------------------------------------
// 6. Conferência da carga
// ------------------------------------------------------------------------------
['usuarios', 'agendas_dia', 'planos_alimentares', 'conversas', 'mensagens'].forEach(c =>
  print(c.padEnd(20) + String(bd[c].countDocuments({})).padStart(4) + ' documentos'));
print('atividades (embutidas) ' + bd.agendas_dia.aggregate([{ $unwind: '$atividades' }, { $count: 'n' }]).toArray()[0].n);
print('refeicoes  (embutidas) ' + bd.planos_alimentares.aggregate([{ $unwind: '$refeicoes' }, { $count: 'n' }]).toArray()[0].n);
```

19.3. Resultado da carga

Saída do `mongosh` para o script 02 (todas as instruções aceitas e conferência final). Os números da linha “UPDATE” correspondem, na ordem, aos itens 5.1 a 5.6 do script (3 agendas diárias com atividades concluídas e 1 documento em cada um dos demais).

```
UPDATE 5.1..5.6 (modificados): 3, 1, 1, 1, 1, 1
usuarios              6 documentos
agendas_dia          11 documentos
planos_alimentares    4 documentos
conversas             3 documentos
mensagens            11 documentos
atividades (embutidas) 16
refeicoes  (embutidas) 16
```

20. Testes e Verificação

20.1. Metodologia

Os scripts foram executados em sequência sobre um servidor MongoDB 3.6.8 recém-iniciado (diretório de dados vazio): o banco foi criado, o esquema montado (nenhum erro), a carga aceita sem erros e, por fim, o script 03 submeteu ao banco 27 testes. Os erros do script 03 são esperados: cada um prova que uma regra foi barrada, seja pelo validador (código 121, “Document failed validation”), por um índice único (código 11000, E11000) ou por uma rotina da aplicação. Foram 20 violações barradas, 1 caso que deve ser aceito (T10) e 6 verificações de consultas, índices, cifragem e cascata (T22 a T27). Cada teste que altera dados desfaz a alteração em seguida.

20.2. Testes de integridade e resultado obtido

Nº	O que se testa	Resultado obtido
T01	E-mail já cadastrado	ERRO 11000: E11000 duplicate key error ... index: uq_usuarios_email
T02	E-mail com formato inválido	ERRO 121: Document failed validation
T03	E-mail com letras maiúsculas	ERRO 121: Document failed validation
T04	Tema fora do domínio (AZUL)	ERRO 121: Document failed validation
T05	Campo não previsto no esquema	ERRO 121: Document failed validation (additionalProperties: false)
T06	Atividade 08:30–09:30 sobreposta a 08:00–09:00 (mesmo usuário e dia)	ERRO (aplicação): Atividade sobrepõe outra do mesmo dia (08:30-09:30)
T07	horaFim anterior a horaInicio	ERRO 121: Document failed validation
T08	Horário com formato inválido (25:00)	ERRO 121: Document failed validation
T09	Segundo documento de agenda para o mesmo usuário e dia	ERRO 11000: E11000 duplicate key error ... index: uq_agenda_usuario_dia
T10	Atividade encostada (07:30 logo após 06:30–07:30) deve ser aceita	ACEITO (sem erro); desfeita com $pull
T11	Segundo plano alimentar para o mesmo usuário	ERRO 11000: E11000 duplicate key error ... index: uq_plano_usuario
T12	Plano sem nenhuma refeição	ERRO 121: Document failed validation (minItems: 1)
T13	Refeição com nome repetido no mesmo plano	ERRO (aplicação): Plano inexistente ou já possui refeição "Almoço"
T14	Remover a última refeição de um plano	ERRO (aplicação): Um plano alimentar deve ter ao menos 1 refeição
T15	Conversa com apenas 1 participante	ERRO 121: Document failed validation (minItems: 2)
T16	Conversa com 3 participantes	ERRO 121: Document failed validation (maxItems: 2)
T17	Conversa com o mesmo usuário duas vezes	ERRO 121: Document failed validation ($expr)
T18	Conta INATIVA em uma nova conversa	ERRO (aplicação): Somente contas ATIVAS usam o chat (RN-01)
T19	Mensagem de quem não participa da conversa	ERRO (aplicação): O remetente não participa da conversa
T20	Mensagem só com espaços	ERRO 121: Document failed validation
T21	Remetente com conta INATIVA envia mensagem	ERRO (aplicação): Somente contas ATIVAS usam o chat (RN-01)
T22	Peso armazenado cifrado (binData) e decifrado com a chave	33 bytes cifrados; decifrado: 62.50 kg
T23	UC-04: plano e refeições decifrados	Plano de 2400 kcal e 4 refeições (Seção 20.3)
T24	UC-03: fita do dia 12/10 em ordem de horaInicio	3 atividades ordenadas, lidas de um único documento (Seção 20.3)
T25	UC-05: histórico da conversa com remetente derivado (visão)	5 mensagens em ordem cronológica (Seção 20.3)
T26	Plano de execução das consultas dos UC-02 a UC-05	Cada consulta usa o índice previsto (Seção 20.3)
T27	UC-06: exclusão definitiva de um usuário de teste com agenda, plano, conversa e mensagem	Contagens voltam ao estado anterior à criação (Seção 20.3)

As mensagens “Document failed validation” são as do MongoDB 3.6.8; versões mais novas acrescentam o detalhe da regra violada. O validador aponta qual documento falhou, mas na versão testada não informa qual regra; por isso a tabela indica, entre parênteses, a regra exercitada em cada teste.

20.3. Consultas dos casos de uso e resultados

Saída real do script 03 para os testes T22 a T27. As horas das mensagens (T25) aparecem em UTC (a carga foi feita em UTC−03:00, por isso 18:42 de Brasília aparece como 21:42).

```
== T22 dado sensível armazenado cifrado (binData) e decifrado com a chave
   Ana Beatriz Lima | tipo BSON: binData | bytes cifrados: 33 | peso decifrado: 62.50 kg

== T23 UC-04: plano alimentar e refeições decifrados (Ana)
   objetivo=Hipertrofia com leve superávit | kcal=2400 | P/C/G=150.0/300.0/70.0
   07:30 | Café da manhã    |  550 kcal | Ovos mexidos, pão integral e fruta
   12:30 | Almoço           |  750 kcal | Arroz, feijão, frango grelhado e salada
   16:00 | Lanche da tarde  |  350 kcal | Iogurte com aveia e banana
   20:00 | Jantar           |  600 kcal | Omelete de legumes e batata-doce

== T24 UC-03: fita do dia 2026-10-12 da Ana (uma única leitura, ordenada por horaInicio)
   06:30-07:30 | Corrida leve       | CARDIO      | concluida=false
   18:00-19:00 | Peito e tríceps    | MUSCULACAO  | concluida=false
   19:00-19:15 | Alongamento final  | ALONGAMENTO | concluida=false

== T25 UC-05: histórico da conversa Ana-Bruno (visão com o remetente derivado)
   2026-10-08T21:42:00.000Z | Ana Beatriz Lima | Fala, Bruno! Bora treinar de manhã amanhã? | ENVIADA
   2026-10-08T21:45:00.000Z | Bruno Carvalho   | Bora! Chego às 7h na academia. | ENVIADA
   2026-10-08T21:46:00.000Z | Ana Beatriz Lima | Fechado. Levo a planilha de cargas. | ENVIADA
   2026-10-09T10:12:00.000Z | Bruno Carvalho   | Cheguei, te espero na entrada. | ENVIADA
   2026-10-09T10:13:00.000Z | Ana Beatriz Lima | Já estou subindo! | ENVIADA

== T26 planos de execução: índices usados pelas consultas dos casos de uso
   UC-02 login por e-mail          -> uq_usuarios_email
   UC-03 fita do dia               -> uq_agenda_usuario_dia
```

```
   UC-04 plano do usuário          -> uq_plano_usuario
   UC-05 lista de conversas        -> ix_conversas_participante
   UC-05 histórico                 -> ix_mensagens_historico
   UC-05 A1 busca por nome (ativos)-> ix_usuarios_ativos_nome
   UC-05 E2 fila de reenvio        -> ix_mensagens_pendentes

== T27 UC-06 / RN-03: exclusão definitiva em cascata (usuário de teste)
   ANTES  usuarios=7 agendas=12 planos=5 conversas=4 mensagens=12
   excluirConta -> usuarios removidos: 1
   DEPOIS usuarios=6 agendas=11 planos=4 conversas=3 mensagens=11
```

20.4. Script 03 — testes de integridade e de consultas

Executado com os erros esperados capturados por `try/catch` (função `tenta`) e impressos, sem interromper o script.

```
// ============================================================================
// GymGramps - Fase C-6 - Script 03: testes de integridade, de consultas e de índices
// Executar após 01_ddl.js e 02_carga.js. Os erros são ESPERADOS: cada um prova que uma regra foi barrada.
// Uso: mongosh "mongodb://localhost:27017/gymgramps" --file 03_testes.js
// ============================================================================
load('lib_gymgramps.js');

const rotulo = (n, txt) => print('\n== ' + n + ' ' + txt);
function tenta(fn) {                              // mostra o erro recebido (ou "ACEITO")
  try { fn(); print('   ACEITO (sem erro)'); }
  catch (e) { print('   ERRO ' + (e.code !== undefined ? e.code + ': ' : '') + String(e.message).replace(/\s+/g, ' ').slice(0, 150)); }
}
const idDe = (email) => bd.usuarios.findOne({ email }, { _id: 1 })._id;
const ana = idDe('ana.lima@example.com'), bruno = idDe('bruno.carvalho@example.com'),
      carla = idDe('carla.menezes@example.com'), diego = idDe('diego.ramos@example.com'),
      elisa = idDe('elisa.teixeira@example.com'), felipe = idDe('felipe.andrade@example.com');
const convAB = bd.conversas.findOne({ 'participantes.usuarioId': { $all: [ana, bruno] } })._id;
const base = (extra) => Object.assign({ email: 'novo@example.com', nome: 'Fulano', dataNascimento: dia('2000-01-01'),
  pesoCif: cifrar('70'), senhaHash: 'x', tema: 'CLARO', statusConta: 'ATIVA', dataCadastro: new Date() }, extra);

rotulo('T01', 'e-mail já cadastrado (uq_usuarios_email)');
tenta(() => bd.usuarios.insertOne(base({ email: 'ana.lima@example.com' })));
rotulo('T02', 'e-mail com formato inválido ($jsonSchema)');
tenta(() => bd.usuarios.insertOne(base({ email: 'sem-arroba' })));
rotulo('T03', 'e-mail com letras maiúsculas ($jsonSchema)');
tenta(() => bd.usuarios.insertOne(base({ email: 'Maiuscula@example.com' })));
rotulo('T04', 'tema fora do domínio (AZUL)');
tenta(() => bd.usuarios.insertOne(base({ tema: 'AZUL' })));
rotulo('T05', 'campo não previsto no esquema (additionalProperties: false)');
tenta(() => bd.usuarios.insertOne(base({ cartao: '1234' })));

rotulo('T06', 'atividade 08:30-09:30 sobreposta a 08:00-09:00 (mesmo usuário e dia)');
tenta(() => agendarAtividade(carla, dia('2026-10-12'), '08:30', '09:30', 'Conflito', 'OUTRO'));
rotulo('T07', 'horaFim anterior a horaInicio (validação $expr)');
tenta(() => bd.agendas_dia.insertOne({ usuarioId: carla, dia: dia('2026-10-14'), atividades: [
  { atividadeId: new ObjectId(), horaInicio: '10:00', horaFim: '09:00', titulo: 'Invertida', tipo: 'OUTRO', concluida: false } ] }));
rotulo('T08', 'horário com formato inválido (25:00)');
tenta(() => bd.agendas_dia.insertOne({ usuarioId: carla, dia: dia('2026-10-14'), atividades: [
  { atividadeId: new ObjectId(), horaInicio: '25:00', horaFim: '26:00', titulo: 'Fora', tipo: 'OUTRO', concluida: false } ] }));
rotulo('T09', 'segundo documento para o mesmo usuário e dia (uq_agenda_usuario_dia)');
tenta(() => bd.agendas_dia.insertOne({ usuarioId: ana, dia: dia('2026-10-12'), atividades: [] }));
rotulo('T10', 'atividade encostada (07:30 logo após 06:30-07:30) deve ser ACEITA');
tenta(() => agendarAtividade(ana, dia('2026-10-12'), '07:30', '08:00', 'Encostada', 'OUTRO'));
bd.agendas_dia.updateOne({ usuarioId: ana, dia: dia('2026-10-12') }, { $pull: { atividades: { titulo: 'Encostada' } } });  // desfaz

rotulo('T11', 'segundo plano alimentar para o mesmo usuário (uq_plano_usuario)');
tenta(() => bd.planos_alimentares.insertOne({ usuarioId: ana, dataInicio: dia('2026-10-09'), caloriasDiariasCif: cifrar('1'),
  proteinasGCif: cifrar('1'), carboidratosGCif: cifrar('1'), gordurasGCif: cifrar('1'),
  refeicoes: [ { refeicaoId: new ObjectId(), nome: 'X', horario: '10:00', caloriasCif: cifrar('1') } ] }));
rotulo('T12', 'plano sem nenhuma refeição (minItems: 1)');
tenta(() => bd.planos_alimentares.insertOne({ usuarioId: elisa, dataInicio: dia('2026-10-09'), caloriasDiariasCif: cifrar('1800'),
  proteinasGCif: cifrar('90'), carboidratosGCif: cifrar('200'), gordurasGCif: cifrar('50'), refeicoes: [] }));
const planoAna = bd.planos_alimentares.findOne({ usuarioId: ana })._id;
rotulo('T13', 'refeição com nome repetido no mesmo plano');
tenta(() => adicionarRefeicao(planoAna, 'Almoço', '13:00', 'Repetida', '100'));
rotulo('T14', 'remover a última refeição de um plano');
const planoTeste = bd.planos_alimentares.insertOne({ usuarioId: elisa, dataInicio: dia('2026-10-09'),
  caloriasDiariasCif: cifrar('1800'),
  proteinasGCif: cifrar('90'), carboidratosGCif: cifrar('200'), gordurasGCif: cifrar('50'),
  refeicoes: [ { refeicaoId: new ObjectId(), nome: 'Único', horario: '12:00', caloriasCif: cifrar('700') } ] }).insertedId;
tenta(() => removerRefeicao(planoTeste, 'Único'));
bd.planos_alimentares.deleteOne({ _id: planoTeste });

const agora = new Date();
rotulo('T15', 'conversa com apenas 1 participante (minItems: 2)');
tenta(() => bd.conversas.insertOne({ dataCriacao: agora, participantes: [ { usuarioId: diego, dataEntrada: agora } ] }));
rotulo('T16', 'conversa com 3 participantes (maxItems: 2)');
tenta(() => bd.conversas.insertOne({ dataCriacao: agora, participantes: [ { usuarioId: ana, dataEntrada: agora },
  { usuarioId: bruno, dataEntrada: agora }, { usuarioId: diego, dataEntrada: agora } ] }));
rotulo('T17', 'conversa com o mesmo usuário duas vezes (validação $expr)');
tenta(() => bd.conversas.insertOne({ dataCriacao: agora, participantes: [ { usuarioId: ana, dataEntrada: agora },
  { usuarioId: ana, dataEntrada: agora } ] }));
rotulo('T18', 'conta INATIVA não entra em nova conversa (RN-01)');
tenta(() => criarConversa(ana, felipe));
rotulo('T19', 'mensagem de quem não participa da conversa');
tenta(() => enviarMensagem(convAB, diego, 'Intruso'));
rotulo('T20', 'mensagem só com espaços ($jsonSchema)');
tenta(() => bd.mensagens.insertOne({ conversaId: convAB, remetenteId: ana, dataHoraEnvio: new Date(),
  conteudo: ' ', status: 'ENVIADA' }));
rotulo('T21', 'remetente com conta INATIVA não envia mensagem (RN-01)');
const convBD = bd.conversas.findOne({ 'participantes.usuarioId': { $all: [bruno, diego] } })._id;
bd.usuarios.updateOne({ _id: diego }, { $set: { statusConta: 'INATIVA' } });
tenta(() => enviarMensagem(convBD, diego, 'Oi'));
bd.usuarios.updateOne({ _id: diego }, { $set: { statusConta: 'ATIVA' } });  // desfaz

rotulo('T22', 'dado sensível armazenado cifrado (binData) e decifrado com a chave');
const ua = bd.usuarios.findOne({ _id: ana });
```

```
print('   ' + ua.nome + ' | tipo BSON: binData | bytes cifrados: ' + ua.pesoCif.length() +
      ' | peso decifrado: ' + decifrar(ua.pesoCif) + ' kg');
rotulo('T23', 'UC-04: plano alimentar e refeições decifrados (Ana)');
const pl = bd.planos_alimentares.findOne({ usuarioId: ana });
print('   objetivo=' + decifrar(pl.objetivoCif) + ' | kcal=' + decifrar(pl.caloriasDiariasCif) + ' | P/C/G=' +
      [pl.proteinasGCif, pl.carboidratosGCif, pl.gordurasGCif].map(decifrar).join('/'));
pl.refeicoes.sort((a, b) => a.horario.localeCompare(b.horario)).forEach(r =>
  print('    ' + r.horario + ' | ' + r.nome.padEnd(16) + ' | ' + decifrar(r.caloriasCif).padStart(4) +
        ' kcal | ' + decifrar(r.descricaoCif)));
rotulo('T24', 'UC-03: fita do dia 2026-10-12 da Ana (uma única leitura, ordenada por horaInicio)');
bd.agendas_dia.aggregate([ { $match: { usuarioId: ana, dia: dia('2026-10-12') } }, { $unwind: '$atividades' },
  { $sort: { 'atividades.horaInicio': 1 } },
  { $project: { _id: 0, ini: '$atividades.horaInicio', fim: '$atividades.horaFim', titulo: '$atividades.titulo',
                tipo: '$atividades.tipo', concluida: '$atividades.concluida' } } ]).forEach(r =>
  print('   ' + r.ini + '-' + r.fim + ' | ' + r.titulo.padEnd(18) + ' | ' + r.tipo.padEnd(11) + ' | concluida=' + r.concluida));
rotulo('T25', 'UC-05: histórico da conversa Ana-Bruno (visão com o remetente derivado)');
bd.vw_mensagem_historico.find({ conversaId: convAB }).sort({ dataHoraEnvio: 1 }).forEach(m =>
  print('   ' + m.dataHoraEnvio.toISOString() + ' | ' + m.remetente.padEnd(16) + ' | ' + m.conteudo + ' | ' + m.status));

rotulo('T26', 'planos de execução: índices usados pelas consultas dos casos de uso');
const indicesUsados = (p) => {                    // nomes dos índices citados no plano vencedor
  const r = []; JSON.stringify(p).replace(/"indexName":"([^"]+)"/g, (_, n) => r.push(n));
  return r.join(', ') || '(varredura de coleção)';
};
const plano_ = (cur) => cur.explain('queryPlanner').queryPlanner.winningPlan;
print('   UC-02 login por e-mail          -> ' +
      indicesUsados(plano_(bd.usuarios.find({ email: 'ana.lima@example.com' }))));
print('   UC-03 fita do dia               -> ' +
      indicesUsados(plano_(bd.agendas_dia.find({ usuarioId: ana, dia: dia('2026-10-12') }))));
print('   UC-04 plano do usuário          -> ' + indicesUsados(plano_(bd.planos_alimentares.find({ usuarioId: ana }))));
print('   UC-05 lista de conversas        -> ' +
      indicesUsados(plano_(bd.conversas.find({ 'participantes.usuarioId': ana }).sort({ ultimaMensagemEm: -1 }))));
print('   UC-05 histórico                 -> ' +
      indicesUsados(plano_(bd.mensagens.find({ conversaId: convAB }).sort({ dataHoraEnvio: 1 }))));
print('   UC-05 A1 busca por nome (ativos)-> ' +
      indicesUsados(plano_(bd.usuarios.find({ nome: 'Carla Menezes', statusConta: 'ATIVA' }))));
print('   UC-05 E2 fila de reenvio        -> ' +
      indicesUsados(plano_(bd.mensagens.find({ conversaId: convAB, remetenteId: ana, status: 'NAO_ENVIADA' }))));

rotulo('T27', 'UC-06 / RN-03: exclusão definitiva em cascata (usuário de teste)');
const n = (c) => bd[c].countDocuments({});
const contagem = (m) => print('   ' + m.padEnd(6) + ' usuarios=' + n('usuarios') + ' agendas=' + n('agendas_dia') +
  ' planos=' + n('planos_alimentares') + ' conversas=' + n('conversas') + ' mensagens=' + n('mensagens'));
const g = bd.usuarios.insertOne(base({ email: 'gustavo.teste@example.com', nome: 'Gustavo Teste' })).insertedId;
agendarAtividade(g, dia('2026-10-12'), '10:00', '11:00', 'Teste', 'OUTRO');
bd.planos_alimentares.insertOne({ usuarioId: g, dataInicio: dia('2026-10-12'), caloriasDiariasCif: cifrar('2000'),
  proteinasGCif: cifrar('100'), carboidratosGCif: cifrar('250'), gordurasGCif: cifrar('60'),
  refeicoes: [ { refeicaoId: new ObjectId(), nome: 'Almoço', horario: '12:00', caloriasCif: cifrar('700') } ] });
const cg = criarConversa(g, elisa); enviarMensagem(cg, g, 'olá');
contagem('ANTES');
print('   excluirConta -> usuarios removidos: ' + excluirConta(g));
contagem('DEPOIS');
```

Resumo. Todas as regras da tabela da Seção 17.3 foram verificadas com violações reais e o banco permaneceu consistente depois delas; as regras garantidas pelo SGBD (validador e índices) foram barradas diretamente, e as garantidas pela aplicação, pelas rotinas de `lib_gymgramps.js`. Os índices secundários são usados pelas consultas dos casos de uso, e a exclusão de conta removeu em cascata todos os documentos do usuário de teste.

Apêndice A — Script DDL (PostgreSQL / Supabase)

Script de criação do esquema da Figura 3. Foi executado em PostgreSQL 16 e testado com casos de violação de cada restrição (e-mail duplicado, sobreposição de atividades, segundo plano, plano sem refeição, conversa com 1 ou 3 participantes, remetente não participante, mensagem vazia, conta INATIVA no chat) e com a exclusão em cascata de uma conta. A política RLS ao final depende da função auth.uid(), disponível apenas no Supabase (no teste, foi usada uma função substituta).

```
-- GymGramps — Esquema lógico relacional (PostgreSQL 15+ / Supabase)
CREATE EXTENSION IF NOT EXISTS btree_gist;   -- necessário para o EXCLUDE
```

```
CREATE TYPE tema_t            AS ENUM ('CLARO', 'ESCURO');
CREATE TYPE status_conta_t    AS ENUM ('ATIVA', 'INATIVA');
CREATE TYPE tipo_atividade_t  AS ENUM ('MUSCULACAO', 'CARDIO', 'ALONGAMENTO', 'OUTRO');
CREATE TYPE status_mensagem_t AS ENUM ('ENVIADA', 'NAO_ENVIADA');
```

```
CREATE TABLE usuario (
  id_usuario      UUID           PRIMARY KEY DEFAULT gen_random_uuid(),
  email           VARCHAR(255)   NOT NULL,
  nome            VARCHAR(120)   NOT NULL,
  data_nascimento DATE           NOT NULL,
  peso            NUMERIC(5,2)   NOT NULL CHECK (peso > 0),        -- sensível (RN-02)
  senha_hash      VARCHAR(255)   NOT NULL,
  tema            tema_t         NOT NULL DEFAULT 'CLARO',
  status_conta    status_conta_t NOT NULL DEFAULT 'ATIVA',
  data_cadastro   TIMESTAMPTZ    NOT NULL DEFAULT now(),
  CONSTRAINT uq_usuario_email UNIQUE (email)
);
```

```
CREATE TABLE atividade (
  id_atividade UUID             PRIMARY KEY DEFAULT gen_random_uuid(),
  id_usuario   UUID             NOT NULL REFERENCES usuario ON DELETE CASCADE,
  data         DATE             NOT NULL,
  hora_inicio  TIME             NOT NULL,
  hora_fim     TIME             NOT NULL,
  titulo       VARCHAR(100)     NOT NULL,
  tipo         tipo_atividade_t NOT NULL,
  concluida    BOOLEAN          NOT NULL DEFAULT FALSE,
  CONSTRAINT uq_atividade UNIQUE (id_usuario, data, hora_inicio),
  CONSTRAINT ck_atividade_horario CHECK (hora_fim > hora_inicio),
  CONSTRAINT ex_atividade_sobreposicao EXCLUDE USING gist (
    id_usuario WITH =,
    tsrange(data + hora_inicio, data + hora_fim) WITH &&)
);
```

```
CREATE TABLE plano_alimentar (
  id_plano         UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
  id_usuario       UUID         NOT NULL REFERENCES usuario ON DELETE CASCADE,
  data_inicio      DATE         NOT NULL,
  objetivo         VARCHAR(200),                                        -- sensível
  calorias_diarias INTEGER      NOT NULL CHECK (calorias_diarias >= 0),  -- sensível
  proteinas_g      NUMERIC(6,1) NOT NULL CHECK (proteinas_g >= 0),       -- sensível
  carboidratos_g   NUMERIC(6,1) NOT NULL CHECK (carboidratos_g >= 0),    -- sensível
  gorduras_g       NUMERIC(6,1) NOT NULL CHECK (gorduras_g >= 0),        -- sensível
  CONSTRAINT uq_plano_usuario UNIQUE (id_usuario)    -- no máximo 1 plano (0..1)
);
```

```
CREATE TABLE refeicao (
  id_refeicao UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
  id_plano    UUID        NOT NULL REFERENCES plano_alimentar ON DELETE CASCADE,
  nome        VARCHAR(60) NOT NULL,
  horario     TIME        NOT NULL,
  descricao   VARCHAR(300),                                -- sensível
  calorias    INTEGER     NOT NULL CHECK (calorias >= 0),  -- sensível
  CONSTRAINT uq_refeicao UNIQUE (id_plano, nome)
);
```

```
CREATE TABLE conversa (
  id_conversa  UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
  data_criacao TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```
-- Classe associativa ParticipanteConversa (relacionamento Usuario–Conversa)
CREATE TABLE participante_conversa (
  id_conversa  UUID        NOT NULL REFERENCES conversa ON DELETE CASCADE,
  id_usuario   UUID        NOT NULL REFERENCES usuario  ON DELETE CASCADE,
  data_entrada TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (id_conversa, id_usuario)
);
CREATE INDEX ix_participante_usuario ON participante_conversa (id_usuario);
```

```
-- Mensagem se relaciona com a classe associativa (e não direto com Usuario/Conversa)
CREATE TABLE mensagem (
  id_mensagem     UUID              PRIMARY KEY DEFAULT gen_random_uuid(),
  id_conversa     UUID              NOT NULL,
  id_remetente    UUID              NOT NULL,
  data_hora_envio TIMESTAMPTZ       NOT NULL DEFAULT now(),
  conteudo        TEXT              NOT NULL CHECK (length(btrim(conteudo)) > 0),
  status          status_mensagem_t NOT NULL DEFAULT 'ENVIADA',
  CONSTRAINT uq_mensagem UNIQUE (id_conversa, id_remetente, data_hora_envio),
  CONSTRAINT fk_mensagem_participante FOREIGN KEY (id_conversa, id_remetente)
    REFERENCES participante_conversa (id_conversa, id_usuario) ON DELETE CASCADE
);
CREATE INDEX ix_mensagem_historico ON mensagem (id_conversa, data_hora_envio);
```

```
-- Participante: no máximo 2 por conversa e somente contas ATIVAS (RN-01)
CREATE FUNCTION fn_valida_participante() RETURNS trigger AS $$
BEGIN
  IF (SELECT count(*) FROM participante_conversa
       WHERE id_conversa = NEW.id_conversa) >= 2 THEN
    RAISE EXCEPTION 'A conversa já possui 2 participantes';
  END IF;
  IF (SELECT status_conta FROM usuario
       WHERE id_usuario = NEW.id_usuario) <> 'ATIVA' THEN
    RAISE EXCEPTION 'Somente contas ATIVAS participam de conversas (RN-01)';
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER tg_valida_participante BEFORE INSERT ON participante_conversa
  FOR EACH ROW EXECUTE FUNCTION fn_valida_participante();
```

```
-- Conversa: exatamente 2 participantes, verificado no COMMIT
CREATE FUNCTION fn_conversa_dois_participantes() RETURNS trigger AS $$
BEGIN
  IF EXISTS (SELECT 1 FROM conversa WHERE id_conversa = NEW.id_conversa)
     AND (SELECT count(*) FROM participante_conversa
           WHERE id_conversa = NEW.id_conversa) <> 2 THEN
    RAISE EXCEPTION 'Uma conversa deve ter exatamente 2 participantes';
  END IF;
  RETURN NULL;
END $$ LANGUAGE plpgsql;
```

```
CREATE CONSTRAINT TRIGGER tg_conversa_dois_participantes
  AFTER INSERT ON conversa DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION fn_conversa_dois_participantes();
```

```
-- Plano alimentar: ao menos 1 refeição (multiplicidade 1..*), verificado no COMMIT
CREATE FUNCTION fn_plano_com_refeicao() RETURNS trigger AS $$
BEGIN
  IF EXISTS (SELECT 1 FROM plano_alimentar WHERE id_plano = NEW.id_plano)
     AND NOT EXISTS (SELECT 1 FROM refeicao WHERE id_plano = NEW.id_plano) THEN
    RAISE EXCEPTION 'Um plano alimentar deve ter ao menos 1 refeição';
  END IF;
  RETURN NULL;
END $$ LANGUAGE plpgsql;
```

```
CREATE CONSTRAINT TRIGGER tg_plano_com_refeicao
  AFTER INSERT ON plano_alimentar DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION fn_plano_com_refeicao();
```

```
-- Exclusão definitiva de conta (RN-03 / UC-06), em uma única transação
CREATE FUNCTION excluir_conta(p_id_usuario UUID) RETURNS void AS $$
BEGIN
  -- conversas do usuário (cascata: participante_conversa -> mensagem)
  DELETE FROM conversa
   WHERE id_conversa IN (SELECT id_conversa FROM participante_conversa
                         WHERE id_usuario = p_id_usuario);
  -- usuário (cascata: atividade; plano_alimentar -> refeicao)
  DELETE FROM usuario WHERE id_usuario = p_id_usuario;
END $$ LANGUAGE plpgsql;
```

```
-- Controle de acesso restrito ao titular (RN-02) — somente no Supabase,
-- onde auth.uid() retorna o id do usuário autenticado. Exemplo:
ALTER TABLE plano_alimentar ENABLE ROW LEVEL SECURITY;
CREATE POLICY pol_plano_titular ON plano_alimentar
  USING (id_usuario = auth.uid()) WITH CHECK (id_usuario = auth.uid());
```

Referências

BEZERRA, E. *Princípios de Análise e Projeto de Sistemas com UML*. 3ª ed. Rio de Janeiro: Elsevier, 2015. Cap. 7 (Modelagem de interações), cap. 9 (Modelagem de estados), cap. 10 (Modelagem de atividades), cap. 11 (Arquitetura do sistema) e cap. 12 (Mapeamento de objetos para o modelo relacional).

MARTIN, R. C. *The Clean Architecture*. The Clean Code Blog, 2012. Disponível em: https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html.

TERRALAB/DECOM-UFOP. *Uma Breve Introdução à Arquitetura Limpa com Node.js*, 2021. Disponível em: http://www2.decom.ufop.br/terralab/uma-breve-introducao-a-arquitetura-limpa-com-node-js/.

JACOBSON, I. et al. *Use-Case 2.0*, 2011/2016. JACOBSON, I. et al. *The Essentials of Modern Software Engineering*, 2019.

CEDERJ. Disciplina Banco de Dados, Curso de Sistemas de Informação. Aula 17 — Mapeamento ER; Aula 18 — Mapeamento de Generalização; Aula 27 — Normalização (1FN e 2FN); Aula 28 — Normalização (3FN).

SBBD 2021. Minicurso Projeto de Banco de Dados NoSQL, 1ª parte. Disponível em: https://youtu.be/DrXOaQkWPfY.

GOODDATA. What is a data model? Disponível em: https://www.gooddata.com/blog/what-is-a-data-model/.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL Documentation* — Constraints (Exclusion Constraints) e Constraint Triggers.

SUPABASE. *Supabase Docs* — Row Level Security e Auth.

MONGODB, INC. MongoDB Manual — Schema Validation (validação por *$jsonSchema*). Disponível em: https://www.mongodb.com/docs/manual/core/schema-validation/.

MONGODB, INC. MongoDB Manual — Data Modeling (dados embutidos e referências entre documentos). Disponível em: https://www.mongodb.com/docs/manual/core/data-model-design/.

MONGODB, INC. MongoDB Manual — Indexes. Disponível em: https://www.mongodb.com/docs/manual/indexes/.

ALGER, K. W.; COUPAL, D. Building with Patterns: The Bucket Pattern. MongoDB Blog, 2019. Disponível em: https://www.mongodb.com/blog/post/building-with-patterns-the-bucket-pattern.

Demais links de referência do enunciado da Fase C-6: https://youtu.be/nfFjXIlRbok e https://youtu.be/RdvpUhlrkwk (o primeiro link do enunciado, https://youtu.be/DrXOaQkWPfY, já consta acima).